

VisionDoc AI

From Scratch to Advanced: A Complete Study & Interview Guide

A complete, from-scratch to advanced guide to the project and every tool it uses - written for deep understanding and technical interview preparation.

Project: **VisionDoc AI** - Fine-tuning a Vision-Language Model (Qwen2.5-VL) with LoRA for document intelligence.

Repository: github.com/Swapnil-byte-798/visiondoc-ai

Table of Contents

1. Introduction: What VisionDoc AI Is and Why It Matters	7
1.1 The three pitches: 30 seconds, 2 minutes, deep	7
1.2 The real-world problem: document intelligence at scale	8
1.3 What the system does, end to end	9
1.4 A single request, step by step	10
1.5 Why this project demonstrates real deep-learning engineering	10
1.6 Roadmap of the rest of this guide	11
2. Machine Learning & Deep Learning From Absolute Zero	12
2.1 What is a "model," really?	12
2.2 Features and labels	12
2.3 Supervised vs. unsupervised learning	13
2.4 Training vs. inference	13
2.5 Parameters, weights, and how a model "holds" knowledge	13
2.6 The loss function - a score for "how wrong"	14
2.7 Gradient descent - how the dials get turned	14
2.8 Backpropagation - computing the gradient efficiently	15
2.9 The training vocabulary: epochs, batches, learning rate	15
2.10 Overfitting, underfitting, and generalization	16
2.11 Train / validation / test splits	17
2.12 Why deep learning needs a GPU	17
2.13 Putting the loop together	18
3. Neural Networks, the Transformer, and Attention	19
3.1 From a neuron to a layer	19
3.2 From a layer to a deep network, and how it learns	19
3.3 Why sequences (text) need special handling	20
3.4 Tokenization: turning text into tokens	20
3.5 The old approach: RNNs, and why they struggled	21
3.6 Self-attention: the core idea	22
3.7 Multi-head attention	23
3.8 Positional information	23
3.9 Putting it together: the Transformer block, and encoder vs decoder	23
3.10 Chapter summary	25

4. Large Language Models and How Text Is Generated	26
4.1 What an LLM actually is	26
4.2 The pretraining objective: next-token prediction	26
4.3 Autoregressive generation: writing one token at a time	27
4.4 From logits to probabilities: softmax	27
4.5 Decoding strategies: choosing the next token	28
4.6 Context window	29
4.7 Prompts, instruction tuning, and chat templates	30
4.8 Confidence from token log-probabilities	31
4.9 Hallucination	32
5. Vision-Language Models and Qwen2.5-VL	33
5.1 What is a Vision-Language Model, and why "multimodal"?	33
5.2 How a computer turns an image into "tokens"	33
5.3 Fusing vision and language: one sequence to rule them all	34
5.4 Qwen2.5-VL specifics (at a teaching level)	34
5.5 The OCR-free contrast: Donut	36
5.6 Putting the pieces together	37
6. Fine-Tuning, Transfer Learning, LoRA and PEFT	39
6.1 Three ways to make a model do what you want	39
6.2 Transfer learning: standing on the model's shoulders	39
6.3 Why full fine-tuning of a 3B+ multimodal model is expensive	40
6.4 PEFT: the general idea	40
6.5 LoRA from scratch, with intuition	40
6.6 The LoRA hyperparameters - and where they live in the repo	41
6.7 How the repo actually applies LoRA	43
6.8 Adapter files, saving, loading, merging	43
6.9 QLoRA: LoRA plus 4-bit quantization	44
6.10 LoRA vs full fine-tuning: when to use which	44
6.11 Mental checklist	45
7. Numerical Precision, Quantization, and QLoRA	46
7.1 What a floating-point number actually is	46
7.2 Why bf16 needs a newer GPU, and why the T4 uses fp16	47
7.3 Mixed-precision training	47

7.4 Quantization: from 16 bits down to 4	48
7.5 QLoRA: 4-bit base + LoRA adapters	48
7.6 bitsandbytes, and the two different quantization jobs	49
7.7 Two more memory levers: gradient checkpointing and accumulation	50
7.8 max_pixels: the vision-specific memory lever	50
7.9 A concrete memory-math example	51
7.10 Putting the T4 config together	52
8. Document AI, Datasets, and Evaluation Metrics	53
8.1 What is Document AI?	53
8.2 The core document-AI tasks	54
8.3 The datasets	54
8.4 Evaluation metrics from scratch	55
8.5 Why string metrics are unfair to JSON - and field-level F1	58
8.6 Systems metrics - latency, throughput, VRAM	59
9. Core Tools I - Python and PyTorch	61
9.1 Python - the lingua franca of machine learning	61
9.2 PyTorch - the deep-learning engine	63
9.3 Putting it together	67
10. Core Tools II - The Hugging Face Stack	69
10.1 Transformers - loading and running the model	69
10.2 Datasets - feeding the model	71
10.3 Accelerate - the device/distribution abstraction	73
10.4 PEFT - the LoRA library	73
10.5 bitsandbytes - 4-bit loading (QLoRA)	74
10.6 How the pieces click together	75
11. Core Tools III - Serving, Ops, and Data Libraries	77
11.1 FastAPI - the HTTP inference server	77
11.2 Streamlit - the dashboard	78
11.3 Docker & docker-compose - reproducible packaging	79
11.4 Weights & Biases - experiment tracking	80
11.5 The numeric & image libraries - NumPy, Pandas, Pillow, OpenCV	81
11.6 Git & GitHub - version control	81
11.7 Google Colab & GPUs - where training runs	82

11.8 Evaluation libraries	82
Summary	83
12. Inside the Codebase I - Config, Utils, Preprocessing, Models	84
12.1 configs/config.py - one typed, immutable config to rule them all	84
12.2 utils/ - the shared toolbelt	86
12.3 preprocessing/schema.py - the DocSample data contract	88
12.4 preprocessing/datasets.py - normalizing four corpora into DocSample s	88
12.5 transforms.py - document-safe augmentation	89
12.6 cache.py and dataset.py	90
12.7 models/base.py - the VisionDocModel contract + registry	90
12.8 models/qwen_vl.py - the default causal-VLM backbone	92
12.9 models/donut.py - the seq2seq alternative (proving swappability)	93
12.10 The through-line	93
13. Inside the Codebase II - Training, Evaluation, Inference, Serving, Deployment	95
13.1 Training - training/trainer.py and training/train.py	95
13.2 Evaluation - measuring quality and cost	98
13.3 Inference - predictor.py , extract.py , batch.py	100
13.4 Research - research/compare.py	102
13.5 Visualization - visualization/plots.py	103
13.6 Serving - api/main.py and app/streamlit_app.py	103
13.7 Running everything, and reading the results	105
14. Interview Prep I - Conceptual Question Bank (Zero to Advanced)	108
14.1 ML / Deep Learning Fundamentals	108
14.2 Neural Networks & Training Mechanics	109
14.3 Transformers & Attention	110
14.4 LLMs & Decoding	111
14.5 Vision-Language Models (VLMs)	112
14.6 Fine-Tuning, LoRA & PEFT	114
14.7 Quantization, Precision & Memory	115
14.8 Document AI & Evaluation Metrics	116
14.9 PyTorch & Practical Engineering	117
14.10 Training Dynamics: Overfitting, Schedules, Early Stopping	118
14.11 Rapid-Fire Round (Say These in One Breath)	119

15. Interview Prep II - Project Deep-Dive, System Design, War Stories, Glossary 120

15.1 Project deep-dive Q&A;	120
15.2 "Walk me through your project" scripts	122
15.3 System-design questions	123
15.4 War stories (behavioral prompts)	123
15.5 One-page cheat sheet	125
15.6 Comprehensive glossary	125

1. Introduction: What VisionDoc AI Is and Why It Matters

VisionDoc AI is a system that reads documents the way a human clerk does - it looks at a scanned invoice, receipt, form, or ID card, and answers plain-English questions about it ("What is the total amount due?"), pulls out structured fields (merchant, date, total), tells you how confident it is, and even highlights the exact spot on the page where the answer came from. Under the hood it takes an open-source **Vision-Language Model** and teaches it to specialize in documents using a technique called **LoRA fine-tuning**. This chapter explains what all of that means from zero, why it is valuable, and how a single request flows through the system. Later chapters go deep on each piece; this one gives you the map and the pitch.

Let us define the two load-bearing terms right away, because the whole project hangs on them.

- **Vision-Language Model (VLM):** a neural network - a large pattern-recognizing program trained on huge amounts of data - that takes in *both* an image and text and produces text. Think of a normal chatbot that can also see. You hand it a picture of a receipt plus the words "What's the total?" and it replies "\$42.10". VisionDoc AI uses **Qwen2.5-VL** (a 3-billion-parameter open-source VLM from Alibaba) as its default "brain," with **Donut** as a swappable alternative. "Open-source" here means the model's weights are downloadable and run on your own machine - nothing is sent to a paid outside service.
- **Fine-tuning:** taking a model that already knows a lot in general and giving it extra, focused training on your specific task so it gets much better at it. Like hiring a smart generalist and then training them for two weeks on your company's paperwork.

Interview Q: In one sentence, what is VisionDoc AI?

A: It is an end-to-end document-intelligence system that LoRA-fine-tunes an open-source Vision-Language Model (Qwen2.5-VL) to do visual question answering and structured field extraction over documents, complete with confidence scoring, region highlighting, a full evaluation harness, and both a REST API and a dashboard - real model training and inference, not a wrapper around someone else's API.

1.1 The three pitches: 30 seconds, 2 minutes, deep

You will be asked "tell me about your project" in almost every interview. Have three versions ready and pick the length to match the room.

The 30-second pitch. "VisionDoc AI turns document images into answers. You upload an invoice or receipt, ask a question in natural language, and get back the answer, a confidence score, the structured fields, and a highlight showing where on the page the answer lives. I built it by fine-tuning an open-source 3-billion-parameter Vision-Language Model with LoRA, so it runs on a single consumer GPU while beating the base model on document tasks. It ships with a FastAPI service, a Streamlit dashboard, and a full evaluation pipeline."

The 2-minute pitch. Same opening, then add the engineering spine:

- The base model is **frozen** (its weights never change) and I train only tiny **LoRA adapter** matrices - under 1% of the parameters. That is what makes real fine-tuning of a multi-billion-parameter model affordable.

- The codebase is **backbone- and dataset-agnostic**: swapping Qwen2.5-VL for Donut, or DocVQA for CORD, is a one-line change in a YAML config file - no code edits - because everything talks through one model interface (`VisionDocModel`) and one sample schema (`DocSample`).
- It evaluates itself honestly with the standard metrics (Exact Match, ANLS, F1, BLEU, ROUGE) and profiles latency and GPU memory, then produces a **base-vs-fine-tuned comparison** so you can prove the training actually helped.
- It runs everywhere: CUDA GPUs for real training, Apple Silicon (MPS) or CPU for the demo and tests, with automatic device detection.

The deep pitch. This is the walk through the architecture: the data layer normalizes four public datasets into one uniform format; the model layer wraps swappable backbones behind a registry; training uses the HuggingFace Trainer with mixed precision, gradient accumulation, gradient checkpointing, cosine learning-rate schedule, early stopping, and Weights & Biases logging; inference is a single `DocumentPredictor` facade that the API, app, and batch jobs all share; evaluation and research produce HTML reports and comparison charts. Each of those is a chapter in this guide - see Ch. 4 (data), Ch. 5-6 (models and LoRA), Ch. 7 (training), Ch. 8 (evaluation), Ch. 9 (inference), Ch. 10 (serving).

Key insight: Interviewers care less about *what* the app does and more about *why the engineering choices were made*. The strongest thread to pull in every pitch is "frozen backbone + small trainable adapters" - it shows you understand the economics of modern ML, not just the API surface.

1.2 The real-world problem: document intelligence at scale

Businesses drown in semi-structured documents: invoices, purchase orders, receipts for expense reports, tax and government forms, bank statements, shipping manifests, insurance claims, and identity documents (passports, driver's licenses). A mid-size company can receive tens of thousands of these a month. Each one has the *same kind* of information (a total, a date, a vendor, line items) but in a *different layout* every time - every vendor's invoice looks different, every country's ID has its own template, scans are skewed, photographed under bad lighting, or split across multiple PDF pages.

Historically this was handled by:

- 1 **Humans typing it in** - accurate but slow, expensive, and mind-numbing.
- 2 **Template/rule systems** - you draw boxes on a known layout and read text from fixed positions. These break the instant a vendor changes their template, and you need a new template per document type.
- 3 **Plain OCR (Optical Character Recognition - software that converts pixels of text into machine-readable characters)**. OCR tells you *what characters are on the page* but not *what they mean* - it cannot tell you which of the five numbers on a receipt is the total.

Document intelligence is the modern answer: a model that *understands* the document - its text, its layout, and the visual relationship between labels and values - so it can answer questions and extract fields on layouts it has never seen before. That "never seen before" generalization is exactly what a fine-tuned VLM buys you and what templates cannot.

Interview Q: Why use a Vision-Language Model instead of OCR plus regular expressions?

A: OCR only recovers characters; it has no notion of meaning or layout, so mapping text to fields still requires brittle, per-template rules. A VLM jointly reasons over the pixels and the language, so it generalizes to unseen layouts and answers free-form questions. In this project OCR is not thrown away - it is used as a *fallback for region highlighting* (grounding the answer to a location), while the VLM does the actual understanding.

Who uses this and why it is valuable. Accounts-payable and expense automation (auto-reading invoices/receipts), banking and lending (KYC - "Know Your Customer" identity checks - and statement processing), insurance (claims intake), logistics (bills of lading), healthcare (forms), and government (tax filings). The value is straightforward: fewer human hours, faster turnaround, and - crucially - **confidence scores** that let you auto-approve high-confidence extractions and route only the uncertain ones to a human. That "human-in-the-loop on the low-confidence tail" pattern is why calibrated confidence (Ch. 9) is a first-class feature here, not an afterthought.

1.3 What the system does, end to end

VisionDoc AI is not just a model file - it is a full pipeline. Its capabilities, and where each lives in the repo, are:

Capability	Plain-English meaning	Where in the code
Accept images and PDFs	Upload a PNG/JPG or a multi-page PDF (auto-rasterized to images)	<code>inference/</code> , <code>app/</code> , <code>api/</code>
Question answering (QA)	Ask any natural-language question about the doc	<code>POST /ask</code> , <code>dashboard</code>
Structured extraction	Pull named fields into JSON (merchant, date, total...)	<code>POST /extract</code> , <code>inference/extract.py</code>
Confidence scores	A 0-1 number for how sure the model is	<code>models/base.py</code> (token log-probabilities)
Region highlighting	Draw a box on the page where the answer came from	OCR grounding, <code>utils/ocr.py</code>
Base vs fine-tuned comparison	Prove the fine-tuning helped, on the same test set	<code>research/compare.py</code>
Full evaluation	EM, ANLS, F1, BLEU, ROUGE, latency, VRAM	<code>evaluation/</code>

A few of these terms, defined now and expanded in later chapters:

- **Confidence score:** the model generates an answer one token (word-piece) at a time, and for each token it produces a probability. Combine those probabilities (the config calls this method `seq_prob`) and you get a single number, e.g. `0.92` , meaning "very sure." Low numbers flag answers a human should double-check.
- **Region highlighting / grounding:** after the model says the answer is "\$1,240.00", the system uses OCR to find where those characters physically sit on the page and draws a rectangle there - so a user can verify at a glance.
- **QA vs extraction:** QA is open-ended ("what is the invoice number?"); extraction asks for a fixed set of fields at once and returns clean JSON. Both are powered by the same underlying model, just prompted differently.

1.4 A single request, step by step

Here is what actually happens when you upload a receipt and ask "What is the total?" - the plain-English version of the `/ask` code path in `api/main.py` and `inference/predictor.py`.

- 1 **Upload.** You send the image (as base64 text in JSON, or as a file) plus your question to the API or dashboard.
- 2 **Decode & validate.** The service decodes the image to a PIL image object. Bad image or empty question -> an immediate `400` error (client's fault), never a crash.
- 3 **Get the model (lazily).** On the *first* request, the service builds a `DocumentPredictor`, which loads the VLM weights and, if configured, applies the LoRA adapter. This is deliberately deferred so the server boots instantly and `/health` works with no GPU. Subsequent requests reuse the already-loaded model (a thread-safe singleton).
- 4 **Build the prompt.** The image and your question are formatted into the exact prompt template the model expects.
- 5 **Generate.** The model produces the answer token by token (up to `max_new_tokens`, 128 by default), recording each token's probability along the way.
- 6 **Score confidence.** Those probabilities are combined into one confidence value.
- 7 **Ground the answer (optional).** If highlighting is on, OCR locates the answer text on the page and computes bounding-box regions; a highlighted copy of the image is returned as base64.
- 8 **Respond.** You get back a JSON object: the answer string, the confidence, the latency in milliseconds, the regions, and the highlighted image.

```
// POST /ask response (shape)
{
  "answer": "$42.10",
  "confidence": 0.92,
  "latency_ms": 380.5,
  "regions": [[112.0, 540.0, 210.0, 566.0]],
  "highlighted_image_base64": "iVBORw0KGGo..."
}
```

Gotcha: The model does **not** load when the process starts - it loads on the first inference request. This is intentional: containers and orchestrators need a fast, weight-free `/health` probe, and importing the API for tests must not drag in PyTorch. If you profile "cold start" you will see the first `/ask` is slow and every one after is fast. Confusing this for a bug is a common misread.

1.5 Why this project demonstrates real deep-learning engineering

A lot of "AI projects" are a thin call to a hosted API - the actual intelligence lives on someone else's servers and the author only wrote glue. VisionDoc AI is deliberately the opposite, and this is the point most worth making in an interview.

- **It loads and runs the model locally.** The API description states it plainly: "All inference runs on the locally-loaded model - no hosted LLM APIs are called." You control the weights, the device, and the latency.
- **It actually fine-tunes.** There is a real training loop (`training/train.py`) using the HuggingFace Trainer with LoRA, mixed precision, gradient accumulation, gradient checkpointing, early stopping, and W&B logging. Only the answer tokens count toward the loss (the prompt is masked) - a detail that shows genuine understanding of how these models learn.
- **It measures itself.** A full evaluation suite with the field-standard metrics, plus a base-vs-fine-tuned comparison that produces a CSV, a markdown table, and a bar chart. You can *prove* the training worked instead of hand-waving.

- **It is engineered like production software.** One typed config (`configs/config.py`) that rejects unknown keys loudly; one model interface behind a registry so backbones are swappable; one sample schema so datasets are swappable; centralized device resolution; a pytest suite that runs without a GPU; Docker packaging; and a Streamlit dashboard. This is the difference between a notebook and a system.

Interview Q: How is this different from just calling GPT-4 with vision?

A: Calling a hosted multimodal API means you don't own the model, can't fine-tune its weights, pay per call, and send your documents to a third party - a non-starter for regulated data like IDs or bank statements. VisionDoc AI runs an open-source VLM on your own hardware and LoRA-fine-tunes it on your documents, so you get data privacy, no per-request cost, controllable latency, and a model specialized to your layouts. Demonstrating that full loop - load, train, evaluate, serve - is the whole point.

Key insight: The single most impressive line of the project is that fine-tuning a 3B-parameter model trains **under 1% of the parameters** (roughly 9 million trainable weights via LoRA) while the ~2.6-billion-parameter backbone stays frozen - which is why it fits on one consumer GPU. If you remember one number, remember that ratio.

1.6 Roadmap of the rest of this guide

This guide builds from absolute fundamentals to advanced topics. Here is where each theme lives so you can navigate:

Chapters	Theme
2-3	ML and deep-learning foundations; how Vision-Language Models and Transformers work
4	The data layer: DocVQA, CORD, FUNSD, SROIE; normalizing to <code>DocSample</code> ; doc-safe augmentation and caching
5-6	The model layer and the swappable-backbone design; LoRA/PEFT and QLoRA in depth
7	The training loop: HuggingFace Trainer, mixed precision, gradient accumulation/checkpointing, early stopping, W&B
8	Evaluation: Exact Match, ANLS, F1, BLEU, ROUGE, latency and VRAM profiling, HTML reports
9	Inference: the <code>DocumentPredictor</code> facade, confidence scoring, region highlighting, PDF handling, batch
10	Serving and packaging: FastAPI, Streamlit, Docker, configuration
11+	Research comparison, visualization, quantization/ONNX, and interview deep-dives

By the end you should be able to explain not just what each piece does, but *why it was built that way* - which is exactly what separates someone who used a model from someone who can engineer one. Continue to Ch. 2 for the ML foundations that make the rest click.

2. Machine Learning & Deep Learning From Absolute Zero

This chapter builds the mental model everything else rests on. If you can already explain gradient descent to a friend, skim it. If you cannot, read it slowly - every later chapter (LoRA in Ch. 6, training in Ch. 8, evaluation in Ch. 9) assumes these ideas.

2.1 What is a "model," really?

Normal programming means **you** write the rules. To detect a total on an invoice you might write: "find the word 'Total', grab the number to its right." That breaks the moment a vendor writes "Amount Due" or puts the number on the next line. There are thousands of layouts; you cannot hand-write a rule for each.

Machine learning (ML) flips this around. Instead of writing rules, you show the computer many examples of *inputs paired with correct answers*, and a **learning algorithm** figures out the rules automatically. The frozen result of that learning - the set of numbers that encode the discovered rules - is called a **model**.

Key insight: A model is just a mathematical function $f(\text{input}) \rightarrow \text{output}$ whose behavior is controlled by a huge pile of adjustable numbers. "Training" is the process of nudging those numbers until the function gives good answers. "Using" the model is just plugging an input in and reading the output.

In VisionDoc AI, the input is an image of a document plus a question ("What is the invoice total?"), and the output is text ("\$1,240.00"). The model is `Qwen/Qwen2.5-VL-3B-Instruct` - a **Vision-Language Model (VLM)**, meaning a model that can look at pixels *and* read/write text in the same system (covered in depth in Ch. 5).

Interview Q: In one sentence, how does machine learning differ from traditional programming?
A: In traditional programming a human writes the rules and the computer applies them; in machine learning the human supplies examples of inputs and correct outputs, and the computer *learns* the rules by adjusting internal parameters to fit those examples.

2.2 Features and labels

Two words you will hear constantly:

- A **feature** is a piece of the input - a signal the model can look at. For a document that could be the raw pixels, the text characters, their positions on the page, font size, and so on.
- A **label** (also called the **target** or **ground truth**) is the correct answer you want the model to produce for that input.

A single **training example** is one `(features, label)` pair. A **dataset** is a big collection of them. VisionDoc's default dataset is **DocVQA** (`lmms-lab/DocVQA`), where each example is a document image + a question (features) and the human-written answer (label).

Term	Plain meaning	VisionDoc example
Feature / input	What the model sees	Document image + the question text

Term	Plain meaning	VisionDoc example
Label / target	The correct answer	"\$1,240.00"
Example	One input-label pair	(invoice + "total?", "\$1,240.00")
Dataset	Many examples	DocVQA

Gotcha: "Ground truth" does not mean *absolute* truth - it means *the answer we are treating as correct for training*. If your labels are wrong or inconsistent, the model faithfully learns the wrong thing. Garbage in, garbage out.

2.3 Supervised vs. unsupervised learning

- **Supervised learning:** every training example comes with a label. The model learns a mapping from input to the *known* correct output. This is like studying with an answer key. VisionDoc's fine-tuning is supervised - every document/question has a target answer.
- **Unsupervised learning:** you have inputs but *no* labels. The algorithm hunts for structure on its own - grouping similar items (clustering) or compressing data. Like sorting a pile of photos into stacks that "look alike" without anyone telling you the categories.
- **Self-supervised learning:** a clever hybrid that powers modern large models. The labels are generated *automatically from the data itself* - e.g. hide a word and make the model predict it. The huge base Qwen model was originally **pre-trained** this way on the open internet before VisionDoc ever fine-tuned it (see Ch. 6 for the pretrain -> fine-tune split).

Interview Q: DocVQA gives you images, questions, and answers. Is training on it supervised or unsupervised, and why?

A: Supervised - each example carries an explicit ground-truth answer, so the model learns a direct mapping from (image, question) to the labeled output.

2.4 Training vs. inference

These are the two phases of a model's life, and interviewers love to check you keep them straight.

- **Training** is the expensive, one-time (or occasional) *learning* phase. You feed labeled data in, compare the model's guesses to the labels, and adjust the model's numbers. It needs the labels, lots of compute, and can take hours or days.
- **Inference** (also "prediction" or "serving") is *using* the trained model on brand-new inputs to get answers. It needs no labels and is comparatively cheap and fast.

In this repo, training lives in `training/train.py` and `training/trainer.py` ; inference lives in the `inference/` package and is exposed through the FastAPI service (Ch. 12) and the Streamlit app (Ch. 13). The `inference:` block in `configs/default.yaml` (e.g. `max_new_tokens: 128` , `do_sample: false`) only affects serving, not learning.

Key insight: During training the model *changes*. During inference the model is *frozen* - the same input gives the same output every time (unless you deliberately add randomness, like `do_sample: true`).

2.5 Parameters, weights, and how a model "holds" knowledge

Inside the model are millions-to-billions of numbers called **parameters** (the most common kind are **weights**, with **biases** as a companion). A weight is a dial that says how strongly one signal should influence the next. Learning = finding good settings for all those dials.

The Qwen backbone here has ~3 **billion** parameters ("3B" in its name). You do not set these by hand - training does. And a central trick of this project (Ch. 6) is that fine-tuning does **not** touch all 3B; **LoRA** trains a tiny fraction while the rest stays frozen. But conceptually, "the knowledge is in the weights."

Analogy: Think of a giant mixing board with billions of sliders. The music (output) depends on every slider position. Training is a robot that listens to the output, compares it to the song you want, and inches the sliders toward it - over and over.

2.6 The loss function - a score for "how wrong"

To improve, the model needs a number that measures how bad its current guesses are. That number is the **loss** (or **cost**), produced by a **loss function**.

- Low loss = predictions close to the labels (good).
- High loss = predictions far from the labels (bad).

The entire goal of training is to make the loss as small as possible. For text-generating models like Qwen, the loss is **cross-entropy**: at each position the model outputs a probability for every possible next token (a token ~ a word-piece; see Ch. 5), and cross-entropy is high when it assigned low probability to the *correct* token. Minimizing it pushes the model to assign high probability to the right next word.

Gotcha: Loss is a *training-time optimization signal*, not the metric humans care about. VisionDoc reports quality with EM, ANLS, F1, etc. (Ch. 9). A model can have slightly lower loss yet worse ANLS - which is exactly why `configs/default.yaml` selects the best checkpoint by `metric_for_best_model: eval_anls`, not by loss.

2.7 Gradient descent - how the dials get turned

So we have billions of dials and a loss score. How do we know *which way* to turn each dial to reduce loss? This is where the one bit of calculus in ML shows up - explained without symbols.

A **gradient** answers, for every parameter: "if I nudge you up a hair, does the loss go up or down, and how steeply?" It is the *slope* of the loss with respect to each dial.

Gradient descent is the simple loop that uses it:

- 1 Run some examples through the model (a **forward pass**) and compute the loss.
- 2 Compute the gradient - the direction of steepest *increase* in loss.
- 3 Step every parameter a little in the **opposite** direction (downhill).
- 4 Repeat.

Analogy: You are on a foggy hillside (the "loss landscape") trying to reach the valley. You cannot see far, but you can feel which way is downhill under your feet (the gradient). You take a step downhill, feel again, step again. Eventually you reach a low point.

```
# The essence of gradient descent, stripped to nothing:  
for batch in data:
```

```

predictions = model(batch.inputs)      # 1. forward pass
loss = loss_fn(predictions, batch.labels)
gradients = compute_gradients(loss)    # 2. which way is uphill?
for p in model.parameters():
    p = p - learning_rate * gradients[p] # 3. step downhill

```

In practice you never call these by hand - PyTorch and HuggingFace's `Trainer` (Ch. 8) do it. The `AdamW` optimizer VisionDoc uses is a smarter variant that adapts the step size per parameter, but the core idea is unchanged: *follow the gradient downhill*.

2.8 Backpropagation - computing the gradient efficiently

A deep model is layers stacked on layers. To get the gradient for a dial buried deep inside, you need to know how a change there ripples all the way through to the final loss. **Backpropagation ("backprop")** is the algorithm that computes all those gradients efficiently, in one backward sweep.

It works by the **chain rule** of calculus: start at the loss, and pass the "blame" backward layer by layer, each layer multiplying by its local slope, until every parameter has its gradient. Forward pass = compute the answer front-to-back; backward pass = compute the gradients back-to-front.

Key insight: Backpropagation is *not* the learning rule - gradient descent is. Backprop is just the fast bookkeeping trick that supplies gradient descent with the numbers it needs. Modern frameworks call this **autograd** (automatic differentiation): you write the forward math, and the framework figures out the backward math for you.

Interview Q: What is the difference between backpropagation and gradient descent?

A: Backpropagation computes the gradients (the derivative of the loss w.r.t. every parameter) by applying the chain rule backward through the network. Gradient descent is the update rule that then uses those gradients to nudge each parameter downhill. Backprop produces the direction; gradient descent takes the step.

2.9 The training vocabulary: epochs, batches, learning rate

These three knobs appear directly in `configs/default.yaml` under `training:`. Understand them and you can read any training config.

- **Batch:** you don't feed all data at once (it wouldn't fit in memory and would be slow). You process a small handful - a **batch** - compute the loss on it, and take one gradient step. The **batch size** is how many examples per step. VisionDoc uses `per_device_train_batch_size: 1` because document images are large and GPU memory is tight.
- **Gradient accumulation:** to *simulate* a bigger batch without more memory, you can add up gradients over several small batches before stepping. `gradient_accumulation_steps: 8` x batch size 1 gives an **effective batch size of 8** - the model updates as if it saw 8 examples at once.
- **Epoch:** one full pass through the *entire* training set. `num_train_epochs: 3.0` means the model sees every training example three times. More epochs = more chances to learn, but also more risk of memorizing (see overfitting below).
- **Learning rate (LR):** the size of each downhill step. `learning_rate: 1.0e-4` (i.e. 0.0001). This is the single most important knob:
 - Too **high** -> you leap over the valley and the loss bounces around or explodes.
 - Too **low** -> you crawl; training takes forever and may stall.
- **Learning-rate schedule & warmup:** you often don't keep the LR fixed. VisionDoc uses `lr_scheduler_type: cosine` with `warmup_ratio: 0.03` - it starts the LR near zero,

ramps up over the first 3% of steps (so early, chaotic gradients don't wreck the weights), then smoothly decays it toward zero.

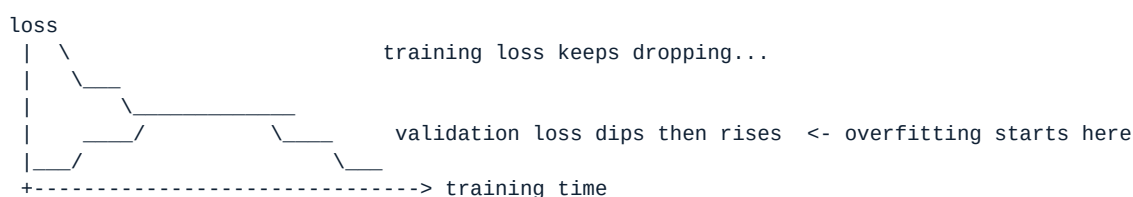
Analogy for LR: hiking downhill in the fog. Giant strides (high LR) get you down fast but might launch you off a cliff or across the valley. Baby steps (low LR) are safe but you might not reach the bottom before dark. Warmup = walk gently until your eyes adjust, then lengthen your stride.

Config key	Concept	VisionDoc value
<code>num_train_epochs</code>	passes over the data	3.0
<code>per_device_train_batch_size</code>	examples per step	1
<code>gradient_accumulation_steps</code>	steps summed before update	8 (-> effective batch 8)
<code>learning_rate</code>	downhill step size	1e-4
<code>warmup_ratio</code>	fraction of steps ramping LR up	0.03
<code>lr_scheduler_type</code>	how LR changes over time	cosine decay

2.10 Overfitting, underfitting, and generalization

The whole point of training is a model that works on **new** documents it never saw - this ability is **generalization**. Two failure modes threaten it:

- **Underfitting:** the model is too weak or under-trained; it fails even on the training data. Like a student who didn't study - bad on practice *and* real exams. Fix: train longer, use a bigger/better model, add useful features.
- **Overfitting:** the model *memorizes* the training examples, including their noise and quirks, instead of learning the general pattern. It aces the training data but flops on new data. Like a student who memorized last year's answer key word-for-word and is lost when the questions change.



The moment validation loss starts *rising* while training loss keeps falling is the classic overfitting signal. Defenses used in this project:

- **Early stopping:** stop training when validation quality stops improving. Config: `early_stopping_patience: 3`, `load_best_model_at_end: true` - keep the best checkpoint, not the last.
- **Weight decay** (`weight_decay: 0.01`): gently pushes weights toward zero so the model can't build overly complex, brittle rules - a form of **regularization** (anything that discourages memorizing).
- **Dropout** (`lora_dropout: 0.05`): randomly ignores a small fraction of internal signals during training so the model can't over-rely on any single one.
- **Data augmentation** (`augment: true`): slightly perturb inputs (rotate/scale images) so the model sees more variety and can't memorize exact pixels (Ch. 4).
- **Using less capacity:** LoRA itself trains only a tiny number of parameters, which limits how much it *can* memorize (Ch. 6).

Interview Q: Your training loss is 0.05 but validation loss is 3.0. What's happening and what do you do?

A: Classic overfitting - the model memorized the training set and doesn't generalize. Remedies: stop earlier (early stopping / pick the best-validation checkpoint), add regularization (weight decay, dropout), augment or add more training data, or reduce model capacity. I'd also sanity-check for a data leak or mismatch between the train and validation distributions.

2.11 Train / validation / test splits

To *measure* generalization honestly, you split your data into three disjoint piles:

Split	Purpose	Rule
Train	the model learns from these	the only data gradients touch
Validation (dev)	tune knobs & pick the best checkpoint	model sees them but never <i>trains</i> on them
Test	final, one-time report of real-world quality	touch only once, at the very end

VisionDoc reads named splits from DocVQA (`train_split: train` , `val_split: validation` , `test_split: test`) and can also carve holdouts from a single pool via `val_fraction: 0.1` and `test_fraction: 0.1` (10% each). The config drives validation with `eval_strategy: steps` , `eval_steps: 100` - every 100 steps it checks validation quality and, thanks to `metric_for_best_model: eval_anls` , remembers the best one.

Gotcha (data leakage): if the *same* invoice sneaks into both train and test, your test score is a lie - the model just recites something it already saw. Splits must be truly disjoint, and ideally split by document/vendor, not by individual question. This is one of the most common ways real projects fool themselves.

Key insight: The validation set is for *making decisions* (which LR? stop when?); the test set is for the *final honest number* and must stay untouched until the end. Peek at the test set to tune things and you've silently turned it into another validation set - your reported score no longer predicts real-world performance. Reproducibility is aided by `seed: 42` , which fixes the random shuffling so runs are comparable.

2.12 Why deep learning needs a GPU

Deep learning (DL) is machine learning with **neural networks** that have many stacked layers ("deep"). A neural network is, at its heart, a tower of matrix multiplications with simple nonlinear functions between them (Ch. 5 opens the box). Training billions of parameters means doing *enormous* numbers of these multiply-add operations, over and over, for every batch.

A **CPU** (central processing unit) has a few very fast, very flexible cores - great at doing one complicated thing at a time, in sequence. A **GPU** (graphics processing unit) has *thousands* of simpler cores that do the same operation on many numbers **in parallel**. Matrix multiplication is exactly "the same multiply-add across thousands of numbers," so a GPU can be tens to hundreds of times faster than a CPU for DL.

Analogy: A CPU is a handful of math PhDs - brilliant, versatile, but few. A GPU is a stadium of grade-schoolers all doing simple arithmetic at once. For adding a million pairs of numbers, the stadium wins by a landslide.

Related hardware facts this project cares about:

- **VRAM (GPU memory)** is the real bottleneck. A 3B-parameter model plus its gradients, optimizer state, and activations must all fit in VRAM. That's why the config keeps `per_device_train_batch_size: 1`, caps image resolution with `max_pixels`, and offers `load_in_4bit: true` (**QLoRA** - squeezing weights into 4-bit numbers to save memory; Ch. 7).
- **Numeric precision:** `bf16: true` (bfloat16) stores numbers in 16 bits instead of 32, halving memory and speeding up math with negligible quality loss on modern GPUs.
- **gradient_checkpointing: true** trades compute for memory - it discards some intermediate values in the forward pass and recomputes them during backprop, letting bigger models fit.
- **Device selection:** `device: auto` in this repo means *prefer CUDA (NVIDIA GPU) -> then Apple's MPS -> then CPU* (see `utils/device.py`). CPU works for tiny smoke tests but is far too slow for real training. The `configs/colab_t4.yaml` variant targets a free Google Colab **T4 GPU**.

Interview Q: Why is a GPU better than a CPU for training neural networks?

A: Neural-network training is dominated by large matrix multiplications, which are *embarrassingly parallel* - the same multiply-add applied across thousands of elements independently. GPUs have thousands of cores plus very high memory bandwidth built for exactly that, so they do the work in massively parallel fashion, often 10-100x faster than a CPU's handful of general-purpose cores. GPU VRAM capacity, not raw speed, is usually the binding constraint on model and batch size.

Gotcha: More GPU cores don't help if the model doesn't fit in VRAM. When people say "I got out-of-memory (OOM)," the fix is usually smaller batch size, gradient accumulation, lower precision (bf16/4-bit), gradient checkpointing, or smaller inputs - all of which appear in VisionDoc's config precisely for this reason.

2.13 Putting the loop together

Everything above assembles into one repeating cycle - the heartbeat of every training run in this project:

- 1 Grab a **batch** of labeled examples from the **train** split.
- 2 **Forward pass:** the model predicts; compute the **loss** vs. the labels.
- 3 **Backpropagation:** compute **gradients** of the loss for every trainable parameter.
- 4 **Gradient descent step:** the optimizer nudges parameters downhill by `~ learning_rate`.
- 5 Every so often (`eval_steps: 100`), score the frozen model on the **validation** split; keep the best checkpoint.
- 6 Repeat for all batches, for `num_train_epochs` passes - stopping **early** if validation quality plateaus.
- 7 Finally, report honest quality on the **test** split, once.

Steps 1-4 are the same for a tiny toy network and for a 3-billion-parameter VLM; only the scale and the plumbing differ. With this bedrock in place, Ch. 5 opens up what a neural network and a transformer actually are, and Ch. 6 explains how LoRA lets you fine-tune a giant model by training only a sliver of it.

3. Neural Networks, the Transformer, and Attention

Everything VisionDoc AI does - reading an invoice, answering "what is the total?", extracting fields - runs on a single family of models called the **Transformer**. To understand the project (and to survive an ML interview) you need to know how a Transformer works from the ground up. This chapter builds that understanding starting from a single artificial neuron, then layers, then the special problem of sequences, and finally the **attention** mechanism that makes modern language and vision-language models possible. Math is kept light but honest.

3.1 From a neuron to a layer

An **artificial neuron** is a tiny math function. It takes some numbers in, multiplies each by a **weight** (a learned importance value), adds them up, adds a **bias** (a learned offset), and passes the result through a nonlinear function.

```
# One neuron: inputs x, weights w, bias b
z = w1*x1 + w2*x2 + w3*x3 + b      # weighted sum (a "linear" step)
y = relu(z)                        # nonlinearity: relu(z) = max(0, z)
```

- **Weight** - a knob the model learns; large weight means "this input matters a lot."
- **Bias** - a constant that shifts the output up or down, like the intercept of a line.
- **Nonlinearity / activation function** - a function like `ReLU ( max(0, z) )` or `GELU` that bends the straight-line math. Without it, stacking neurons would collapse back into one big straight line and the network could only learn straight-line relationships. The nonlinearity is what lets networks learn curves, corners, and complex patterns.

Interview Q: Why do neural networks need nonlinear activation functions?

A: A neuron's core operation is linear (a weighted sum). Composing linear functions yields another linear function, so any number of stacked linear layers is mathematically equivalent to a single linear layer - it can only fit straight-line relationships. Inserting a nonlinearity (ReLU, GELU, etc.) between layers lets the network approximate arbitrary nonlinear functions, which is the whole point of "deep" learning.

A **layer** is just many neurons computed in parallel, each with its own weights. In practice we don't loop over neurons; we pack the weights into a **matrix** and do one matrix multiply. This packed layer is called a **linear layer** (or "fully-connected" / "dense" layer). In PyTorch it is `nn.Linear`. You will see this exact object all over VisionDoc AI - for example the model's final `lm_head` is described in `models/onnx_export.py` as "a plain `Linear`" that maps hidden states to vocabulary scores, and the LoRA fine-tuning in `models/qwen_vl.py` targets seven linear layers by name: `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`. Every one of those is a matrix multiply.

- **Vector** - an ordered list of numbers, e.g. `[0.2, -1.1, 0.7]`. Think of it as a point in space.
- **Matrix** - a grid of numbers; multiplying a vector by a matrix transforms it into a new vector. This is the fundamental operation of all these models.

3.2 From a layer to a deep network, and how it learns

Stack many layers (linear -> nonlinearity -> linear -> nonlinearity -> ...) and you have a **deep neural network** - "deep" simply means "many layers." Early layers learn simple features; later layers combine

them into complex concepts. A network with hundreds of millions or billions of weights is called a **large model**; Qwen2.5-VL, the model VisionDoc AI fine-tunes, has billions.

How does it learn the right weights? Three ideas:

- 1 **Loss function** - a number measuring how wrong the model's output is versus the correct answer. Lower is better. For text generation the standard loss is **cross-entropy** (covered in Ch. 5), which penalizes the model for assigning low probability to the correct next token.
- 2 **Backpropagation** - an algorithm that computes, for every weight, "if I nudge this weight up a little, does the loss go up or down, and how strongly?" That sensitivity is the **gradient**. Backprop uses calculus's chain rule to compute all gradients efficiently in one backward pass.
- 3 **Gradient descent** - nudge every weight a tiny step in the direction that lowers the loss. Repeat over millions of examples. The step size is the **learning rate**.

Key insight: "Training" is nothing more than: run data forward -> measure loss -> backpropagate gradients -> take a small step downhill -> repeat. Everything fancy (Adam, schedulers, LoRA) is a refinement of this loop. See Ch. 5 for how VisionDoc AI runs it.

Interview Q: What is a gradient, in one sentence?

A: The gradient of the loss with respect to a weight is the derivative that tells you how much (and in which direction) the loss changes if you change that weight; gradient descent moves weights opposite the gradient to reduce loss.

3.3 Why sequences (text) need special handling

An image can be handed to a network as a fixed grid of pixels. **Text is different: it is a sequence** - an ordered list of words where order changes meaning ("dog bites man" != "man bites dog") and length varies (a sentence, a paragraph, a whole document). A plain linear layer expects a fixed-size input and has no notion of order or "look back at earlier words." So sequence models need two extra abilities:

- **Handle variable length** - process 5 tokens or 5000 with the same weights.
- **Model relationships between positions** - the meaning of "it" depends on a noun mentioned earlier; "total" on an invoice refers to a number elsewhere on the page.

Before we get to the mechanism, we need to turn words into numbers.

3.4 Tokenization: turning text into tokens

Models don't see letters or words; they see **tokens** - integer IDs from a fixed vocabulary. **Tokenization** is the process of chopping text into these units. Modern models use **subword tokenization** (e.g. Byte-Pair Encoding, BPE): common words become one token, rare words split into pieces.

```
"invoice total" -> ["invoice", " total"] -> [15931, 2790]
"Fribourg"      -> ["Fri", "bourg"]      -> [37, 24601]
```

- **Vocabulary** - the fixed set of all tokens the model knows (often 30k-150k entries).
- **Subword** - a piece of a word. Subwords keep the vocabulary small while still representing any word, even unseen ones, by combining pieces.
- **Special tokens** - reserved tokens with structural meaning rather than literal text. VisionDoc AI uses these heavily: `models/donut.py` registers task tokens like `<s_docvqa>` and `<s_cord-v2>` and grows the embedding table to fit them; Qwen2.5-VL uses chat-template tokens and **image tokens** that stand in for the picture inside the text sequence.

Gotcha: Token count is not word count. "Fribourg" may be several tokens; whitespace and capitalization change the split. This matters for context-length limits and for cost/latency, and it is why the code in `models/qwen_vl.py` carefully measures the *prompt length including expanded image tokens* so it can mask the prompt and compute loss only over the answer tokens.

Embeddings: turning tokens into vectors

A token ID like `2790` is just an arbitrary label; the model can't do math on "2790" meaningfully. So each token ID is mapped to a **vector** of learned numbers called an **embedding**.

- **Embedding** - a dense vector (say 3584 numbers for Qwen2.5-VL's hidden size) that represents a token's meaning. It lives in an **embedding table**: one row per vocabulary entry, looked up by token ID.
- Crucially, embeddings are *learned*, so tokens with similar meaning end up with similar vectors. Classic intuition: `vector("king") - vector("man") + vector("woman") ~ vector("queen")`. Meaning becomes geometry.
- **Hidden size / hidden dimension** - the length of these vectors (the model's "width"). Every token flows through the network as a vector of this size; `models/onnx_export.py` reads `hidden_size` off the model to build the final projection.

Key insight: The pipeline is always **text -> tokens (integers) -> embeddings (vectors)**. From here on, the network only ever manipulates vectors. When VisionDoc AI adds new special tokens, it must `resize_token_embeddings` (as `models/donut.py` does) so the embedding table gains rows for the new IDs.

3.5 The old approach: RNNs, and why they struggled

Before Transformers, sequences were handled by **RNNs (Recurrent Neural Networks)**. An RNN reads one token at a time and carries a **hidden state** - a running summary vector - forward, updating it at each step. It's like reading a sentence word by word while trying to keep a mental note of everything so far.

Two fatal weaknesses:

Problem	Why it hurts
Sequential computation	Token t can't be processed until $t-1$ is done. No parallelism, so training on modern GPUs is slow.
Long-range forgetting	Information from far-back tokens must survive many update steps; gradients shrink ("vanishing gradient") and early context is forgotten. LSTMs/GRUs helped but didn't fully solve it.

The Transformer, introduced in the 2017 paper "Attention Is All You Need," replaced recurrence with **attention**, fixing both problems at once: every token can look at every other token directly, and all tokens are processed in parallel.

Interview Q: Why did Transformers replace RNNs?

A: RNNs process tokens sequentially (no parallelism, slow training) and struggle to carry information across long distances due to vanishing gradients. Self-attention lets every token attend directly to every other token in a single step, giving constant path length between any two positions (better long-range modeling) and full parallelism across the sequence (much faster on GPUs). Transformers also scale better with data and parameters.

3.6 Self-attention: the core idea

Attention answers the question: *when building a richer representation of this token, which other tokens should I pay attention to, and how much?* On the sentence "The invoice total is 42, pay it by Friday," the token "it" should attend strongly to "invoice" (or "total"). Self-attention lets it do exactly that, in one step, regardless of distance.

The mechanism gives each token three vectors, produced by three learned linear layers - and these are precisely the `q_proj`, `k_proj`, `v_proj` modules that VisionDoc AI fine-tunes with LoRA in `models/qwen_vl.py`:

- **Query (Q)** - "what am I looking for?" Produced by `q_proj`.
- **Key (K)** - "what do I offer / what am I about?" Produced by `k_proj`.
- **Value (V)** - "the actual information I'll pass along if attended to." Produced by `v_proj`.

An analogy: it's like a search engine over the sentence. Each token issues a **query**; every token advertises a **key**. You match your query against all keys to get relevance scores, then retrieve a blended mix of the matching **values**.

The computation for one token:

- 1 **Score** - take this token's Query and compute a similarity (dot product) with every token's Key. High score = "this other token is relevant to me."
- 2 **Softmax** - convert the raw scores into **attention weights** that are all positive and sum to 1. **Softmax** is a function that turns arbitrary numbers into a probability-like distribution; it makes the largest scores dominate while keeping some weight on others.
- 3 **Weighted sum** - multiply each token's Value by its attention weight and add them up. The result is the new, context-aware vector for this token.

```
# Self-attention, conceptually (one head). Q, K, V are (seq_len, d)
scores = Q @ K.T / sqrt(d)      # every token vs every token -> (seq_len, seq_len)
weights = softmax(scores, axis=-1) # each row sums to 1
context = weights @ V           # blended values -> new (seq_len, d) representation
```

- The `/ sqrt(d)` is **scaling** (the "scaled" in *scaled dot-product attention*): it keeps scores from growing too large as the vector dimension `d` grows, which would make softmax too spiky and gradients unstable.
- After attention, an output projection (`o_proj` in the repo) mixes the result back to hidden size.

Key insight: Attention is *content-based routing*. There are no fixed connections between positions; the weights are computed on the fly from the actual tokens. That is why the same layer handles any sequence length and connects any two positions directly - the "distance" between "it" and "invoice" is always one attention hop.

Interview Q: Explain query, key, and value in your own words.

A: For each token, the model derives three vectors via learned linear projections. The query represents what this token is looking for; keys represent what each token can be matched against; values carry the information to be aggregated. Attention scores each query against all keys (scaled dot product), softmaxes them into weights summing to one, and returns a weighted sum of values - a context-aware representation of the token.

3.7 Multi-head attention

One set of Q/K/V learns *one* kind of relationship. Language needs many simultaneously - syntax, coreference ("it" -> "invoice"), number-label pairing, and so on. **Multi-head attention** runs several attention operations ("heads") in parallel, each with its own smaller Q/K/V projections, then concatenates their outputs and mixes them with `o_proj`.

- **Head** - one independent attention computation over a slice of the hidden vector. Think of a committee of specialists each reading the sentence for a different kind of pattern, then pooling their notes.
- If hidden size is 3584 and there are 28 heads, each head works in a 128-dim subspace. More heads = more relationship types captured per layer, at the same total width.

Gotcha: More heads is not strictly better. Heads split the hidden dimension, so too many heads make each head's subspace tiny and weak. Head count is a tuned architectural choice, not a free dial.

3.8 Positional information

Attention as described has a surprising blind spot: it is **permutation-invariant** - shuffle the tokens and the set of Q/K/V is the same, so "man bites dog" and "dog bites man" would look identical. Attention sees a *set*, not a *sequence*. We must inject **positional information**.

- **Positional encoding** - extra signal added so each token knows *where* it sits. Early Transformers added fixed sinusoidal patterns to embeddings.
- **RoPE (Rotary Position Embedding)** - the modern approach used by Qwen2.5-VL: it *rotates* the Query and Key vectors by an angle that depends on position, so the dot product between two tokens naturally encodes their relative distance. `models/onnx_export.py` explicitly notes that "rotary embeddings" have no clean ONNX lowering - a real, repo-visible consequence of RoPE living inside the attention math.

Interview Q: Why does a Transformer need positional encodings when an RNN does not?

A: An RNN processes tokens in order, so position is implicit in the timing of the recurrence. Self-attention processes all tokens simultaneously and is permutation-invariant - without added positional signal it cannot distinguish word order. Positional encodings (sinusoidal, learned, or rotary/RoPE) inject the missing order information; RoPE in particular encodes *relative* position directly into the attention dot product.

3.9 Putting it together: the Transformer block, and encoder vs decoder

A **Transformer block** (one layer) stacks two sub-layers, each wrapped with two stabilizers:

- 1 **Multi-head self-attention** - tokens exchange information.
- 2 **Feed-forward network (MLP)** - two linear layers with a nonlinearity applied to each token independently, to further transform it. In the repo these are the `gate_proj` , `up_proj` , `down_proj` layers.

The two stabilizers, applied around each sub-layer:

- **Residual connection** - add the sub-layer's input back to its output ($x + \text{sublayer}(x)$). This gives gradients a shortcut path, letting very deep stacks train without the signal vanishing.
- **Layer normalization** - rescale each token's vector to a stable mean/variance so numbers don't blow up or collapse across dozens of layers.

Stack N such blocks (Qwen2.5-VL has dozens) and you have the model. Now, the three architectural shapes:

Architecture	How attention flows	Good for	Example in project
Encoder-only	Every token attends to all tokens (bidirectional)	Understanding/classification; produces representations, not new text	BERT-style (not the generator here)
Encoder-decoder (seq2seq)	Encoder reads input bidirectionally; decoder generates output while <i>cross-attending</i> to the encoder	Input->output transforms where input and output differ	Donut in <code>models/donut.py</code> : a Swin image encoder + BART-style text decoder
Decoder-only	Each token attends only to itself and earlier tokens (causal masking); generates left-to-right	Free-form text generation	Qwen2.5-VL in <code>models/qwen_vl.py</code> : a vision encoder feeding a language decoder

Two definitions that table depends on:

- **Causal (masked) attention** - in a decoder, a token may only attend to positions *at or before* itself, never the future. Enforced by setting future scores to -inf before softmax. This is what lets the model *generate*: predict token t using only tokens $1...t-1$, then append and repeat. It's why `models/qwen_vl.py` uses **left padding** at generation time - so newly appended tokens line up across a batch.
- **Cross-attention** - attention where the Queries come from one sequence (the decoder's text) and the Keys/Values from another (the encoder's output). This is how Donut's text decoder "looks at" the encoded image while writing the answer.

Interview Q: What is the difference between encoder-only, decoder-only, and encoder-decoder Transformers?

A: Encoder-only (BERT) uses bidirectional self-attention to build rich representations for understanding tasks but doesn't generate text. Decoder-only (GPT, Qwen2.5-VL's language model) uses causal masking so each token sees only past tokens, enabling autoregressive generation. Encoder-decoder (T5, BART, Donut) pairs a bidirectional encoder over the input with a causal decoder that cross-attends to the encoder output - ideal for sequence-to-sequence transforms. VisionDoc AI ships both a decoder-only path (Qwen2.5-VL) and an encoder-decoder path (Donut).

Key insight for vision-language models: A VLM slots the image into this same token machinery. Qwen2.5-VL runs the picture through a **Vision Transformer (ViT)** encoder to produce image *tokens*, which are inserted into the text sequence as embeddings; from then on self-attention treats image and text tokens uniformly - text tokens can attend to image regions, which is exactly what makes visual question answering and region grounding possible. (The ViT is kept frozen during LoRA fine-tuning; see Ch. 4.)

3.10 Chapter summary

- A neuron is a weighted sum plus a nonlinearity; a layer is many neurons as a matrix multiply; a deep network stacks layers and learns via loss -> backpropagation -> gradient descent.
- Text is a variable-length ordered sequence, so it is turned into **tokens** (integer IDs), then **embeddings** (learned vectors). Everything downstream is vectors.
- **RNNs** handled sequences serially and forgot long-range context; **Transformers** replaced them using **self-attention**.
- **Self-attention** routes information by content: each token's **Query** matches all **Keys**, softmax gives weights, and a weighted sum of **Values** yields a context-aware vector. **Multi-head** attention does this many ways at once; **positional encodings/RoPE** restore word order.
- A Transformer block = multi-head attention + feed-forward, each with residual connection and layer norm. **Decoder-only** (Qwen2.5-VL) generates with causal masking; **encoder-decoder** (Donut) generates while cross-attending to an encoded input.
- These exact components are visible in the codebase: LoRA targets the attention/MLP linear layers (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`), Donut is a documented Swin-encoder + BART-decoder seq2seq model, and RoPE/embedding-resize details surface in `models/onnx_export.py` and `models/donut.py` . Chapter 4 explains how the two model adapters are built, and how LoRA fine-tunes just these layers cheaply.

4. Large Language Models and How Text Is Generated

VisionDoc AI is built on a **Vision-Language Model** - but the "language" half of that model is, under the hood, an ordinary **Large Language Model (LLM)**. If you understand how an LLM reads text and produces new text one piece at a time, you understand the engine that writes the answers, the extracted fields, and the JSON that VisionDoc returns. This chapter teaches that engine from zero.

4.1 What an LLM actually is

A **Large Language Model** is a neural network - a big mathematical function with billions of adjustable numbers called **parameters** (also called *weights*) - that has been trained to predict text. "Large" refers to the parameter count (often 1-100+ billion) and the amount of text it was trained on.

An analogy: imagine someone who has read a substantial fraction of the public internet, books, and code, and whose *only* skill is this - given some text, guess what word most naturally comes next. That is literally all an LLM does. Everything else it appears to do (answer questions, write code, extract invoice totals) is a consequence of doing that one thing extremely well.

Key insight: An LLM is not a database that "looks up" answers. It is a *predictor*. It compresses statistical patterns of language into its weights and reconstructs plausible continuations. This is why it can be fluent and confident while still being wrong - a theme we return to under hallucination (§4.9).

Tokens: the unit an LLM works in

LLMs do not read letters or whole words. They read **tokens** - chunks of text produced by a **tokenizer**. A token is typically a word, a word-piece, or a punctuation mark. For example "invoice" might be one token, while "VisionDoc" might split into "Vision" + "Doc". A rough rule of thumb for English: **1 token ~ 0.75 words**, or ~4 characters.

Every token has an integer ID. The full set of possible tokens is the **vocabulary** (often 30,000-150,000 tokens). When you see `input_ids` in the VisionDoc code (e.g. in `models/base.py`), those are exactly these token IDs.

Interview Q: Why do models use subword tokens instead of whole words?

A: Whole-word vocabularies can't represent rare or unseen words (out-of-vocabulary problem) and would be enormous. Subword tokenization gives a fixed, compact vocabulary that can still spell out any string by composing pieces - "VisionDoc" becomes "Vision" + "Doc". It balances vocabulary size against sequence length.

4.2 The pretraining objective: next-token prediction

How does a network learn language? Through **pretraining** - the first, most expensive training phase, done on a huge unlabeled text corpus. The training task is deceptively simple: **next-token prediction** (also called *causal* or *autoregressive* language modeling).

The recipe:

- 1 Take a real sentence from the corpus, e.g. "The total on the invoice is".

- 2 Feed the model those tokens and ask it to predict the next one ("").
- 3 Compare its prediction to the real next token.
- 4 Nudge the weights so the correct token becomes slightly more likely.
- 5 Repeat trillions of times.

The "compare and nudge" uses a **loss function** called **cross-entropy loss** - a number that is small when the model assigned high probability to the true next token, and large when it did not. Training minimizes this loss. (You will see cross-entropy again in Ch. 7 on training, because fine-tuning uses the *same* objective on document data.)

Interview Q: Why is next-token prediction such a powerful training signal?

A: Because to predict the next token well across billions of diverse examples, the model is forced to implicitly learn grammar, facts, reasoning patterns, code syntax, and formatting. There is no separate "learn grammar" step - all capability is a side effect of relentless next-token prediction. It's also self-supervised: the label (the next token) comes free from the text itself, so no human annotation is needed.

4.3 Autoregressive generation: writing one token at a time

Once trained, the model *generates* text the same way it was trained - one token at a time, feeding its own output back in. This is called **autoregressive generation** ("auto" = self, "regressive" = feeding prior outputs back).

The loop:

```
prompt tokens -> model -> predict token 1 -> append it
prompt + token1 -> model -> predict token 2 -> append it
prompt + token1 + token2 -> model -> predict token 3 -> ...
... until an End-Of-Sequence (EOS) token is produced, or a length cap is hit.
```

The **EOS token** is a special "I'm done" token. Generation stops when the model emits it, or when `max_new_tokens` is reached. In VisionDoc, `max_new_tokens` defaults to `128` (see `InferenceConfig` in `configs/config.py`) - a cap that prevents runaway generation and controls latency.

Gotcha: Generation is inherently sequential - token *N* can't start until token *N-1* is chosen. This is why LLM inference latency scales with output length, and why VisionDoc caps `max_new_tokens`. Extraction answers are short, so a tight cap is both faster and safer.

4.4 From logits to probabilities: softmax

At each step, the model does *not* directly output a token. It outputs a **logit** for every token in the vocabulary - a raw, unbounded score (can be negative or positive) saying how much the model "likes" that token as the next one. A vocabulary of 100k tokens means 100k logits per step.

Logits are not probabilities (they don't sum to 1, and can be negative). To turn them into a proper **probability distribution**, we apply the **softmax** function:

$$\text{softmax}(z_i) = \exp(z_i) / \sum_j \exp(z_j)$$

In words: exponentiate every logit (making them all positive), then divide by their sum (so they add up to 1). The result: each token gets a probability in $[0, 1]$, and the whole vocabulary sums to 1.0. The token with the highest logit also has the highest probability.

Key insight: Softmax is *the* bridge from a neural network's raw scores to something we can interpret as "probability of this token." Every decoding strategy below operates on this post-softmax distribution.

You will see the exact operation in VisionDoc's `models/base.py`, where `_token_logprobs_from_scores` calls `torch.log_softmax(step_scores.float(), dim=-1)` - the log of the softmax, computed in one numerically-stable step. We'll see why *log* probabilities matter for confidence in §4.8.

4.5 Decoding strategies: choosing the next token

Given a probability distribution over the vocabulary at each step, how do we pick the actual token? That choice is called the **decoding** (or **sampling**) **strategy**, and it dramatically changes the output's character. VisionDoc exposes all the relevant knobs in `InferenceConfig`.

Greedy decoding

Greedy decoding always picks the single highest-probability token at each step. It is **deterministic**: the same input always yields the same output.

- Pro: predictable, reproducible, tends to be "safe."
- Con: can be repetitive or bland for open-ended creative text, and can get stuck (a locally-best token can lead to a globally worse sentence).

VisionDoc uses greedy by default. In `configs/config.py`, `do_sample: bool = False` with the comment "*greedy is deterministic + best for extraction*", and `num_beams: int = 1`. Together these mean: no randomness, no beam search - just pick the top token each step.

Sampling, and the temperature knob

Sampling means: instead of always taking the top token, roll a weighted die according to the probability distribution. A token with probability 0.3 is chosen ~30% of the time. This introduces **randomness** and thus variety - the same prompt can produce different outputs.

Temperature (`temperature`, default `0.7` in VisionDoc) reshapes the distribution *before* sampling by dividing the logits by T :

```
softmax(z_i / T)
```

- **$T = 1.0$** - unchanged distribution.
- **$T < 1.0$** (e.g. 0.2) - "colder": sharpens the distribution toward the top tokens -> more deterministic, more focused.
- **$T > 1.0$** (e.g. 1.5) - "hotter": flattens the distribution -> more surprising, more diverse, more error-prone.
- **$T \rightarrow 0$** - approaches greedy.

Analogy: temperature is a creativity dial. Low = cautious and repetitive; high = imaginative and risky.

Top-k and top-p (nucleus) sampling

Pure sampling can occasionally pick a truly bad, low-probability token (the long tail of 100k tokens has a lot of garbage). Two techniques restrict the candidate pool first:

- **Top-k sampling:** keep only the k most probable tokens, renormalize, then sample. E.g. $k=50$ ignores everything past the top 50.

- **Top-p (nucleus) sampling:** keep the smallest set of tokens whose probabilities *sum to p* (e.g. 0.9), then sample from that "nucleus." Unlike top-k, the pool size adapts - it's small when the model is confident, larger when it's uncertain.

VisionDoc sets `top_p: float = 0.9`. Crucially, look at `models/base.py`: `temperature` and `top_p` are only applied **when sampling is on** - `if icfg.do_sample: gen_config.update(temperature=..., top_p=...)`. Under the default greedy path they are inert. This is a common and correct pattern: those knobs are meaningless without `do_sample=True`.

Gotcha: A frequent bug (and interview trap) is setting `temperature=0` and expecting greedy, while passing `do_sample=True`. Different libraries handle `T=0` differently and may divide by zero or error. The clean way to get deterministic output - the way VisionDoc does it - is `do_sample=False`, not `temperature=0`.

Beam search

Beam search keeps the *b* most promising partial sequences ("beams") alive at once instead of committing to one token. At each step it expands all beams, scores the resulting sequences by total (log) probability, and keeps the best *b*. At the end it returns the highest-scoring complete sequence.

- Pro: finds higher-overall-probability sequences than greedy; good for tasks with one "correct" answer (translation, structured extraction).
- Con: slower (bx the work), can be repetitive, and doesn't help open-ended creativity.

VisionDoc keeps `num_beams=1` (i.e. beam search off) by default but the generation code fully supports it: `_token_logprobs_from_scores` even handles `beam_indices` - the bookkeeping that maps each returned token back to the beam row that produced it - so confidence stays correct under beam search.

Which strategy for which job?

Task	Recommended	Why
Field extraction / structured JSON	Greedy (<code>do_sample=False</code>)	Deterministic, reproducible, no creativity needed - you want <i>the</i> answer
Factual VQA (what's the total?)	Greedy or low-temperature	Correctness over variety
Brainstorming / creative writing	Sampling, <code>T~0.8-1.0</code> , <code>top-p 0.9</code>	Diversity is the point
Translation / summarization	Beam search (<code>b=4</code>)	One high-quality target sequence

Interview Q: VisionDoc extracts invoice fields. Why greedy, not sampling?

A: Extraction is a task with a single correct answer, and downstream systems (and evaluation with Exact Match / ANLS - see Ch. 8) need reproducibility. Sampling would make the same invoice yield different outputs across runs, breaking determinism, making bugs non-reproducible, and adding no value since there's nothing to be "creative" about. Greedy (`do_sample=False` , `num_beams=1`) gives a stable, defensible answer.

4.6 Context window

The **context window** is the maximum number of tokens the model can attend to at once - prompt *plus* generated output combined. Think of it as the model's working-memory span. Everything outside the window is invisible to it.

- Modern LLMs range from a few thousand to hundreds of thousands of tokens of context.
- If your prompt + document text + question exceeds the window, something must be truncated - and truncation silently drops information.
- For VisionDoc, the image is encoded into a *fixed number of visual tokens* (Ch. 3 covers the vision encoder) that also consume context budget alongside the text prompt and the answer.

Gotcha: "The model ignored the top of my document" is often not a bug in the model - it's context overflow. The relevant text fell outside the window or was truncated. Always check token counts before blaming the model.

4.7 Prompts, instruction tuning, and chat templates

Prompts

A **prompt** is the input text you give the model to condition its generation. In VisionDoc, the prompt is the user's question (e.g. "What is the invoice total?") plus the encoded image. In `models/base.py` the question is carried through as `prompt=qs[i]` on each `GenerationOutput`.

Instruction tuning

A raw pretrained model only continues text - ask it a question and it might continue with *more questions* rather than answer. **Instruction tuning** is a second training phase where the model is fine-tuned on many (instruction, desired response) pairs, teaching it to *follow instructions* and *answer* rather than merely autocomplete. Models like Qwen2.5-VL ship in instruction-tuned ("-Instruct") variants for exactly this reason.

Key insight: Pretraining teaches language; instruction tuning teaches *helpfulness and format-following*. VisionDoc's LoRA fine-tuning (Ch. 6-7) goes one step further, specializing an already-instruction-tuned model on document tasks.

Chat templates and roles

Instruction-tuned chat models expect input formatted into **roles** - usually `system`, `user`, and `assistant`:

- **system** - standing instructions / persona ("You are a document extraction assistant. Respond in JSON.").
- **user** - the actual question or request.
- **assistant** - the model's reply (during generation, this is what's being produced).

A **chat template** is the model-specific recipe that stitches these roles into the exact token string the model was trained on, using special delimiter tokens (e.g. `<|im_start|>user ... <|im_end|>`). Every model family has its own template, and using the *wrong* one degrades quality badly. HuggingFace tokenizers expose `apply_chat_template()` to do this correctly; VisionDoc's processor handles this formatting when it builds `input_ids` inside `prepare_inference_inputs`.

Interview Q: Why can't you just concatenate "System: ... User: ..." as plain text?

A: Because the model learned to recognize *specific* special tokens as role boundaries during instruction tuning. Plain-text imitations use different tokens than the model expects, so it doesn't cleanly separate instructions from content - leading to prompt-injection-like leakage and worse instruction-following. Always use the model's own chat template.

4.8 Confidence from token log-probabilities

Here is where the theory pays off in VisionDoc's actual code. Recall that softmax gives each generated token a probability. VisionDoc turns those into a single **confidence score** in $[0, 1]$ for the whole answer.

The mechanics, traced through `models/base.py` :

- 1 `generate()` is called with `return_dict_in_generate=True, output_scores=True` . This makes the model return, for every generation step, the **scores** (logits) over the vocabulary - not just the chosen tokens.
- 2 `_token_logprobs_from_scores` converts each step's logits to **log-probabilities** with `torch.log_softmax(...)` , then gathers the log-prob of the *token that was actually chosen* at that step. Accumulation **stops at the first EOS** per sequence, so padding tokens on early-finished sequences aren't counted.
- 3 `_sequence_confidence` maps that list of per-token log-probs into one number.

Why **log**-probabilities? A sequence's joint probability is the *product* of its per-token probabilities. Multiplying many numbers below 1 underflows toward zero. Working in log space turns the product into a *sum* ($\log(a*b) = \log a + \log b$), which is numerically stable.

VisionDoc offers two `confidence_method` options (default `"seq_prob"`):

```
# models/base.py - _sequence_confidence
if method == "min_prob":
    return float(math.exp(min(token_logprobs))) # the least-confident token
mean_lp = sum(token_logprobs) / len(token_logprobs)
return float(math.exp(mean_lp)) # geometric-mean token prob
```

- `seq_prob` - $\exp(\text{mean log } p)$. This is the **geometric mean** of the per-token probabilities: a smooth, length-normalized "how probable was this answer overall." Length-normalizing (dividing by token count) matters so that longer answers aren't automatically penalized.
- `min_prob` - $\exp(\text{min log } p)$ - the probability of the *single least-confident token* . A conservative floor: if even one token was a coin-flip, the whole answer is flagged as risky. Useful for "should a human review this extraction?" gates.

Interview Q: Is a high token-probability confidence the same as the answer being correct?

A: No - and this is the crucial caveat. It measures the model's *internal certainty* (how peaked its own distribution was), not ground-truth correctness. Models can be confidently wrong (poorly **calibrated**). It's a useful *relative* signal - low confidence reliably flags shaky outputs for review - but it must not be treated as a probability of correctness without calibration against real labels. VisionDoc uses it to *rank and flag*, not to certify.

Gotcha: Under *sampling*, the chosen token isn't the max-probability token, so `seq_prob` confidence can be legitimately lower even for good answers. This is another reason extraction uses greedy: greedy maximizes each token's probability, making the confidence signal cleaner and comparable across documents.

4.9 Hallucination

A **hallucination** is when an LLM produces text that is fluent, plausible, and *confidently stated* but factually wrong or unsupported by the input. The model isn't "lying" - recall §4.1: it is a next-token predictor optimizing for *plausibility*, not *truth*. When it lacks the real answer, the statistically-likely continuation is often a well-formed guess.

For document intelligence, hallucination is the central risk: an LLM asked for an invoice total it can't read may *invent* a realistic-looking number. This is exactly why VisionDoc's design leans on several guardrails covered elsewhere:

- **Grounding** - tying extracted values to regions of the actual image via attention/region highlighting (the `grounded` field in `inference/extract.py`, and Ch. 9 on visualization), so an answer can be checked against where it "came from."
- **Confidence scoring** (§4.8) to flag low-certainty outputs for human review.
- **Deterministic greedy decoding** to avoid sampling-induced fabrication.
- **Optional OCR cross-check** (`ocr_used` in `extract.py`) to corroborate text the model claims to see.

Interview Q: How would you *reduce* hallucination in a document-extraction system?

A: Combine several tactics: (1) ground outputs in source regions so every field is traceable to pixels; (2) use low-temperature/greedy decoding to avoid inventive tokens; (3) surface a confidence score and route low-confidence extractions to human review; (4) constrain the output format (e.g. JSON schema / constrained decoding) so the model can't ramble; (5) cross-verify against an independent signal like OCR; and (6) fine-tune on in-domain documents (Ch. 6-7) so the model has actually *learned* the layouts it will face rather than guessing.

Key insight: You cannot fully eliminate hallucination from a probabilistic generator - you *manage* it. VisionDoc's answer is not "trust the model" but "make every answer checkable, scored, and reproducible." That mindset - treating LLM output as evidence to be verified, not truth to be accepted - is the professional posture interviewers look for.

With generation, decoding, and confidence understood, you now know how the language engine turns an encoded document + question into scored text. Ch. 5 opens the model up further, and Ch. 6-7 show how LoRA fine-tuning reshapes *which* tokens this engine predicts for document tasks.

5. Vision-Language Models and Qwen2.5-VL

Everything in VisionDoc AI hangs off one idea: a single neural network that can *look at a document image and read/reason about it in language*. That kind of network is called a **Vision-Language Model (VLM)**. This chapter builds the concept from zero - what a VLM is, how it turns pixels into something a language model can consume, the specifics of the default backbone (**Qwen2.5-VL**), and why the project also ships a very different alternative (**Donut**). By the end you should be able to explain, in an interview, exactly what happens between "here is a photo of an invoice" and "the total is \$412.50."

5.1 What is a Vision-Language Model, and why "multimodal"?

A **modality** is a type of data: text is one modality, images are another, audio is another. A model is **multimodal** when it accepts more than one modality at once. A plain language model (like the text-only chat models you know) only reads and writes text. A **Vision-Language Model** reads *both* an image and text, and writes text.

Key insight: A VLM is not "an OCR engine bolted to a chatbot." OCR (Optical Character Recognition - software that transcribes pixels into a string of characters) is a separate, lossy step. A VLM instead learns to *look directly at the pixels* and answer, so it can use layout, boxes, logos, stamps, and handwriting that a plain text transcript would throw away.

Why does multimodality matter *specifically for documents*? A document is not just its words. An invoice's meaning lives in **spatial structure**: the number next to the word "Total," the column a value sits under, the box a field is printed in, the small print in a footer. Consider the two-column receipt where "Tax" and "Subtotal" each have a dollar amount to their right. A pure text transcript ("Subtotal Tax 12.00 3.00") loses which number goes with which label. A VLM sees the geometry and keeps them aligned. This is the core reason VisionDoc AI is built on VLMs rather than "OCR + a text model."

5.2 How a computer turns an image into "tokens"

Language models operate on **tokens** - small chunks of text (roughly word-pieces), each mapped to a vector of numbers (an **embedding**) that the network can do math on. To feed an image into the same machinery, we must also turn the image into tokens. That is the job of a **Vision Transformer**.

The Vision Transformer (ViT): images -> patches -> visual tokens

A **Vision Transformer (ViT)** is the standard modern way to encode an image. The idea is surprisingly simple:

- 1 **Cut the image into a grid of small squares called patches** (Qwen uses 28x28-pixel patches). Think of laying a checkerboard over the photo; each cell is one patch.
- 2 **Flatten each patch into a vector of numbers** and pass it through a small learned layer. Each patch now becomes one **visual token** - the visual analogue of a word-piece.
- 3 **Feed all the visual tokens through Transformer layers.** A **Transformer** is a neural network whose defining trick is **self-attention**: every token is allowed to "look at" every other token and decide which ones are relevant to it. So the patch containing the word "Total" can attend to the patch containing "\$412.50" three cells to its right, and the model learns their relationship.

Analogy: Reading a page, your eyes don't process it pixel-by-pixel; they jump between meaningful regions and relate them. Attention is that "relate any region to any other region" ability, made into math.

The output is a sequence of visual tokens - one per patch - each a rich vector summarizing what's in that part of the image *and* how it relates to the rest. Crucially, **more patches = more tokens = more compute and more VRAM** (VRAM is the GPU's memory). Remember this; it is the lever Qwen2.5-VL exposes and section 5.4 returns to it.

Interview Q: Why patches instead of feeding raw pixels?

A: Attention cost grows roughly with the square of the number of tokens. One token per pixel would be astronomically expensive for any real image. Patches (e.g. 28x28) cut token count by three orders of magnitude while preserving local structure inside each patch, making the sequence length tractable for a Transformer.

5.3 Fusing vision and language: one sequence to rule them all

Here is the elegant part. A VLM does **not** run two separate networks that vote at the end. It converts the image into visual tokens and then **places those visual tokens into the same token sequence as the text tokens**, and lets one shared Transformer process the whole thing.

Concretely, the input the language model sees looks like a single flat list:

```
[system tokens] [<image> visual-token visual-token ... visual-token </image>] [question tokens] [answer tokens...]
```

The visual tokens are projected into the same vector space as text embeddings (via a small learned adapter/projection), so from the language model's point of view they are "just more tokens it can attend to." When it generates the answer, its attention reaches back over both the question words *and* the image patches. That single-sequence fusion is what lets it say "the total, that number in the bottom-right box, is \$412.50."

You can see this design directly in how VisionDoc builds a training example in `models/qwen_vl.py`. The `_messages` method assembles a chat-format list where the user turn's `content` is a list mixing an image and text:

```
{
  "role": "user",
  "content": [
    {"type": "image", "image": image},
    {"type": "text", "text": question},
  ],
}
```

The processor later expands that single `{"type": "image", ...}` placeholder into the actual run of visual tokens inside the sequence. A code comment in the file's `collate_train` calls out the subtlety this creates: the prompt length must be measured "*including expanded image tokens*," because one image can become hundreds of tokens.

Gotcha: In interviews people say "the image is one input to the model." More precisely: the image becomes *many* tokens interleaved with text in a *single* sequence. This is why an image can blow up your context length and why the number of image tokens is variable, not fixed.

5.4 Qwen2.5-VL specifics (at a teaching level)

Qwen2.5-VL is the open-source VLM VisionDoc AI uses by default (`Qwen/Qwen2.5-VL-* -Instruct`). Three properties matter for understanding the code.

(a) It is a causal language model

Causal (also "autoregressive") means it generates text **one token at a time, left to right, each new token conditioned on everything before it** - the same way text chat models write. It cannot peek at future tokens. This has a concrete consequence spelled out at the top of `qwen_vl.py` :

```
"Qwen2.5-VL is a causal LM, so generate returns prompt + completion ."
```

Because the model literally continues the prompt, the raw generation output contains your whole prompt followed by the answer. VisionDoc's `decode` method slices the prompt off before returning text:

```
def decode(self, generated_ids, prompt_width):
    trimmed = generated_ids[:, prompt_width:] # drop the echoed prompt
    return self.processor.batch_decode(trimmed, skip_special_tokens=True, ...)
```

The same causal nature drives two padding decisions in the file (padding = filling short sequences to equal length so a batch forms a rectangle):

Phase	Padding side	Why
Training (<code>collate_train</code>)	right	Keeps the prompt prefix aligned across the batch, so per-sample "mask everything up to the answer" masking is correct.
Inference (<code>prepare_inference_inputs</code>)	left	New tokens are appended on the right; left-padding makes those append positions line up across a batch of different-length prompts.

(b) Dynamic resolution: `min_pixels` / `max_pixels` tiling

This is Qwen2.5-VL's signature feature and the most interview-worthy detail. Older vision models forced every image into a fixed square (say 224x224), which crushes a tall receipt or blurs the fine print on a form. Qwen instead uses **dynamic resolution**: it resizes each image to *whatever* dimensions fit within a **pixel budget**, snapped to a multiple of the 28-pixel patch grid (Qwen calls this "smart resize"). More pixels kept => more patches => more visual tokens => sharper text but more VRAM.

You control that budget with two numbers, wired straight into the processor in `Load()` :

```
self.processor = AutoProcessor.from_pretrained(
    processor_id,
    min_pixels=mcfg.min_pixels, # floor: don't shrink below this (keep small text legible)
    max_pixels=mcfg.max_pixels, # ceiling: the main VRAM lever on big scans
    ...
)
```

The project's configs make the trade-off explicit. Note the values are always `N * 28 * 28` , i.e. "at most N patches":

Config	<code>max_pixels</code>	Comment in repo
<code>default.yaml</code>	1,003,520 (<code>1280*28*28</code>)	"caps VRAM on large scans"
<code>cord_qwen.yaml</code>	802,816 (<code>1024*28*28</code>)	"receipts are small"
<code>colab_t4.yaml</code>	602,112 (<code>768*28*28</code>)	"keep VRAM low for receipts"

```
configs/config.py defaults min_pixels = 256*28*28 and max_pixels = 1280*28*28 , with a comment that capping max_pixels is what bounds memory.
```

Interview Q: You're fine-tuning Qwen2.5-VL on a 16 GB GPU and hitting out-of-memory. Name three levers, cheapest first.

A: (1) Lower `max_pixels` - fewer image tokens per sample, directly shrinking the sequence and its attention/activation memory; this is the VLM-specific lever. (2) Turn on gradient checkpointing. (3) Use QLoRA (4-bit weights). The Colab-T4 config in this repo does exactly this: `max_pixels` down to `768*28*28` plus 4-bit loading.

Gotcha: Setting `max_pixels` too low to "save memory" can silently destroy accuracy on dense documents - the fine print literally becomes unreadable to the model. `min_pixels` exists to prevent the opposite failure: a small image being shrunk until its text vanishes. It's a legibility-vs-VRAM dial, not a free speedup.

(c) Where LoRA attaches

VisionDoc fine-tunes Qwen efficiently with **LoRA** (covered fully in Ch. 6). Worth noting here *what* gets trained: the model's `default_lora_target_modules` lists the language-decoder attention and MLP projections (`q_proj` , `k_proj` , `v_proj` , `o_proj` , `gate_proj` , `up_proj` , `down_proj`), and a comment states "**We leave the ViT frozen.**" So the vision encoder keeps its pretrained "how to see" ability unchanged, and adaptation happens in the language half that reasons and answers. On CUDA the loader can wrap this in 4-bit **QLoRA** quantization (`_maybe_quantization_config` builds an `nf4` bitsandbytes config); on Mac (MPS) or CPU it silently skips quantization - details in Ch. 6.

5.5 The OCR-free contrast: Donut

To appreciate why Qwen is the default, compare it with the alternative backbone the project ships, **Donut** (`naver-clova-ix/donut-base`), in `models/donut.py` . Donut is also **OCR-free** - it too reads pixels directly, never running a separate text-transcription step - but its architecture is fundamentally different.

Donut is a **Vision-Encoder-Decoder**: a **Swin image encoder** (a hierarchical vision Transformer) feeds a **BART-style text decoder** (a classic sequence-to-sequence text generator). The repo's own docstring lists the consequences cleanly:

- ****There is no image token.**** Pixels enter the encoder as `pixel_values` ; they are never interleaved into the text sequence the way Qwen's visual tokens are.
- **Generation is seq2seq, not causal continuation.** `generate` returns *only* the decoder's output, seeded by a task/question prompt supplied as `decoder_input_ids` - it does not echo the prompt back the way a causal LM does.
- **It uses task-specific special tokens** like `<s_docvqa>` and `<s_cord-v2>` ; the adapter even adds structural tokens (`<s_question>` , `<s_answer>`) and grows the embedding table to fit them.

Aspect	Qwen2.5-VL (default)	Donut (alternative)
Architecture	Decoder-only causal VLM	Vision-encoder + text-decoder (seq2seq)
Image enters as	Visual tokens in the text sequence	<code>pixel_values</code> into a separate encoder

Aspect	Qwen2.5-VL (default)	Donut (alternative)
Resolution	Dynamic (<code>min/max_pixels</code>)	Fixed square (<code>image_size</code>)
Prompting	Natural-language chat template	Task special tokens (<code><s_docvqa></code> ...)
<code>generate</code> returns	prompt + completion (slice prompt off)	completion only
Role in repo	Preferred, general, instructable	"Lightweight, OCR-free alternative" - proves the pipeline is backbone-agnostic

The Donut adapter's docstring says its real purpose plainly: it "exists mainly to demonstrate that the whole training/eval/serving pipeline is backbone-agnostic: selecting it is a one-line config change." Both backbones implement the same `VisionDocModel` interface (`load` , `collate_train` , `prepare_inference_inputs` , `decode`), so training, evaluation, and serving code never has to know which one is loaded - the concrete class is chosen by a registry keyed off the config (`@register_model("qwen2_5_vl")` vs `@register_model("donut")`).

Interview Q: "OCR-free" is thrown around a lot. What does it actually buy you, and what's the catch?

A: OCR-free means the model maps pixels straight to the answer, so it (1) can't inherit OCR transcription errors, (2) keeps layout/visual cues an OCR string discards, and (3) collapses a two-stage pipeline into one trainable model. The catch: it's a black box over the raw image, so it needs enough resolution to read the text itself (hence `min_pixels`), it can hallucinate plausible-but-wrong values, and you need good evaluation metrics (EM/ANLS/F1 - see Ch. 9) and confidence scoring to catch that.

Key insight: VisionDoc AI standardizes on Qwen2.5-VL because a *causal, instructable* VLM handles open-ended visual question answering, structured extraction, and free-form reasoning through the *same* natural-language interface - you change the task by changing the prompt, not the model head. Donut, tied to fixed task tokens and a seq2seq head, is the lightweight foil that proves the surrounding pipeline is model-agnostic.

5.6 Putting the pieces together

The end-to-end mental model for a single VisionDoc prediction with Qwen2.5-VL:

- 1 Load an invoice image and a question ("What is the total?").
- 2 The processor smart-resizes the image within the `[min_pixels, max_pixels]` budget and cuts it into 28x28 patches.
- 3 The **ViT** turns patches into **visual tokens**; a projection maps them into the language model's embedding space.
- 4 Those visual tokens are **interleaved with the system + question text** into one sequence (built by `_messages` + the chat template).
- 5 The **causal** decoder attends over the whole sequence and generates the answer **left to right**, echoing the prompt.
- 6 `decode` slices the prompt off; the answer text is returned for scoring, confidence estimation, and region highlighting (Ch. 8-9).

With the "what" and "how" of the model settled, Ch. 6 explains *how we cheaply teach it new document skills* with LoRA/QLoRA, and Ch. 4 covers how raw images become the `DocSample` records these

collators consume.

6. Fine-Tuning, Transfer Learning, LoRA and PEFT

This chapter is the conceptual and practical heart of VisionDoc AI. It explains *how* a giant pretrained model is cheaply specialized for reading invoices and receipts, and *why* the project uses a technique called LoRA instead of retraining the whole thing.

6.1 Three ways to make a model do what you want

Before any code, fix three words that people constantly confuse.

- **Pretraining** - the original, hugely expensive training of a model on a massive, general dataset. Qwen2.5-VL was pretrained by Alibaba on billions of image-text and text pairs so it learns general language and general visual understanding. You almost never do this yourself; it costs millions of dollars and thousands of GPU-hours.
- **Fine-tuning** - taking those *already-trained* weights and training them a little more on *your* smaller, specific dataset (here: CORD receipts, DocVQA documents). The model keeps its general knowledge and picks up your task and your output format.
- **Prompting** - not training at all. You just write clever instructions in the input and hope the frozen model already knows enough. Cheap and instant, but you cannot teach it a new output schema or a new domain reliably.

Key insight: Prompting changes the *input*; fine-tuning changes the *weights*. If you need the model to reliably emit a fixed JSON schema for receipts, prompting alone is fragile - fine-tuning bakes the behavior into the parameters.

A **parameter** (also called a **weight**) is just one number inside the model. A modern model has billions of them, arranged in layers of matrices; "training" means nudging these numbers so the model's outputs get closer to the correct answers.

Interview Q: When would you fine-tune instead of just prompting a strong model?

A: When you need a consistent output format, domain-specific behavior the base model gets wrong, lower latency/cost per call, or on-prem/offline deployment. Prompting is better when the task is already within the base model's competence, when you have very little labeled data, or when you need to iterate in minutes. Fine-tuning trades upfront training cost for cheaper, more reliable, more controllable inference.

6.2 Transfer learning: standing on the model's shoulders

Transfer learning is the umbrella idea behind fine-tuning: knowledge learned on one task *transfers* to another. A model that learned to read text in natural images already knows edges, characters, layout, and grammar - so teaching it "extract the total from a receipt" is a small delta, not a from-scratch effort.

The analogy: hiring a fluent, well-read adult and giving them a one-week course on your company's invoice format, versus raising and educating a child from birth just to read your invoices. Transfer learning is the one-week course.

Practically, transfer learning is *why* fine-tuning needs only hundreds or thousands of examples instead of billions. The heavy lifting is already done inside the pretrained weights.

6.3 Why full fine-tuning of a 3B+ multimodal model is expensive

VisionDoc AI's default backbone is `Qwen/Qwen2.5-VL-3B-Instruct` (see `configs/cord_qwen.yaml`) - roughly **3 billion parameters**. "Full fine-tuning" means every one of those parameters is trainable and updated. The problem is memory. Training needs to hold **four** things in GPU memory (VRAM), not one:

What lives in memory	Size for a 3B model (rough)	Why it exists
Weights	~6 GB (bf16, 2 bytes each)	The model itself
Gradients	~6 GB	One number per weight, saying which way to nudge it
Optimizer states	~24 GB	Adam keeps <i>two</i> extra numbers (momentum + variance) per weight, usually in fp32
Activations	varies with batch/sequence	Intermediate outputs saved for backprop

- A **gradient** is the direction and amount to change each weight to reduce error - computed by **backpropagation** (the algorithm that pushes the error backward through the layers).
- The **optimizer** is the rule that applies gradients. The standard one, **Adam**, stores two running statistics per weight so updates are smooth and adaptive - but that means ~8 bytes of optimizer state per parameter, dwarfing the weights themselves.

Add it up: a 3B model in full fine-tuning easily needs **40+ GB of VRAM**, before activations. A free Colab T4 GPU has **16 GB**. That is why `configs/colab_t4.yaml` cannot do full fine-tuning at all - it simply does not fit.

Gotcha: People assume "the model is 6 GB, so a 16 GB GPU is plenty." Wrong. In *training*, optimizer states and gradients multiply the footprint ~5-6x. The weights are the small part of the bill.

6.4 PEFT: the general idea

PEFT = Parameter-Efficient Fine-Tuning. The family of techniques that get fine-tuning results while training only a *tiny fraction* of the parameters - often under 1%. The rest of the model is **frozen** (`requires_grad = False` , meaning "do not compute gradients or update this").

Freezing kills the expensive parts of the memory table above: if a weight is frozen it needs **no gradient and no optimizer state**. So if only 0.5% of weights are trainable, you pay optimizer/gradient cost on 0.5% of the model. That is the whole trick.

HuggingFace ships a library literally named **PEFT**, and VisionDoc AI uses it (`from peft import LoraConfig, get_peft_model` in `models/base.py`). LoRA is the specific PEFT method the project uses.

6.5 LoRA from scratch, with intuition

LoRA = Low-Rank Adaptation. It is the most popular PEFT method, and understanding it well is a common interview filter.

The observation

Every big linear layer in the model is a weight matrix W . Fine-tuning learns a *change* to it, call it ΔW (delta-W), so the new layer is $W + \Delta W$. Full fine-tuning learns a full-sized ΔW - as big as W itself.

The LoRA paper's key claim: the *useful* change ΔW during fine-tuning has **low intrinsic rank**. In plain terms, ΔW is highly redundant - it can be well-approximated by multiplying two much smaller, "skinny" matrices instead of storing the whole thing.

Rank, informally, is how much genuinely independent information a matrix carries. A low-rank matrix looks big but is built from a few underlying patterns - like a mixing board with 1000 sliders that are all secretly controlled by 8 master knobs.

The construction

Replace the full ΔW with a product of two small matrices:

```
DeltaW = B * A          (this is the "B*A" you'll see written as W + BA)
```

If W is a $d \times d$ matrix (say 4096 x 4096), then:

- A has shape $r \times d$ (a few rows, full width)
- B has shape $d \times r$ (full height, a few columns)
- r is the **rank** - a small number like 8, 16, or 32.

The forward pass becomes:

```
h = W*x + (B*A)*x      # frozen W plus the small trainable detour
```

W stays **frozen**. Only A and B are trained. At initialization B is set to zeros (so $B*A = 0$) and the model starts *exactly* as the pretrained model - no shock), while A gets small random values.

Why this trains <1% of parameters

Count the numbers. Full ΔW for a 4096x4096 layer = 16.7M parameters. LoRA with $r = 16$:

```
A: 16 x 4096    = 65,536
B: 4096 x 16    = 65,536
total           = 131,072 ~ 0.8% of 16.7M
```

Across the whole model this is why VisionDoc AI's own logging (`models/base.py`, the `apply_lora` log line and `models/lora.py::format_parameter_summary`) prints something like `trainable=... (0.4xx%)`. The project computes this live:

```
# models/lora.py
def count_parameters(model):
    module = getattr(model, "model", model)
    trainable = sum(p.numel() for p in module.parameters() if p.requires_grad)
    total = sum(p.numel() for p in module.parameters())
    return {"trainable": trainable, "total": total,
            "trainable_pct": 100.0 * trainable / max(total, 1)}
```

Interview Q: Why does LoRA save memory if $W + BA$ still runs the full W ?

A: LoRA does not save on the *frozen forward pass* - W is still there. It saves on **training state**: gradients and optimizer states are only allocated for A and B , which are <1% of the parameters. Since Adam's optimizer states are the biggest memory consumer in full fine-tuning, shrinking the trainable set by 100x is what makes a 3B model trainable on a 16 GB GPU.

6.6 The LoRA hyperparameters - and where they live in the repo

VisionDoc AI's `LoRAConfig` dataclass (`configs/config.py`) and the `lora:` block of `configs/default.yaml` expose exactly the knobs PEFT needs:

```
# configs/default.yaml
lora:
  r: 16
  lora_alpha: 32
  lora_dropout: 0.05
  bias: none
  task_type: CAUSAL_LM
  target_modules: [q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj]
  modules_to_save: null
```

Knob	What it means	Effect of raising it
<code>r</code> (rank)	Width of the <code>A / B</code> bottleneck	More capacity to learn, more trainable params, more memory; too high wastes the whole point
<code>lora_alpha</code>	A scaling factor; the adapter output is multiplied by <code>alpha / r</code>	Amplifies the adapter's influence. Common practice: <code>alpha = 2 x r</code> (here 32 vs 16)
<code>lora_dropout</code>	Randomly zeroes some adapter activations during training	Regularization - fights overfitting (memorizing the training set instead of generalizing)
<code>bias</code>	Whether bias terms are also trained (<code>none / all / lora_only</code>)	<code>none</code> keeps things minimal; the project uses <code>none</code>
<code>task_type</code>	Tells PEFT the head shape; <code>CAUSAL_LM</code> for text generation	Must match the backbone type
<code>target_modules</code>	<i>Which</i> linear layers get an adapter	More modules = more coverage but more params
<code>modules_to_save</code>	Extra layers trained <i>fully</i> (e.g. a new task head)	Use when a component didn't exist during pretraining

On `alpha` : the effective update is $\Delta w = (\alpha / r) * B * A$. This decoupling lets you change `r` without also changing how strongly the adapter speaks - you re-tune one scalar instead of re-tuning learning rates. With `alpha=32, r=16` , the scale is `2.0` .

Gotcha: Bigger `r` is not automatically better. If the true `Delta w` really is low-rank, a large `r` mostly adds parameters that learn noise and can overfit. `r=8-16` is a strong default for most tasks; the repo uses 16.

Which modules are targeted, and why not the vision encoder

Look at the default `target_modules` : `q_proj, k_proj, v_proj, o_proj` are the four projection matrices inside **attention** (query, key, value, output), and `gate_proj, up_proj, down_proj` are the three matrices of the **MLP/feed-forward** block. Together they cover the "thinking" layers of the Qwen **language decoder**.

The repo deliberately does **not** adapt the **ViT** (Vision Transformer, the image-reading half). The docstring in `configs/config.py` says it plainly:

"We deliberately do NOT target the vision encoder by default - fine-tuning only the language side is cheaper and usually sufficient for document QA."

And in `models/qwen_vl.py` :

```
@property
def default_lora_target_modules(self):
    # Language-decoder attention + MLP projections. We leave the ViT frozen.
    return ["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"]
```

The Donut backbone (`models/donut.py`) instead returns `None` , which tells PEFT to **auto-detect** the linear layers in its BART-style decoder - a nice example of the same LoRA config adapting to a different architecture.

Interview Q: Why adapt attention/MLP projections but freeze the image encoder?

A: The pretrained vision encoder already extracts strong visual features; document QA mostly requires teaching the *language* side a new output format and reasoning pattern over those features. Freezing the ViT saves parameters and memory and avoids destabilizing well-trained visual representations. If a task needed genuinely new visual perception (novel document types the encoder can't see), you'd extend targets to vision layers - but that's the exception.

6.7 How the repo actually applies LoRA

The whole thing is one shared method, `VisionDocModel.apply_lora()` in `models/base.py` . Trimmed:

```
def apply_lora(self):
    from peft import LoraConfig, TaskType, get_peft_model
    lcfg = self.config.lora
    targets = lcfg.target_modules or self.default_lora_target_modules
    if self.config.training.gradient_checkpointing and hasattr(
        self.model, "enable_input_require_grads"):
        self.model.enable_input_require_grads()
    peft_config = LoraConfig(
        r=lcfg.r, lora_alpha=lcfg.lora_alpha, lora_dropout=lcfg.lora_dropout,
        bias=lcfg.bias, task_type=getattr(TaskType, lcfg.task_type, TaskType.CAUSAL_LM),
        target_modules=targets, modules_to_save=lcfg.modules_to_save)
    self.model = get_peft_model(self.model, peft_config) # wraps + freezes base
    trainable, total = self.trainable_parameters()
    logger.info("LoRA applied: %s trainable / %s total (%.3f%%)", ...)
```

`get_peft_model()` is the PEFT call that walks the model, injects `A / B` into every `target_module` , and freezes everything else. After this line, `trainable_parameters()` reports the <1% figure. Note the `enable_input_require_grads()` step: with **gradient checkpointing** (a memory-saving trick that recomputes activations instead of storing them), the frozen input embeddings must be nudged to allow gradients to flow into the adapters - a standard PEFT-for-VLMs detail the code handles.

6.8 Adapter files, saving, loading, merging

Because only `A / B` are trained, the thing you save is tiny - a few megabytes, not gigabytes. This saved bundle is the **adapter**.

- **Save** (`save_adapter`): calls `self.model.save_pretrained(path)` , writing `adapter_config.json` (the hyperparameters) and `adapter_model.safetensors` (the `A / B` weights) plus the processor.

- **Load** (`load_adapter`): loads the frozen base, then `PeftModel.from_pretrained(base, path)` snaps the adapter back on. The repo adds friendly validation - it checks the path exists and contains `adapter_config.json`, because PEFT otherwise misreads a missing local path as a HuggingFace Hub repo id and throws a confusing error (a common footgun when you evaluate before training finished writing).
- **Merge** (`merge_and_unload`): folds $B \times A$ back into W to produce a plain, adapter-free model.

```
def merge_and_unload(self):
    if self._lora_applied and hasattr(self.model, "merge_and_unload"):
        self.model = self.model.merge_and_unload() # W <- W + (alpha/r)*B*A
```

Key insight: You can keep the base model on disk *once* and store many small adapters - one per customer, per document type, per language - swapping them in seconds. That "one backbone, many adapters" pattern is a major operational win of LoRA and a great interview point.

Interview Q: Why merge adapters for production instead of always loading them separately?
A: Merging folds $B \times A$ into W , so inference runs a single matmul per layer with **zero adapter overhead** and no PEFT dependency at serving time - lower latency and simpler deployment. The trade-off: a merged model is baked to one adapter, so you lose the hot-swappability. Keep adapters separate when you serve many tasks from one base; merge when you ship a single specialized model.

6.9 QLoRA: LoRA plus 4-bit quantization

The Colab config (`configs/colab_t4.yaml`) sets `load_in_4bit: true`. That is **QLoRA** - the frozen backbone weights are loaded in **4-bit** precision (`nf4`, "4-bit NormalFloat", via the `bitsandbytes` library) while the LoRA adapters train in full precision. The repo's `models/quantization.py::get_bnb_config` builds this, with **double quantization** (quantizing the quantization constants too), straight from the QLoRA paper.

Quantization means storing each weight in fewer bits (4 instead of 16), shrinking the frozen backbone ~4x. That is what makes a 3B VLM *fit* on a 16 GB T4 at all. The weights are dequantized on the fly for each matmul - a small compute cost for a big memory saving. QLoRA is covered in depth in the quantization chapter; here the point is that it stacks *on top of* LoRA and is purely a train-time memory play.

6.10 LoRA vs full fine-tuning: when to use which

	Full fine-tuning	LoRA / PEFT
Trainable params	100%	<1%
VRAM for a 3B model	40+ GB	~10-16 GB (less with QLoRA)
Artifact size	Full model (GBs)	Adapter (MBs)
Multiple tasks	A full copy each	One base + many small adapters
Peak quality ceiling	Slightly higher (more capacity)	Very close on most tasks
Risk of catastrophic forgetting	Higher	Lower (base frozen)
Best when	You have big GPUs, one task, need every last point	Limited hardware, many tasks, fast iteration

Catastrophic forgetting = when fine-tuning overwrites the model's general knowledge to over-specialize. Because LoRA freezes the backbone, the original knowledge is physically preserved, so LoRA forgets less.

Interview Q: Does LoRA reach the same accuracy as full fine-tuning?

A: On most downstream tasks, LoRA matches full fine-tuning within a small margin, and the QLoRA paper showed 4-bit-base + LoRA rivaling 16-bit full fine-tuning. There can be a gap on tasks needing large-magnitude weight changes or genuinely new capabilities, where a higher `r`, more target modules, or (rarely) full fine-tuning helps. For document intelligence like VisionDoc AI, LoRA is the pragmatic default - near-full quality at a fraction of the cost, and adapter swapping is a bonus.

6.11 Mental checklist

- Pretraining = general, once, expensive. Fine-tuning = specialize on your data. Prompting = no weight change.
- Full fine-tuning is memory-bound because of **gradients + optimizer states**, not the weights themselves.
- PEFT freezes the backbone and trains a tiny add-on; **LoRA** is the specific method - inject `B*A` (rank `r`) into chosen linear layers, keep `W` frozen.
- In this repo: `r=16`, `alpha=32`, `dropout=0.05`, targeting the Qwen language decoder's attention + MLP projections; ViT frozen; wired through `apply_lora()` and `LoRAConfig`.
- Adapters save/load/merge cheaply; **QLoRA** adds 4-bit weights to fit tiny GPUs.

See Ch. 5 for the model architectures being adapted, Ch. 7 for the training loop that calls `apply_lora()`, and the quantization chapter for QLoRA internals.

7. Numerical Precision, Quantization, and QLoRA

Every number inside a neural network lives somewhere in the computer's memory, and how many *bits* you spend to store each number decides two things at once: how much GPU memory the model needs, and how accurate its arithmetic is. This chapter is about that trade-off. It is the single most practical piece of ML-systems knowledge, because it is the reason a 3-billion-parameter Vision-Language Model (a "VLM" - a model that reads both images and text; see Ch. 5) can be fine-tuned on a free Google Colab T4 GPU with only 16 GB of memory. We will build from "what is a floating-point number" all the way to the exact settings in VisionDoc AI's `configs/colab_t4.yaml`.

Key insight: A model's parameters (the learned numbers, also called *weights*) are the biggest memory cost. If each weight is smaller, the whole model is smaller. Quantization is the art of making each weight smaller *without* destroying the model's accuracy.

7.1 What a floating-point number actually is

A computer stores a real number like `3.14159` in *floating-point* format - think of scientific notation (`3.14159 x 100`) in binary. Each number is split into three parts:

- **Sign** - 1 bit, positive or negative.
- **Exponent** - how big or small the number is (the *range*, e.g. can it represent `0.00001` or `100000`?).
- **Mantissa** (also called the *fraction* or *significand*) - the actual digits of precision (how *finely* it can distinguish `3.14159` from `3.14160`).

More exponent bits = wider range. More mantissa bits = finer precision. The total number of bits is the memory cost. Here are the four formats that matter for this project:

Format	Total bits	Sign	Exponent	Mantissa	Range	Precision	Bytes/param
fp32 (float32, "full precision")	32	1	8	23	very wide	very fine	4
fp16 (float16, "half precision")	16	1	5	10	narrow	fine	2
bf16 (bfloat16, "brain float")	16	1	8	7	wide (same as fp32)	coarse	2
int8	8	integer	-	-	-128..127	integer steps	1
int4	4	integer	-	-	16 values	very coarse	0.5

Interview Q: fp16 and bf16 are both 16-bit. Why would you ever choose one over the other?

A: They spend their 16 bits differently. fp16 keeps 5 exponent bits and 10 mantissa bits - good precision but a *narrow* range (roughly up to 65,504), so large gradients or activations can *overflow to infinity* and training goes NaN. bf16 keeps 8 exponent bits (identical range to fp32) but only 7 mantissa bits - it almost never overflows, at the cost of coarser precision. bf16 is far more forgiving for training, which is why it is the default when the hardware supports it. The catch is hardware: bf16 needs a newer GPU.

7.2 Why bf16 needs a newer GPU, and why the T4 uses fp16

bf16 is only accelerated on NVIDIA GPUs from the **Ampere** generation onward (A100, RTX 30-series, and later) and on Google's TPUs. The free Colab GPU is a **Tesla T4**, which is the older **Turing** generation - it has *no* bf16 hardware support. If you ask a T4 for bf16, you either crash or fall back to slow emulation.

That is exactly why `configs/colab_t4.yaml` sets:

```
model:
  torch_dtype: float16  # T4 has no bf16; fp16 is the correct choice
training:
  fp16: true            # <-- T4: fp16, not bf16
  bf16: false
```

The repo does not leave this to chance. `utils/device.py` has a `resolve_dtype` helper that *downgrades* bf16 automatically when the hardware can't do it:

```
if chosen == torch.bfloat16 and device.type == "cuda":
    if not torch.cuda.is_bf16_supported():
        logger.warning("bfloat16 unsupported on this GPU; using float16.")
    return torch.float16
```

So even if you copied a bf16 config onto a T4, the code protects you. Contrast this with `configs/default.yaml`, which targets a modern GPU and uses `torch_dtype: bfloat16`, `bf16: true`, `fp16: false`.

Gotcha: bf16 and fp16 are mutually exclusive during training - you enable one or the other.

VisionDoc's `training/trainer.py` guards against configs that accidentally set both: "Both bf16 and fp16 requested; using bf16 and disabling fp16." bf16 wins because of its wider range.

7.3 Mixed-precision training

You might expect training to run entirely in 16-bit. It doesn't. **Mixed-precision training** means the *math* (matrix multiplications, the forward and backward passes) runs in fast 16-bit, but a **master copy of the weights and the optimizer's running statistics are kept in 32-bit** for numerical safety. The framework (PyTorch + HuggingFace Accelerate) shuttles between the two automatically.

Why keep a 32-bit copy? Because weight *updates* are tiny - a gradient step might change a weight by `0.00000001`. In 16-bit, such a tiny change can round to zero and the model stops learning ("underflow"). The fp32 master copy accumulates these tiny changes faithfully.

For fp16 specifically, mixed precision also uses **loss scaling**: it multiplies the loss by a big constant before computing gradients (to push small gradient values up out of fp16's underflow zone) and divides them back afterward. bf16's wide range makes loss scaling largely unnecessary - another reason bf16

is easier.

Interview Q: If we train in fp16 for speed, why is memory not simply halved?

A: Because mixed precision keeps fp32 master weights and fp32 optimizer state (for Adam, that's two extra fp32 tensors per parameter - momentum and variance). So a "16-bit" full fine-tune can actually use *more* memory per parameter than fp32 alone. This is precisely the problem QLoRA solves: freeze the base weights so they need no optimizer state at all, and quantize them to 4-bit.

7.4 Quantization: from 16 bits down to 4

Quantization means mapping continuous floating-point values onto a small set of discrete levels so each value fits in fewer bits.

- **int8** uses 8 bits -> 256 possible levels. You take a block of weights, find their min and max, and stretch that range across the 256 integer slots. A *scale* factor records how to convert back. This shrinks 32-bit weights ~4x and 16-bit weights ~2x.
- **int4 / "4-bit"** uses just 4 bits -> **16 possible levels** per weight. That is aggressive: every weight is snapped to one of only 16 values. Done naively it would wreck accuracy - but QLoRA uses a smarter 4-bit scheme (below) that makes it work.

"4-bit" literally means each stored weight number occupies four bits - half a byte. A weight that was 2 bytes in fp16 becomes 0.5 bytes. That is a **4x memory reduction on the base weights**.

Key insight: Quantization is lossy compression for numbers. The whole game is choosing *where* to place the 16 (or 256) levels so the reconstructed weights are as close as possible to the originals.

7.5 QLoRA: 4-bit base + LoRA adapters

QLoRA (Quantized Low-Rank Adaptation) is the technique that makes fine-tuning a big model on a small GPU possible. It combines two ideas:

- 1 **Freeze the huge base model and store it in 4-bit.** The billions of pretrained weights are read-only during fine-tuning, so they never need gradients or optimizer state - they just need to be *readable* for the forward pass. Storing them 4-bit shrinks them 8x versus fp32.
- 2 **Train tiny LoRA adapters in higher precision on top.** LoRA (covered fully in Ch. 6) inserts small low-rank matrices next to each frozen weight matrix; only these - a fraction of a percent of all parameters - receive gradients. In `colab_t4.yaml` these are `r: 16`, `lora_alpha: 32`, applied to `[q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj]`.

VisionDoc's `models/quantization.py` documents and builds the QLoRA config. Its `get_bnb_config` for 4-bit is:

```
BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",           # NormalFloat-4
    bnb_4bit_use_double_quant=True,     # quantize the scale constants too
    bnb_4bit_compute_dtype=compute_dtype # matmul dtype after dequantization
)
```

Two refinements from the QLoRA paper appear here:

- **nf4 (4-bit NormalFloat).** Instead of spacing the 16 levels evenly, nf4 spaces them to match the *bell-curve* (normal) distribution that trained neural-network weights actually follow - more levels where weights are dense (near zero), fewer out in the tails. This packs information "more informatively than plain int4," as the source comment puts it.
- **Double quantization.** The scale factors used to dequantize each block are themselves stored as numbers - and there are many of them. Double quant quantizes *those constants too*, saving a further ~0.4 bits per parameter for negligible quality loss.

Dequantization on the fly. The 4-bit weights are not used directly for math. For each matrix multiply, `bitsandbytes` *dequantizes* the needed weights back up to `bnb_4bit_compute_dtype` (fp16 on the T4), does the matmul, and discards the temporary. You pay a little compute for a big memory saving. On the T4 the loader (`models/qwen_vl.py`) sets `bnb_4bit_compute_dtype=self.dtype` , which resolves to fp16.

Interview Q: In QLoRA, are the 4-bit base weights ever updated?

A: No. The base weights stay frozen and quantized for the entire run - that's why they need no optimizer state, which is where the memory savings come from. Gradients flow *through* the frozen layers only to reach the LoRA adapters, and only the adapter matrices (kept in higher precision) are updated. At the end you save just the small adapter, not a modified base model.

Gotcha: `bitsandbytes` ships **CUDA-only** kernels. On an Apple-Silicon (MPS) laptop or CPU there is no 4-bit path. The repo handles this gracefully: `_maybe_quantization_config` in `models/qwen_vl.py` returns `None` when `device.type != "cuda"` ("Quantization requested but device is ...; ignoring"), and `get_bnb_config` only *warns* rather than crashing so the config object is still inspectable in CPU unit tests.

7.6 bitsandbytes, and the two different quantization jobs

bitsandbytes is the library that provides the actual 4-bit and 8-bit CUDA kernels; HuggingFace Transformers plugs into it via `BitsAndBytesConfig` . It is worth knowing that VisionDoc uses quantization for *two completely different purposes*, and `models/quantization.py` is explicit about not conflating them:

	QLoRA (<code>get_bnb_config</code>)	Dynamic INT8 (<code>apply_dynamic_quantization</code>)
When	Training time	Inference time
Goal	Fit a big model's <i>training</i> on one GPU	Serve a trained model cheaply on CPU
What's quantized	Frozen base weights -> 4-bit nf4	<code>nn.Linear</code> weights -> int8
Hardware	CUDA only (bitsandbytes)	CPU only (<code>torch.ao.quantization.quantize_dynamic</code>)
Retraining / calibration?	Adapters trained on top	None - "dynamic" scales computed per forward pass

The 8-bit option (`load_in_8bit=True` , `LLM.int8()`) also exists as a "numerically gentler" fallback for tasks where 4-bit hurts quality, but `colab_t4.yaml` keeps `load_in_8bit: false` because 4-bit is what fits comfortably in 16 GB.

7.7 Two more memory levers: gradient checkpointing and accumulation

Quantizing the weights isn't enough on its own - *activations* (the intermediate outputs of every layer, which must be kept around for the backward pass) also eat memory, and so does the batch size. Two techniques stretch the budget further; both are on in `colab_t4.yaml`.

Gradient checkpointing (`gradient_checkpointing: true`). Normally every layer's activations are saved during the forward pass so they're available to compute gradients. Gradient checkpointing *throws most of them away* and **recomputes** them during the backward pass instead. It trades extra compute (roughly one extra forward pass) for a large activation-memory saving - often the difference between fitting and an out-of-memory crash. `training/trainer.py` enables it with `gradient_checkpointing_kwargs={"use_reentrant": False}` (the modern, more robust variant).

Analogy: Instead of photographing every step of a recipe so you can review them later, you keep only a few "checkpoint" photos and re-cook the missing steps when you need them. Slower, but you need far less shelf space.

Gradient accumulation (`gradient_accumulation_steps: 8`). A T4 can only fit `per_device_train_batch_size: 1` - one image at a time. But training is more stable with a larger *effective* batch. Gradient accumulation runs 8 tiny batches, *adds up* their gradients, and only then takes one optimizer step - giving an **effective batch size of $1 \times 8 = 8$** with the memory cost of a batch of 1. (`training/trainer.py` logs exactly this product: `per_device_train_batch_size * gradient_accumulation_steps`.)

Lever in <code>colab_t4.yaml</code>	Value	What it saves
<code>load_in_4bit</code>	true	Base weight memory (~8x vs fp32)
<code>gradient_checkpointing</code>	true	Activation memory (recompute instead of store)
<code>gradient_accumulation_steps</code>	8	Lets tiny batch (1) act like batch 8
<code>per_device_train_batch_size</code>	1	Fits activations for one sample
<code>max_pixels</code>	602112	Caps image tokens -> activation memory

7.8 max_pixels: the vision-specific memory lever

For a VLM there is one more knob that a text-only model doesn't have. Qwen2.5-VL turns an image into a *variable* number of visual tokens depending on the image's resolution - a bigger image becomes more tokens, and every extra token costs memory and compute in the transformer. `colab_t4.yaml` bounds this:

```
min_pixels: 200704 # 256 * 28 * 28
max_pixels: 602112 # 768 * 28 * 28 - keep VRAM low for receipts
```

The `28*28` factor is Qwen's patch/merge granularity: the processor resizes each image so its pixel count lands between these bounds before tiling it into tokens. Capping `max_pixels` caps the worst-case number of visual tokens, which directly caps activation memory - a sensible trade for

receipts and invoices, which are legible at modest resolution. The loader passes these into `AutoProcessor.from_pretrained(...)` ; `colab_t4_fast.yaml` keeps the same pixel budget but shrinks the dataset (`max_train_samples: 64`) and accumulation (`gradient_accumulation_steps: 4`) for a quicker smoke run.

7.9 A concrete memory-math example

Let's estimate why a ~3B-parameter model needs QLoRA on a 16 GB T4. Use **P = 3 x 109 parameters** and remember: fp32 = 4 bytes, fp16/bf16 = 2 bytes, int4 = 0.5 bytes per parameter.

Attempt A - full fine-tune in mixed precision (the naive plan):

Component	Cost	Bytes	GB
Model weights (fp16 for compute)	P x 2	6.0e9	~6
fp32 master weights	P x 4	1.2e10	~12
Adam momentum (fp32)	P x 4	1.2e10	~12
Adam variance (fp32)	P x 4	1.2e10	~12
Subtotal (before activations)			~42 GB

That is ~42 GB just for weights and optimizer state - before a single activation. It cannot fit in 16 GB. This is the "16 bits didn't halve my memory" surprise from §7.3 made concrete: the optimizer state dominates.

Attempt B - QLoRA (what VisionDoc actually does):

Component	Cost	GB
Base weights in 4-bit nf4	P x 0.5	~1.5
LoRA adapters (fp16, well under 1% of P)	tiny	~0.1
Optimizer state - only for adapters	tiny	~0.2
Activations (batch 1 + gradient checkpointing + capped <code>max_pixels</code>)	-	a few
Total		comfortably < 16 GB

The base weights drop from ~12 GB (fp32) to ~1.5 GB, and - the real win - the giant optimizer-state columns *vanish* because the base is frozen. Only the adapters carry optimizer state, and they are a rounding error. That is the whole reason a 3B VLM trains on free Colab.

Interview Q: Walk me through why QLoRA turns a 42 GB job into a sub-16 GB one. Which term dominates the saving?

A: The dominant saving is the **optimizer state**, not the weights. A full Adam fine-tune stores fp32 momentum + variance for every parameter - that alone is ~24 GB for a 3B model. QLoRA freezes the base, so those two tensors exist only for the tiny adapters. Quantizing the frozen base to 4-bit adds a further ~4x cut on the weight term (12 GB -> 1.5 GB). Gradient checkpointing and a batch size of 1 then keep activations small. Together they turn an impossible job into an easy one, at the cost of some recompute and on-the-fly dequantization.

7.10 Putting the T4 config together

Every precision-related line in `configs/colab_t4.yaml` now has a reason:

- `torch_dtype: float16` , `fp16: true` , `bf16: false` - the T4 (Turing) has no bf16 hardware; fp16 is the correct 16-bit choice, and `utils/device.py` would auto-downgrade bf16 anyway.
- `load_in_4bit: true` , `load_in_8bit: false` - QLoRA with 4-bit nf4 + double-quant fits the frozen 3B base in ~1.5 GB.
- `attn_implementation: sdpa` - FlashAttention needs Ampere+, unavailable on the T4, so it uses PyTorch's scaled-dot-product attention instead.
- `gradient_checkpointing: true` + `per_device_train_batch_size: 1` + `gradient_accumulation_steps: 8` - keep activations tiny while getting an effective batch of 8.
- `min_pixels` / `max_pixels` - bound the visual-token count so image activations stay cheap.

Key insight: No single trick makes a 3B VLM trainable on a 16 GB T4 - it's the *stack*: 4-bit frozen base (QLoRA/bitsandbytes) removes the weight and optimizer cost, gradient checkpointing removes activation cost, gradient accumulation recovers batch quality, fp16 matches the hardware, and `max_pixels` tames the vision side. Understanding how these layers compose is the mark of an ML engineer who has actually shipped training on real hardware.

8. Document AI, Datasets, and Evaluation Metrics

This chapter teaches what "document AI" actually means, the public datasets VisionDoc AI learns from, and - the part that trips up most interview candidates - how you *measure* whether a document model is any good. Every metric here is defined from first principles, with a formula and a worked example, and mapped to the exact code in `evaluation/metrics.py` and the loaders in `preprocessing/datasets.py`.

8.1 What is Document AI?

Document AI (also "document intelligence" or "document understanding") is the field of getting a computer to read a *page image* - an invoice, receipt, form, or ID - and turn it into structured, usable information: answering questions about it, pulling out specific fields, or classifying its type.

The key word is *image*. A document is not clean text in a database; it is pixels arranged in a layout, where **position carries meaning**. On a receipt, the number in the bottom-right labelled "TOTAL" is the amount due; the same number elsewhere is a line item. Reading a document well means understanding text *and* where that text sits.

OCR vs OCR-free ("OCR-free" document understanding)

OCR stands for **Optical Character Recognition** - software that scans an image and returns the characters it finds ("\$", "4", ".", "0", "0"). It is the classic first step in a document pipeline.

There are two competing architectures:

Approach	Pipeline	Analogy
OCR-based	Image -> OCR engine (e.g. Tesseract) -> raw text + word boxes -> a text model reasons over it	A translator who first has a typist transcribe the page, then reads the transcript
OCR-free (end-to-end)	Image -> single neural model -> answer directly	A person who just <i>looks</i> at the page and answers

VisionDoc AI is deliberately **OCR-free**. The Vision-Language Model (a neural network that takes an image and text prompt and produces text - see Ch. 5) reads the pixels itself. Both models in the project - Qwen2.5-VL and Donut ("Document understanding transformer") - are OCR-free by design; the name Donut literally advertises it.

Interview Q: Why would you choose an OCR-free model over a traditional OCR-then-NLP pipeline?

A: Three reasons. (1) *Error propagation* - an OCR pipeline's mistakes are frozen in before the language model ever sees them; a misread "8" as "3" can never be recovered downstream. An end-to-end model can use visual context to disambiguate. (2) *Layout loss* - OCR flattens a 2-D page into a 1-D text stream, discarding spatial relationships that matter for tables and forms. (3) *Operational simplicity* - one model, one dependency, no brittle OCR-engine tuning per document type. The trade-offs are that OCR-free models are heavier, need GPUs, and can hallucinate text that isn't there, whereas OCR is cheap and grounded in real pixels.

Gotcha: "OCR-free" does *not* mean "no text supervision." These models are still trained on images paired with the correct text/answer - they just learn to read internally instead of relying on a separate OCR module at inference time.

8.2 The core document-AI tasks

VisionDoc AI supports several task shapes. Every dataset in the project is funneled into one internal record type, `DocSample`, tagged with a `task` field of either `"qa"` or `"extraction"` (see `preprocessing/datasets.py`).

- **Visual Question Answering (VQA):** given a page and a natural-language question ("What is the invoice total?"), produce a short free-form answer ("\$4.00"). `task="qa"`.
- **Key-Information Extraction (KIE) / structured extraction:** given a page, pull out a fixed or open set of fields as structured data, typically JSON (`{"company": "ACME", "date": "2021-05-01", "total": "4.00"}`). `task="extraction"`.
- **Document classification:** label the whole page by type (invoice vs receipt vs ID). Scored by `compute_classification_metrics` in the metrics module.
- **Region highlighting / grounding:** return the bounding boxes where each answer lives (the `boxes` and `box_labels` on a `DocSample`).

Key insight: Structured extraction is *harder to score* than to perform. The model can produce a perfectly correct set of fields in a slightly different JSON string, and a naive string comparison will call it wrong. Section 8.5 is entirely about fixing this.

8.3 The datasets

VisionDoc AI's loaders (`preprocessing/datasets.py`) support four public benchmarks. Each is a real, widely-cited academic dataset - knowing them cold is table stakes in a document-AI interview.

Dataset	Documents	Task	Ground truth shape	task
DocVQA	Scanned pages (industry docs)	Visual QA	Question + list of accepted answers	qa
CORD	Indonesian receipts	Receipt parsing	Nested JSON (<code>gt_parse</code>) of menu/total fields	extraction
SROIE	Scanned receipts	Key-info extraction	4 canonical fields: company, date, address, total	extraction
FUNSD	Noisy scanned forms	Form understanding	Key-value pairs + entity boxes	extraction

DocVQA - Document Visual Question Answering

DocVQA is *the* canonical document-QA benchmark: a page image, a natural question, and - crucially - **several accepted answer strings**, because different human annotators phrase the same answer differently ("May 1 2021" vs "01/05/2021"). The loader keeps *all* of them:

```
answers = _first(row, "answers", "answer", default=[]) # keep every variant
answer = answers[0] if answers else ""                # primary for display
```

This multi-reference design is exactly why ANLS (Section 8.5) scores against the *best-matching* reference. Note the loader comment: DocVQA's official test split is an *unlabelled challenge set*, so the project carves its own test split out of train (registry entry `{"train": True, "validation": True, "test": False}`).

CORD & SROIE - receipt extraction

CORD ("Consolidated Receipt Dataset") ground truth is a JSON tree under a `gt_parse` key. The loader **flattens** the nested structure into a flat dotted-key dict - `{"menu.0.nm": "Coke", "total.total_price": "4.00"}` - which is a far friendlier generation target for a VLM and trivial to score field-by-field:

```
parse = gt.get("gt_parse", gt)          # unwrap CORD's nesting
fields = _flatten_fields(parse)          # {"total.total_price": "4.00", ...}
answer = json.dumps(fields, sort_keys=True) # deterministic JSON string target
```

SROIE ("Scanned Receipts OCR and Information Extraction") task-2 targets four fixed fields: `company`, `date`, `address`, `total` . Its loader is defensively written to accept three different mirror layouts (a JSON blob, flat columns, or token-level tags) because community copies of the same dataset disagree on column names.

FUNSD - Form Understanding in Noisy Scanned Documents

FUNSD is small (~200 forms) but influential: it labels forms as **entities** (`question` , `answer` , `header` , `other`) with **linking** relations that pair a question to its answer. The loader handles both the rich "linked-entity" shape and a token-level `words` / `ner_tags` / `bboxes` shape, converting **BIO tags** into grouped entities.

What is a BIO tag? In token labelling, each word gets a tag: **B**-egin (first word of an entity), **I**-inside (continuation), or **O**-outside (not part of any entity). "New York City" as a location becomes `B-LOC I-LOC I-LOC` . The loader strips the `B-` / `I-` prefix and concatenates words sharing an entity to rebuild "New York City".

Interview Q: Why does the project normalize four different datasets into one `DocSample` schema?

A: Decoupling. The model adapters, trainer, evaluator, and API never touch a raw HuggingFace row - they only see `DocSample` . Swapping the training corpus is a one-line YAML change (`data.dataset_name`) with zero downstream code edits. It also centralizes cross-cutting concerns like deterministic split-carving and sample caps, so every dataset gets reproducible partitions for free.

Gotcha (data leakage): When a dataset lacks a labelled val/test split, VisionDoc carves one from train using a *seeded* permutation (`random.Random(config.seed).shuffle`). This matters enormously: an un-seeded or overlapping split would let training data leak into evaluation, inflating every metric below. Reproducible, disjoint splits are non-negotiable for trustworthy eval numbers.

8.4 Evaluation metrics from scratch

Before the document-specific metrics, four foundations everyone must know cold. Imagine a task where each answer is either right or wrong, and we compare predictions to a "gold" truth.

Confusion-matrix building blocks

For a given class (say, "is this field present and correct?"):

- **True Positive (TP)**: predicted yes, actually yes.
- **False Positive (FP)**: predicted yes, actually no (a false alarm).
- **False Negative (FN)**: predicted no, actually yes (a miss).
- **True Negative (TN)**: predicted no, actually no.

Accuracy, Precision, Recall, F1

Accuracy = fraction of *all* predictions that were correct:

$$\text{accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

Accuracy is misleading when classes are imbalanced. If 99% of emails are not spam, a model that says "never spam" scores 99% accuracy while catching zero spam. Hence precision and recall.

Precision = of the items I flagged positive, how many were right? *"When it speaks, is it correct?"*

$$\text{precision} = \text{TP} / (\text{TP} + \text{FP})$$

Recall (a.k.a. sensitivity) = of the items that were actually positive, how many did I catch? *"Does it find everything?"*

$$\text{recall} = \text{TP} / (\text{TP} + \text{FN})$$

There is a tension: you can get perfect recall by flagging *everything* (but precision collapses), or high precision by flagging only sure things (but recall collapses). **F1** is the *harmonic mean* that balances them - harmonic, not arithmetic, so it punishes lopsided scores:

$$\text{F1} = 2 * \text{precision} * \text{recall} / (\text{precision} + \text{recall})$$

Worked example. Gold has 10 real fields. The model outputs 8 fields, 6 of them correct.

- TP = 6, FP = 2 (wrong outputs), FN = 4 (missed real fields).
- precision = 6/8 = 0.75, recall = 6/10 = 0.60.
- F1 = 2 * 0.75 * 0.60 / (0.75 + 0.60) = 0.90 / 1.35 ~ **0.667**.

Note F1 (0.667) sits below the arithmetic mean (0.675) - the harmonic mean always leans toward the smaller number.

Micro vs macro averaging. *Macro* averages the F1 of each class equally (each field type counts the same regardless of frequency). *Micro* pools all TP/FP/FN counts first, then computes one F1 (frequent classes dominate). In the code, `compute_classification_metrics` uses **macro** (`average="macro"`) so rare document types aren't ignored, while `structured_field_metrics` uses **micro** - pooling counts across the whole corpus - so documents with many fields aren't over-weighted. Knowing when to use which is a classic interview question.

Exact Match (EM)

Exact Match is the strictest metric: score 1.0 only if the prediction string equals a reference string *exactly*, else 0.0. But "exactly" after **normalization**. `normalize_text` in the code applies SQuAD-style cleanup so surface noise doesn't cause false failures:

- 1 lowercase
- 2 replace punctuation with spaces
- 3 drop the articles `a / an / the`
- 4 collapse repeated whitespace

So "Total: \$4.00" -> "total 4 00" and "total 4 00" both normalize identically. Because DocVQA supplies multiple references, EM credits a match against *any* of them:

```
def exact_match(pred, refs):
    pred_norm = normalize_text(pred)
    return 1.0 if any(pred_norm == normalize_text(r) for r in refs) else 0.0
```

Gotcha: EM after normalization still fails on "\$4.00" vs "4 dollars" - they're semantically equal but not string-equal. EM is *unforgiving of paraphrase*. That's the motivation for ANLS.

ANLS - the DocVQA metric

ANLS = Average Normalized Levenshtein Similarity. It is the official DocVQA metric and the one every document-VQA paper reports. Build it up from the inside out:

Levenshtein distance (edit distance) = the minimum number of single-character insertions, deletions, or substitutions to turn one string into another. "kitten" -> "sitting" needs 3 edits (k->s, e->i, add g), so distance = 3.

Normalized Levenshtein (NL) scales that into $[0, 1]$ by dividing by the longer string's length - so length doesn't dominate:

```
NL(a, b) = levenshtein(a, b) / max(len(a), len(b))
```

NL = 0 means identical, NL = 1 means maximally different. **Similarity** is $1 - \text{NL}$. ANLS for one sample takes the best similarity over all references, then applies a **threshold gate** (default 0.5): a near-miss earns partial credit, but anything below the threshold is zeroed:

```
def anls(pred, refs, threshold=0.5):
    best = max(1.0 - _normalized_levenshtein(normalize_text(pred),
                                             normalize_text(r)) for r in refs)
    return best if best >= threshold else 0.0
```

Worked example. Prediction "Mircrosoft" (typo), reference "Microsoft" (9 chars). One substitution -> distance 1. NL = 1/9 ~ 0.111, similarity ~ **0.889** >= 0.5, so ANLS credits ~0.889. EM would have scored a flat 0. That's the whole point.

Interview Q: Why is ANLS preferred over Exact Match for DocVQA?

A: DocVQA answers are read off scanned images, so predictions differ from gold by minor, blameless variations - OCR-style character slips, spacing, casing. EM's all-or-nothing scoring throws away a near-perfect answer over a single character, which under-reports true model quality and gives a noisy training signal. ANLS grants graded partial credit via edit-distance similarity while its 0.5 threshold still zeros genuinely wrong answers, so it doesn't leak similarity points to garbage. It also scores against the best of multiple annotator references, matching how the data was actually labelled.

Key insight: The threshold is a deliberate anti-cheat. Without it, an answer that shares a few letters with the gold would leak fractional credit; the gate says "be at least half-right or get nothing."

BLEU and ROUGE - n-gram overlap metrics

These come from machine translation and summarization and appear in the roll-up for longer, free-form answers. An **n-gram** is a run of n consecutive words; a **2-gram (bigram)** of "the total due" is {"the total", "total due"}.

- **BLEU** (Bilingual Evaluation Understudy) is **precision-oriented**: of the n-grams the model produced, how many appear in a reference? It multiplies precisions for $n=1..4$ and applies a **brevity penalty** so a model can't cheat by emitting one safe word. Reported on a 0-100 scale. The code prefers `sacrebleu`, falls back to `nltk`, else returns 0.
- **ROUGE** (Recall-Oriented Understudy for Gisting Evaluation) is **recall-oriented**: of the n-grams in the reference, how many did the model recover? `compute_rouge` returns ROUGE-1 (unigrams), ROUGE-2 (bigrams), and ROUGE-L (longest common subsequence).

BLEU vs ROUGE in one line: BLEU asks "is what I said correct?" (precision); ROUGE asks "did I cover everything?" (recall). For short document answers both are weak signals - they exist in the roll-up for completeness, not as the headline metric.

Gotcha (the code's own brevity-penalty fix): `compute_bleu` refuses to pad missing references with empty strings, because a zero-length reference corrupts sacrebleu's brevity penalty and *inflates* corpus BLEU on short-answer data. It repeats a genuine reference instead. Subtle, and exactly the kind of detail that separates a careful evaluation engineer from a careless one.

8.5 Why string metrics are unfair to JSON - and field-level F1

Here is the crux for structured extraction. Suppose the gold answer is:

```
{"company": "ACME", "date": "2021-05-01", "total": "4.00"}
```

and the model outputs the *semantically identical* fields but as a different string - reordered keys, extra whitespace, `"$4.00"` instead of `"4.00"` :

```
{"total": "$4.00", "company": "ACME", "date": "2021-05-01"}
```

- **Exact Match:** the two JSON *strings* differ -> **0.0**. Brutally wrong verdict.
- **ANLS:** edit distance between the two long strings is large -> low or zeroed. Also wrong.

The problem: EM and ANLS treat the answer as one opaque string, so *key ordering*, *whitespace*, and *one bad field out of ten* all tank the whole score. That is both unfair and useless as a training signal.

The fix is **field-level F1** - score the *set of (key, value) pairs*, not the string. This is exactly `structured_field_metrics` :

```
# A predicted field is a TP only if the (normalized) key exists in gold
# AND its (normalized) value matches. Micro-averaged across the corpus.
if norm_key in gold_norm and gold_norm[norm_key] == _norm_field_value(value):
    tp += 1
precision = tp / n_pred          # of fields I output, how many correct?
recall    = tp / n_gold          # of gold fields, how many I recovered?
f1        = 2*precision*recall / (precision + recall)
```

Both keys and values are normalized first (`_norm_field_value` : strip, lowercase, collapse whitespace) so `"$4.00 "` vs `"4.00"` don't count as a mismatch on formatting alone. Now the reordered-JSON example above scores field-F1 near **1.0** - the correct verdict.

Interview Q: Your model extracts 9 of 10 invoice fields perfectly but formats the JSON slightly differently from the gold string. Exact Match reports 0%. What's wrong and how do you fix it?

A: EM is scoring the serialized *string*, so key order and whitespace dominate and one imperfect field zeroes the whole document - it's measuring formatting, not extraction. The fix is to *parse both sides into field dictionaries and compute F1 over (key, value) pairs* with normalized values. That reports ~90% (9/10), which reflects reality, gives credit per field, and pinpoints *which* field failed. In VisionDoc this is `structured_field_metrics`, micro-averaged across the corpus so documents with many fields don't dominate.

Key insight: Choose the metric to match the *answer shape*. Free-form QA -> ANLS. Short exact fields -> EM/ANLS. Multi-field structured output -> field-level F1. The `aggregate_qa_metrics` roll-up computes several at once (`exact_match` , `anls` , `token_f1` , `precision` , `recall` , `bleu` , `rouge*`) and always returns a stable key schema - even a degraded metric returns 0.0, never a missing key - so downstream report tables never break. As the module `docstring` puts it, no single scalar tells the whole story.

Token-level F1 (a middle ground)

Between exact string match and field match sits **token-level F1** (`token_level_prf`): treat each answer as a *bag of words* and compute F1 over word overlap using multiset counts. Predicting "New York" when gold is "New York City" scores partial credit (precision 1.0, recall 0.67, F1 0.8) instead of a flat zero. This is the SQuAD token-F1 formulation and rewards partially-correct free-form answers.

8.6 Systems metrics - latency, throughput, VRAM

A model that is accurate but too slow or too memory-hungry to deploy is a failed product. VisionDoc measures three **systems metrics** in `evaluation/profiler.py`, exposed as a `ProfileResult`:

Metric	What it measures	Field in code
Latency	Wall-clock time for <i>one</i> inference, in ms. Reported as mean and percentiles.	<code>latency_ms_mean</code> , <code>latency_ms_p50</code> , <code>latency_ms_p90</code>
Throughput	How many samples per second the model sustains under load.	<code>throughput_samples_s</code>
Peak VRAM	The most GPU memory the model pinned during the run, in GB.	<code>peak_memory_gb</code>

Two details worth stating in an interview:

- **Percentiles beat averages.** The profiler reports **p50** (median) and **p90** latency, not just the mean. A user cares about *tail* latency - the slow requests. A great mean with a terrible p90 means one in ten users waits far too long. **VRAM** ("Video RAM") is GPU memory; it caps how big a model and batch you can run.
- **Warmup passes.** `measure_latency` runs untimed warmup iterations first. The first inference includes one-time costs (CUDA kernel compilation, lazy allocation, caches filling) that would "slander the model" if counted - a cold first call can be 10x the steady-state latency. It also resets the CUDA peak-memory counter on entry so `peak_memory_gb` reflects *this* run's inference, not leftover memory from model loading.

Interview Q: You benchmark a model and the first request takes 4 seconds but the next hundred take 200 ms each. What's happening and how do you benchmark honestly?

A: The 4 s is cold-start overhead - CUDA context creation, kernel autotuning/compilation, lazy weight allocation, and cache warming - none of which recur on subsequent calls. Reporting it as "latency" misrepresents steady-state serving performance. The honest approach is warmup: run several untimed inferences to reach steady state, then measure, and report the *distribution* (p50/p90/p99) rather than a mean, because tail latency drives real user experience and SLOs. VisionDoc's profiler does exactly this, and separately captures peak VRAM after resetting the counter so model-load memory isn't blamed on inference.

Key insight: Production evaluation is two-dimensional. *Quality* metrics (ANLS, field-F1) tell you if the answers are right; *systems* metrics (latency, throughput, VRAM) tell you if you can afford to serve them. A senior engineer always reports both, because the deployment decision depends on the whole picture - and techniques from Ch. 6 (QLoRA/quantization) exist precisely to move the VRAM number down. Ch. 9 covers how these metrics flow into the evaluation report and dashboards.

9. Core Tools I - Python and PyTorch

Every line of VisionDoc AI is written in Python, and every number the model learns lives inside a PyTorch tensor. Before you can understand training, LoRA, or inference, you need to understand these two foundations. This chapter teaches both from zero: what they are, why an ML project cannot avoid them, and exactly how they show up in the real code.

9.1 Python - the lingua franca of machine learning

What it is. Python is a general-purpose programming language known for readable, English-like syntax. You write `for sample in dataset:` instead of a wall of braces and semicolons. It is *interpreted* (run line by line, no separate compile step) and *dynamically typed* (a variable's type is decided at runtime, not declared up front).

Why ML lives in Python. This surprises beginners: Python is actually *slow* at raw number-crunching. The trick is that Python is a thin, friendly *steering wheel* on top of fast engines written in C, C++, and CUDA (NVIDIA's language for programming GPUs). When you call `torch.matmul(a, b)` to multiply two matrices, Python spends microseconds dispatching the call, and then a heavily optimized C++/CUDA kernel does the actual millions of multiplications. So you get C-speed math with Python-speed *development*. Combine that with the richest ecosystem of ML libraries anywhere - PyTorch, HuggingFace, NumPy, Pandas - and Python became the default.

Interview Q: Python is slow, so why is it the dominant language for deep learning?

A: Because the performance-critical work isn't done in Python. Frameworks like PyTorch are Python *bindings* over compiled C++/CUDA kernels. Python orchestrates - building the model, looping over data, logging - while the heavy tensor math runs in optimized native code on the GPU. You get fast iteration and readable research code without paying a speed penalty where it matters. The ecosystem (Transformers, PEFT, Datasets) and community momentum then reinforce the choice.

Type hints

Python doesn't *require* you to declare types, but modern codebases add **type hints** - annotations that document what a function expects and returns. They don't change runtime behavior; they're read by humans and by tools like `mypy` (a static type checker) and your editor's autocomplete.

Look at `resolve_device` in `utils/device.py` :

```
def resolve_device(preference: str = "auto") -> torch.device:
    ...
```

`preference: str` says "pass a string"; `-> torch.device` says "you get back a `torch.device` object." The newer `X | None` syntax (a *union type* meaning "an X or nothing") appears throughout, e.g. `device: torch.device | None = None`. In `models/base.py`, richer hints appear like `Sequence[DocSample | dict]` (a list-like of either a `DocSample` or a plain dict) and `dict[str, torch.Tensor]` (a dictionary from strings to tensors).

Key insight: Type hints are the cheapest documentation you can write. In a project where a batch is "a dict with `input_ids`, `attention_mask`, `labels`, `pixel_values`," the hint `dict[str, torch.Tensor]` instantly tells the next engineer the shape of the contract without reading the body.

Dataclasses

A **dataclass** is a small decorator (`@dataclass`) that turns a class into a plain data container. It auto-generates the boilerplate `__init__` , `__repr__` , and equality methods so you just list the fields. VisionDoc uses them for structured records. From `utils/device.py` :

```
@dataclass(frozen=True)
class DeviceInfo:
    device: str          # "cuda" | "mps" | "cpu"
    device_name: str
    dtype: str
    n_gpus: int
    total_memory_gb: float
```

`frozen=True` makes instances *immutable* - once created you can't reassign a field, which is exactly what you want for a "snapshot of the environment" that gets logged into an evaluation report. In `models/base.py` , `GenerationOutput` is a (mutable) dataclass holding a model's answer plus its confidence, token ids, and latency. The `field(default_factory=list)` there means "each new instance gets its own fresh empty list" - a critical detail, because using a plain `= []` default would share one list across all instances (a classic Python bug).

Gotcha: Never write `def f(x=[])` or a dataclass field `items: list = []` . Mutable default arguments are created *once* and shared. Use `field(default_factory=list)` (dataclass) or `x=None` then build inside the function.

Packaging with `pyproject.toml` and virtual environments

A **virtual environment** (venv) is an isolated folder holding one project's Python and its exact library versions, so Project A's `torch==2.2` can't collide with Project B's `torch==2.6` . You create and activate one, then install into it:

```
python -m venv .venv
source .venv/bin/activate
pip install -e .          # editable install of this project
```

`pyproject.toml` is the modern standard file describing how a project is built and installed. VisionDoc's declares its name, that it needs Python `>=3.11` , and its core dependencies:

```
[project]
name = "visiondoc-ai"
requires-python = ">=3.11"
dependencies = [
    "torch>=2.2.0",
    "transformers>=4.49.0",
    "peft>=0.13.0",
    "accelerate>=0.34.0",
    ...
]
```

The `-e` in `pip install -e .` means **editable install**: the package points back at your source folder, so edits take effect immediately without reinstalling. The project's own comment explains *why* it bothers: installing as a package lets every module use absolute imports like `from models.base import VisionDocModel` no matter what directory a training job is launched from - essential when `accelerate` or `torchrun` (distributed launchers, see Ch. 11) start your script from arbitrary paths. The file also registers console commands via `[project.scripts]` , so `visiondoc-train` becomes a runnable command mapped to `training.train:main` .

Interview Q: What problem does a virtual environment solve, and how is it different from Docker?
A: A venv isolates *Python packages* for one project so version conflicts don't happen between projects on the same machine. Docker isolates the *entire OS-level environment* - system libraries, CUDA runtime, the Python interpreter itself - into a portable image. Venvs give reproducibility on one machine; Docker gives reproducibility across machines. VisionDoc uses both: a venv for local dev and Docker (Ch. 13) for shippable, GPU-ready containers.

9.2 PyTorch - the deep-learning engine

What it is. PyTorch is the library that does the actual math of neural networks. It provides three things: (1) **tensors** - fast multi-dimensional arrays that can live on a GPU; (2) **autograd** - automatic calculus that computes how to nudge each weight to reduce error; and (3) `nn.Module` - building blocks for assembling networks. Every other tool in this project (Transformers, PEFT, Accelerate) is built *on top of* PyTorch.

Tensors

A **tensor** is an n-dimensional array of numbers - the universal data type of deep learning. A single number is a 0-D tensor (scalar); a list is 1-D (vector); a table is 2-D (matrix); a batch of RGB images is 4-D (batch, channels, height, width). A tensor knows three things about itself: its **shape** (dimensions), its **dtype** (number format, e.g. `float32`), and its **device** (where it lives - CPU or GPU).

```
import torch
x = torch.randn(2, 3)          # 2x3 tensor of random floats
x.shape                       # torch.Size([2, 3])
x.dtype                       # torch.float32
x.device                      # device(type='cpu')
```

You see shapes read directly in the codebase. In `models/base.py`, `prompt_width = int(inputs["input_ids"].shape[1])` grabs the *sequence-length* dimension of the token-id tensor; `batch = sequences.shape[0]` grabs the *batch* dimension. Understanding "which axis is which" is half of practical PyTorch.

dtype - the number format

A **dtype** is the numeric precision of each element. It's a memory-vs-accuracy tradeoff:

dtype	bits	range	typical use
<code>float32</code> (fp32)	32	wide	safe default, CPU
<code>float16</code> (fp16)	16	narrow	GPU speedup, can overflow
<code>bfloat16</code> (bf16)	16	wide (fp32-like)	modern GPU training, fewer numeric issues

`utils/device.py`'s `resolve_dtype` maps config strings (`"bf16"`, `"fp16"`, `"float32"`) to real `torch.dtype` objects, and - crucially - *downgrades safely* based on hardware: `bfloat16` is only kept on CUDA GPUs that actually support it (`torch.cuda.is_bf16_supported()`), otherwise it drops to `float16` or `float32`. Off-GPU, `bf16` matmul is poorly supported, so it falls back to `float32`. This is why the project never hard-codes a dtype.

`.to(device)` and CUDA

A **GPU** (Graphics Processing Unit) does thousands of arithmetic operations in parallel, which is why training runs 10-100x faster on one. **CUDA** is NVIDIA's platform for using their GPUs. A tensor or model must be explicitly *moved* onto the device before you can compute with it, using `.to(device)`:

```
device = torch.device("cuda")
x = x.to(device)          # copy tensor to GPU
model = model.to(device)  # move all model weights to GPU
```

A tensor and the model must be on the same device, or PyTorch raises a "expected all tensors on the same device" error. VisionDoc centralizes device choice in `resolve_device` (`utils/device.py`), whose "auto" mode prefers CUDA, then Apple's **MPS** (Metal Performance Shaders - GPU acceleration on Apple-Silicon Macs), then CPU. In `models/base.py`, the base class stores `self.device = resolve_device(config.device)` in its constructor and later moves adapters onto it (`self.model.to(self.device)` inside `load_adapter`). The design goal, stated in the file's docstring, is that no other module ever hard-codes "cuda" - the same code runs on a training GPU box, a laptop, or a CPU-only CI runner.

Interview Q: What's the difference between `torch.cuda.is_available()` returning False and a runtime device-mismatch error?

A: `is_available()` is a *capability check* - is there a usable CUDA GPU at all? A device-mismatch error happens at compute time when two tensors (say, input on CPU and weights on GPU) are operated on together. The fix for the first is to fall back to CPU/MPS gracefully (what `resolve_device` does); the fix for the second is to `.to(device)` every input, which VisionDoc handles by moving inference inputs onto `self.device` in `prepare_inference_inputs`.

Autograd and backpropagation - how a network learns

This is the heart of deep learning. A neural network is a giant function with millions of tunable numbers called **parameters** (or weights). Training means: feed in data, measure how wrong the output is with a **loss** (a single number - bigger means worse), then adjust every parameter slightly in the direction that reduces the loss. Repeat millions of times.

The "which direction reduces loss" question is answered by **gradients** - the calculus derivative of the loss with respect to each parameter. Computing gradients by hand for millions of parameters is impossible, so PyTorch does it automatically with **autograd**. As your data flows *forward* through the network, PyTorch silently records every operation into a graph. When you call `.backward()` on the loss, it walks that graph *in reverse*, applying the chain rule to compute every gradient. This reverse pass is **backpropagation** ("backprop").

```
w = torch.tensor([2.0], requires_grad=True)  # a trainable weight
loss = (w * 3 - 1) ** 2                      # forward pass, graph recorded
loss.backward()                             # backprop: compute d(loss)/dw
w.grad                                       # the gradient, ready for the optimizer
```

`requires_grad=True` is the switch that tells autograd "track this tensor." This exact flag is why LoRA (Ch. 5) saves so much memory: freezing the base model means setting `requires_grad=False` on its weights so autograd builds *no* gradient graph for them. VisionDoc's `trainable_parameters()` in `models/base.py` literally counts this: `sum(p.numel() for p in self.model.parameters() if p.requires_grad)` - the numerator is "params autograd will train," and after `apply_lora()` it's a tiny fraction of the total.

Key insight: "Forward pass = predict and record; backward pass = compute gradients; optimizer step = apply them." Every training loop, from a two-line toy to Qwen2.5-VL fine-tuning, is that same three-beat rhythm.

`nn.Module` - the network building block

`torch.nn.Module` is the base class for every neural-network component. You subclass it, register sub-layers, and define a `forward()` method describing the computation. PyTorch then tracks all its parameters automatically. A Qwen or Donut model is one big `nn.Module` made of thousands of nested ones.

```
import torch.nn as nn

class TinyNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(10, 2)  # a learnable layer
    def forward(self, x):
        return self.fc(x)          # define the computation
```

VisionDoc's `VisionDocModel` (in `models/base.py`) is *not* itself an `nn.Module` - it's an abstract **wrapper** (`ABC` = Abstract Base Class, a class you can't instantiate directly, only subclass). It *holds* the real `nn.Module` in `self.model` and adds project-specific behavior around it: loading, LoRA plumbing, generation, confidence scoring. Methods like `.eval()`, `.parameters()`, `.to()`, and `.save_pretrained()` are called on that inner module. This "wrapper owns a module" pattern keeps backbone-specific code (Qwen vs Donut) swappable behind one interface.

The training loop

Although VisionDoc delegates the actual loop to HuggingFace's `Trainer` (Ch. 11), you must know what that loop contains, because interviewers ask you to write it:

```
model.train()  # training mode (dropout on)
for batch in dataloader:
    batch = {k: v.to(device) for k, v in batch.items()}
    optimizer.zero_grad()  # clear old gradients
    outputs = model(**batch)  # forward pass
    loss = outputs.loss
    loss.backward()  # backprop: fill .grad
    optimizer.step()  # nudge every weight
```

The **optimizer** is the algorithm that turns gradients into weight updates. The default in transformers is **AdamW** - Adam with proper weight decay - which adapts the step size per-parameter and is the workhorse for fine-tuning. `zero_grad()` is essential: PyTorch *accumulates* gradients by default, so you must clear them each step or updates from previous batches leak in.

Gotcha: Forgetting `optimizer.zero_grad()` is a classic bug - gradients pile up across batches and training diverges. Forgetting `model.train()` / `model.eval()` is another: it silently leaves dropout and batch-norm in the wrong mode.

`model.eval()` vs `model.train()` and `torch.no_grad()`

Some layers behave differently during training vs inference. **Dropout** (randomly zeroing some activations to prevent overfitting) is active in training, off at inference. You switch with `model.train()` / `model.eval()`. VisionDoc's `to_eval()` helper wraps exactly this:

`self.model.eval()` - "put the model in eval mode (no dropout) for inference/evaluation."

Separately, at inference you don't need gradients at all, and building the autograd graph wastes memory and time. `torch.no_grad()` turns off gradient tracking. VisionDoc decorates its whole `generate()` method with it:

```
@torch.no_grad()
def generate(self, images, questions, ...):
    ...
    out = self.model.generate(**inputs, **gen_config)
```

The `@torch.no_grad()` decorator means every tensor op inside `generate` skips graph-building - the right call for pure inference.

Interview Q: What's the difference between `model.eval()` and `torch.no_grad()` ? Do you need both?

A: They're orthogonal. `model.eval()` changes *layer behavior* - dropout off, batch-norm uses running statistics. `torch.no_grad()` changes *autograd* - it stops recording operations, saving memory and time. For inference you want both: `eval()` so the network behaves deterministically, and `no_grad()` so you don't build a useless gradient graph. Using only one is a subtle bug - `no_grad()` alone still leaves dropout active.

Mixed precision: autocast and GradScaler

Mixed-precision training runs most operations in fast 16-bit precision while keeping a few sensitive ones (like the loss) in 32-bit, giving big speed and memory wins with almost no accuracy loss. Two tools implement it:

- `autocast` - a context manager that automatically picks 16-bit for safe ops (matmuls) and 32-bit for risky ones.
- `GradScaler` - only needed for `float16` : it multiplies the loss by a large factor before backprop so tiny gradients don't *underflow* to zero in 16-bit, then unscales before the optimizer step. `bfloat16` has a wide enough range that it needs *no* scaler.

```
scaler = torch.cuda.amp.GradScaler()
with torch.autocast(device_type="cuda", dtype=torch.float16):
    loss = model(**batch).loss
scaler.scale(loss).backward()
scaler.step(optimizer)
scaler.update()
```

VisionDoc doesn't write this by hand - the HuggingFace `Trainer` does it internally when you set `bf16=True` or `fp16=True` . In `training/trainer.py` the config's `bf16 / fp16` flags are passed straight through, and if a config sets *both* the code prefers bf16 (wider dynamic range, no scaler needed) and disables fp16. The dtype-resolution logic in `utils/device.py` is the companion piece that ensures bf16 is only requested where the hardware supports it.

Interview Q: Why does fp16 training need a GradScaler but bf16 doesn't?

A: Both are 16-bit, but they spend those bits differently. fp16 has more precision bits but a *narrow* exponent range, so small gradients underflow to zero - GradScaler multiplies the loss up to keep them representable, then unscales. bf16 has the *same wide exponent range* as fp32 (fewer precision bits), so gradients rarely underflow and no scaling is needed. That's why bf16 is the preferred choice on modern GPUs and why VisionDoc prefers it when both flags are set.

Gradient checkpointing - trading compute for memory

Normally the forward pass stores every intermediate activation so backprop can reuse them. For a large VLM that's a huge memory cost. **Gradient checkpointing** stores only a few checkpoints and *recomputes* the rest during the backward pass - you pay ~30% extra compute to fit a much bigger model or batch in the same GPU memory.

There's a subtle interaction with frozen models: if the base weights don't require gradients, the recomputed graph can lose its connection to the trainable LoRA weights. The standard fix is to force the input embeddings to require grad. VisionDoc does exactly this in `apply_lora()` (`models/base.py`):

```
if self.config.training.gradient_checkpointing and hasattr(
    self.model, "enable_input_require_grads"
):
    self.model.enable_input_require_grads()
```

The `Trainer` then enables checkpointing itself, passing `gradient_checkpointing_kwargs={"use_reentrant": False}` (the modern, more robust implementation) as seen in `training/trainer.py`.

Gotcha: Enabling gradient checkpointing on a LoRA/frozen model *without* `enable_input_require_grads()` produces the infamous "element 0 of tensors does not require grad" error, or silently trains nothing. The two features must be wired together - which is why VisionDoc guards them in one place.

Saving and loading

PyTorch's raw primitive is `torch.save(obj, path)` / `torch.load(path)`, which serialize a tensor or a **state dict** (a plain dictionary mapping layer names to weight tensors). But higher-level libraries add friendlier methods. VisionDoc works at the HuggingFace/PEFT level:

- **Save a LoRA adapter** - `save_adapter()` calls `self.model.save_pretrained(path)`, writing only the tiny adapter weights plus an `adapter_config.json`, and also saves the processor. Because LoRA is small, an adapter is megabytes, not gigabytes.
- **Load an adapter** - `load_adapter()` attaches a trained adapter onto the loaded base model with `PeftModel.from_pretrained(self.model, path)`, then `.to(self.device).eval()`. It first *validates the path*, because PEFT otherwise misreads a missing local folder as a Hub repo id and throws a confusing error - so the code checks for `adapter_config.json` and raises a clear message instead.
- **Merge for deployment** - `merge_and_unload()` folds the LoRA math back into the base weights for faster, adapter-free inference (Ch. 6).

Interview Q: When you fine-tune with LoRA, what exactly do you save - the whole model?
A: No. You save only the LoRA *adapter*: the small low-rank matrices plus a config describing where they attach. The multi-gigabyte base model is unchanged and re-downloaded from its source. `save_pretrained` on a PEFT model writes just those adapter weights (often a few MB) and `adapter_config.json`. To deploy without the adapter runtime overhead you can `merge_and_unload()` to fold them into the base weights, then save a full merged model. This is a key reason LoRA is cheap to store and share.

9.3 Putting it together

The two files you grounded on show the division of labor cleanly. `utils/device.py` is pure environment plumbing - resolving *where* (device) and *in what precision* (dtype) computation happens, so the same code runs on a CUDA box, an Apple laptop, or CI. `models/base.py` is where Python's abstractions (dataclasses, ABCs, type hints, decorators like `@torch.no_grad()` and `@register_model`) wrap PyTorch's primitives (tensors, `.to()`, `eval()`, `requires_grad`, `save_pretrained`) into a clean, swappable model interface. With Python and PyTorch understood, you're ready for the libraries built on top of them: HuggingFace Transformers, PEFT, Datasets, and Accelerate (see Ch. 10 and Ch. 11).

10. Core Tools II - The Hugging Face Stack

Chapter 9 covered the low-level engine (PyTorch tensors, autograd, devices). This chapter covers the layer VisionDoc AI actually spends most of its lines in: the **Hugging Face (HF) ecosystem** - a family of Python libraries that turn "I want to fine-tune a state-of-the-art model" from a months-long research project into a few hundred lines of glue code.

"Hugging Face" is a company that hosts the **Hub** (a GitHub-like site for pretrained models and datasets) and maintains the open-source libraries below. Think of it as an app store plus SDK for machine learning. The five libraries you must know cold:

Library	One-line job	Where VisionDoc uses it
<code>transformers</code>	Load and run pretrained models + their text/image preprocessors	<code>models/qwen_vl.py</code>
<code>datasets</code>	Download, cache, and transform training data efficiently	<code>preprocessing/datasets.py</code> , <code>preprocessing/cache.py</code>
<code>accelerate</code>	Hide the details of GPUs / multi-GPU / mixed precision	Under the hood of <code>Trainer</code>
<code>peft</code>	Inject small trainable LoRA adapters into a frozen model	<code>models/base.py</code> , <code>models/lora.py</code>
<code>bitsandbytes</code>	Load model weights in 4-bit to fit on small GPUs (QLoRA)	<code>models/qwen_vl.py</code>

We take them one at a time, always grounding in the real code.

Key insight: These libraries are deliberately layered. `transformers` sits on `accelerate` sits on PyTorch; `peft` and `bitsandbytes` plug into `transformers`. You rarely touch the lower layers directly - but interviewers love asking *what each layer is responsible for*, so keep the mental map above.

10.1 Transformers - loading and running the model

A **pretrained model** is a neural network whose weights (the millions of numbers from Ch. 9) were already trained by someone else on huge data. Downloading and reconstructing one by hand would be miserable. `transformers` does it in one call.

`from_pretrained` and the Auto classes

`from_pretrained("repo/name")` downloads the weights + config from the Hub (caching them locally so the second run is instant) and returns a ready-to-use object. The **Auto classes** are convenience wrappers that read the model's config file and pick the right concrete class for you.

In `models/qwen_vl.py` :

```
from transformers import AutoProcessor, Qwen2_5_VLForConditionalGeneration

self.processor = AutoProcessor.from_pretrained(processor_id, ...)
self.model = Qwen2_5_VLForConditionalGeneration.from_pretrained(mcfg.model_id, **load_kwargs)
```

Two objects come back, and the distinction is the single most important thing in this section:

- **The model** (`Qwen2_5_VLForConditionalGeneration`) - the actual neural network that maps input tensors to output tensors.
- **The processor** (`AutoProcessor`) - the translator that turns human inputs (a PIL image + a text question) into the exact tensors the model expects, and back.

Interview Q: Why not just `AutoModel.from_pretrained` ? Why the specific `Qwen2_5_VLForConditionalGeneration` class?

A: `AutoModel` returns the *base* transformer that emits raw hidden states - no language-modeling head, so it cannot generate text. The `...ForConditionalGeneration` class adds the head that projects hidden states to vocabulary logits and provides `.generate()` . For a VLM doing question-answering you need that head. The repo names the class explicitly because the exact architecture is known and fixed.

`from_pretrained` accepts the knobs you see in `load_kwargs` :

```
load_kwargs = dict(
    torch_dtype=self.dtype,          # e.g. bfloat16 - half-precision weights (Ch. 9)
    attn_implementation=mcfg.attn_implementation, # e.g. "flash_attention_2"
    trust_remote_code=mcfg.trust_remote_code,    # allow model-repo custom code
)
```

- `torch_dtype` controls the precision the weights load in - the lever for VRAM.
- `trust_remote_code=True` lets the Hub repo ship custom Python that runs on your machine. It is a genuine security decision: only enable it for repos you trust. The config makes it explicit rather than silently on.

Why the processor matters *especially* for VLMs

For a text-only model the preprocessor is just a **tokenizer** (splits text into integer token IDs). A **VLM** (Vision-Language Model) also has to turn the image into tokens and splice them into the text stream at the right position. That is a **processor** - tokenizer + image processor bundled together.

VisionDoc passes two image-sizing bounds when building it:

```
self.processor = AutoProcessor.from_pretrained(
    processor_id, min_pixels=mcfg.min_pixels, max_pixels=mcfg.max_pixels, ...
)
```

Qwen2.5-VL uses **dynamic resolution**: instead of squashing every scan to a fixed size, it tiles the image into patches and produces *more image tokens for bigger images*. `max_pixels` caps that - a huge invoice scan could otherwise expand into thousands of image tokens and blow up memory. The config default `1280 * 28 * 28` reflects Qwen's 28x28-pixel patch grid.

Gotcha: The processor is not optional decoration. If you tokenize text yourself and feed the model, the image placeholder tokens will not match the number of visual features the vision encoder produced, and generation crashes with a shape-mismatch. Always route inputs through the *same* processor the model was published with - and save it alongside your checkpoint (the trainer does exactly this, §10.3).

Chat templates

Instruction-tuned models were trained on conversations wrapped in special formatting tokens (system / user / assistant turns, plus role markers). You must reproduce that exact formatting or the model behaves erratically. A **chat template** is a small template (shipped inside the processor) that renders a list of message dicts into the correct string.

VisionDoc builds messages, then applies the template:

```
messages = [
    {"role": "system", "content": _SYSTEM_PROMPT},
    {"role": "user", "content": [
        {"type": "image", "image": image},
        {"type": "text", "text": question},
    ]},
]
text = self.processor.apply_chat_template(
    messages, tokenize=False, add_generation_prompt=True
)
```

Two flags carry real meaning:

- `tokenize=False` returns the formatted *string* (the code tokenizes later, together with the images, in one processor call).
- `add_generation_prompt=True` appends the tokens that say "assistant, your turn" - used at **inference** so the model starts answering. At **training** the full example already contains the answer, so `collate_train` uses `add_generation_prompt=False`.

That asymmetry is the hinge of the whole training setup: the code renders the prompt *with* the generation cue and the full example *without* it, so the difference in token count tells it exactly how many tokens are "prompt" versus "answer" - which drives the loss masking (see Ch. 6 for LoRA and Ch. 11/12 for the training loop details).

generate

`.generate()` is the method that produces text autoregressively - predict one token, append it, feed it back, repeat, until an end-of-sequence token or a length cap. Qwen2.5-VL is a **causal LM** (it only ever attends to tokens on its left), so `generate` returns `prompt + completion` concatenated. The repo strips the prompt off in `decode`:

```
def decode(self, generated_ids, prompt_width):
    trimmed = generated_ids[:, prompt_width:] # drop the prompt columns
    return self.processor.batch_decode(trimmed, skip_special_tokens=True, ...)
```

Interview Q: Training uses *right* padding but inference uses *left* padding - why?

A: Padding fills short sequences to a common length so they stack into one tensor. During generation, new tokens are appended on the right; if sequences were right-padded the pad tokens would sit *between* the prompt and the new tokens, misaligning the batch. Left padding pushes all real prompt tokens flush to the right edge so appended tokens line up. During training there is no appending - instead we mask the loss per-sample by prompt length, and right padding keeps every prompt's prefix at the same starting offset, which makes that masking correct. The code sets `self.processor.tokenizer.padding_side` accordingly in each path.

10.2 Datasets - feeding the model

`datasets` is the data counterpart to `transformers`. Its headline features: streaming download from the Hub, **memory-mapped** storage (data lives on disk in the Apache **Arrow** columnar format and is paged into RAM on demand, so a dataset far larger than your RAM still works), and cheap transforms.

load_dataset and splits

`load_dataset(id, subset, split=...)` fetches a corpus. A **split** is a named partition - conventionally `train` (learn from), `validation` (tune on, watch for overfitting), and `test` (final unbiased score). In `preprocessing/datasets.py` :

```
from datasets import load_dataset
return load_dataset(config.data.dataset_id, config.data.dataset_subset,
                    split=hf_split, cache_dir=config.data.cache_dir)
```

Real corpora are messy about splits, and the repo handles it thoughtfully. DocVQA's `test` split is an *unlabelled* challenge set; FUNSD and SROIE ship only `train + test` . So VisionDoc records, per dataset, which splits are "native":

```
"funsd": _DatasetSpec(_load_funsd, {"train": True, "validation": False, "test": True}),
```

and when a split is missing it **carves** one out of the train pool using a *seeded shuffle* (`random.Random(config.seed).shuffle(indices)`) so the exact same rows land in each split on every machine. Reproducibility of evaluation numbers depends on this determinism.

Gotcha: Carving eval data out of train risks **leakage** - the same example appearing in both train and eval, which inflates scores. `_partition_pool` reserves the validation and test slices *first* from a shuffled index list and gives train only what's left, guaranteeing the three sets are disjoint.

The Image feature and map

Every HF dataset column has a typed **feature**. `datasets.Image` is a feature that stores an image efficiently and re-hydrates it as a PIL object when you read the row. VisionDoc casts its image column to it:

```
from datasets import Dataset, Image as HFImage
ds = Dataset.from_list(records)
ds = ds.cast_column("image", HFImage()) # store/re-load images efficiently
```

The other workhorse is `.map(fn)` - apply a function to every row (optionally batched and multiprocessed) to produce a new dataset, with results cached to disk keyed by a hash of the function and inputs, so an unchanged pipeline never recomputes. VisionDoc does its heavy normalization in plain Python loaders that emit `DocSample` objects rather than through `map` , but `map` is the canonical HF pattern and interviewers expect you to know it.

save_to_disk / caching

`preprocessing/cache.py` persists the fully-built splits so training does not re-download and re-normalize every run:

```
splits_hf.save_to_disk(str(path)) # writes Arrow files + metadata
# later:
from datasets import load_from_disk
return load_from_disk(str(path))
```

`save_to_disk` writes the Arrow dataset to a directory; `load_from_disk` memory-maps it straight back. Because the images were cast to the `Image` feature first, they are stored compactly and reappear as PIL objects on load.

Interview Q: What is the difference between the Hub cache and `save_to_disk` ?

A: The Hub cache (under `cache_dir`) holds the *raw downloaded* corpus so you don't re-download from the internet. `save_to_disk` snapshots *your processed* dataset - after split-carving, casting, and any transforms - so you skip both re-download and re-processing. They are separate layers of caching.

10.3 Accelerate - the device/distribution abstraction

`accelerate` is the library that answers "which device does this tensor go on, and how do I run across several GPUs or in mixed precision?" - without you writing device-specific code. It abstracts single-GPU, multi-GPU, multi-machine, and **mixed precision** (doing math in fast 16-bit while keeping a 32-bit master copy for stability - see Ch. 9) behind one interface.

VisionDoc never instantiates an `Accelerator` directly - and that is the point. The Hugging Face `Trainer` (built in `training/trainer.py`) *uses accelerate internally*. When the config sets `bf16=True` or `gradient_accumulation_steps`, those flow into `TrainingArguments`, and the Trainer's accelerate backend arranges the mixed-precision autocast, the gradient scaling, device placement, and (if you launched with `accelerate launch`) the multi-GPU synchronization. The relevant config passes through untouched:

```
kwargs = dict(
    fp16=fp16, bf16=bf16,
    gradient_accumulation_steps=tcfg.gradient_accumulation_steps,
    gradient_checkpointing=tcfg.gradient_checkpointing, ...
)
```

The one place VisionDoc *does* place devices by hand is when quantization is **not** used: `self.model.to(self.device)` in `qwen_vl.py`. When bitsandbytes quantization *is* used it passes `device_map={"": 0}` and lets the loader place the (immovable) 4-bit weights - you must not call `.to()` on a quantized model, which is why the code guards it with `if quant is None:`.

Key insight: "Accelerate is used" does not mean "you see the word Accelerate." In this repo it is entirely mediated by `Trainer`. Being able to say *the Trainer delegates device/precision/distribution to accelerate* is exactly the layered-architecture understanding interviewers probe for.

10.4 PEFT - the LoRA library

Chapter 6 explained the **LoRA concept**: freeze the giant pretrained model and train tiny low-rank matrices injected next to selected weight matrices, so you update ~0.1% of parameters instead of 100%. **PEFT** (Parameter-Efficient Fine-Tuning) is the *library* that implements that concept - keep the two straight: LoRA = the math, PEFT = the code.

The core three-step dance lives in `models/base.py`:

```
from peft import LoraConfig, TaskType, get_peft_model

peft_config = LoraConfig(
    r=lcfg.r,                                # rank of the adapter matrices (capacity vs. size)
    lora_alpha=lcfg.lora_alpha,              # scaling applied to the adapter output
    lora_dropout=lcfg.lora_dropout,
    bias=lcfg.bias,
    task_type=getattr(TaskType, lcfg.task_type, TaskType.CAUSAL_LM),
    target_modules=targets,                  # which layers get adapters
    modules_to_save=lcfg.modules_to_save,
```

```
)
self.model = get_peft_model(self.model, peft_config) # wraps + freezes base
```

- 1 `LoraConfig` - a dataclass describing the adapters. `r` (rank) sets adapter capacity and size; `lora_alpha` scales their contribution; `target_modules` names the layers to adapt. In `qwen_vl.py` those are the language decoder's attention and MLP projections (`q_proj` , `k_proj` , `v_proj` , `o_proj` , `gate_proj` , `up_proj` , `down_proj`) - the ViT vision encoder is left frozen. `task_type=CAUSAL_LM` tells PEFT the model is a left-to-right language model.
- 2 `get_peft_model` - wraps the base model: freezes every original weight, injects the trainable low-rank matrices at the targeted modules, and returns a `PeftModel` . After this, `trainable_parameters()` reports the dramatic drop (the log prints e.g. `0.1%` trainable).
- 3 **Save / load / merge** - the adapter is tiny, so you ship *just* the adapter, not a full model copy:

```
self.model.save_pretrained(str(path)) # writes adapter_config.json + weights
self.model = PeftModel.from_pretrained(self.model, str(path)) # re-attach onto base
self.model = self.model.merge_and_unload() # fold adapter into base weights
```

`save_pretrained` writes a directory containing `adapter_config.json` plus the small adapter weights (a few MB). `PeftModel.from_pretrained(base, path)` re-attaches a saved adapter onto a freshly loaded base for inference. `merge_and_unload()` mathematically adds the low-rank update into the base weight matrices and returns a plain model - no PEFT layers, so inference has *zero* adapter overhead. The repo uses `merge` when you want the fastest possible serving path; it keeps the adapter separate when you want to swap adapters or keep the base pristine.

Interview Q: You trained a 7B model with LoRA. What do you actually ship, and how big is it?

A: Just the adapter directory from `save_pretrained` - `adapter_config.json` plus the low-rank matrices, typically a few megabytes, versus ~14 GB for the full model. At serve time you load the public base model once and attach the small adapter with

`PeftModel.from_pretrained` . If you never need to swap adapters, `merge_and_unload` folds it in so there's no runtime cost.

Gotcha: `load_adapter` in this repo validates the path before handing it to PEFT, because PEFT treats a *non-existent local path* as a Hub repo id and throws a confusing

`HFValidationError` . The usual real cause is running evaluation before training finished writing the adapter - a great example of translating a library's opaque error into an actionable message.

Two subtle interactions with the rest of the stack:

- **Gradient checkpointing + PEFT:** `apply_lora` calls `self.model.enable_input_require_grads()` when checkpointing is on. Gradient checkpointing recomputes activations to save memory, and it needs the input embeddings to require gradients even though the base is frozen - the standard PEFT-for-VLMs recipe. The trainer pairs this with `use_reentrant=False` , the modern autograd path that cooperates with frozen base weights.
- `modules_to_save` : some layers (e.g. a resized embedding) must be fully trained, not just adapted; PEFT trains those in full while LoRA-adapting the rest.

10.5 bitsandbytes - 4-bit loading (QLoRA)

`bitsandbytes` (often "bnb") provides low-precision (8-bit and 4-bit) implementations so a model that needs, say, 14 GB in 16-bit fits in ~4 GB. Combining 4-bit base weights with LoRA adapters is **QLoRA** (Quantized LoRA) - the technique that lets you fine-tune a 7B VLM on a single consumer GPU.

VisionDoc builds the config in `_maybe_quantization_config` :

```
from transformers import BitsAndBytesConfig
BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",          # 4-bit "NormalFloat" - tuned for weight distributions
    bnb_4bit_use_double_quant=True,     # quantize the quantization constants too, extra savings
    bnb_4bit_compute_dtype=self.dtype, # math runs in bf16/fp16, storage in 4-bit
)
```

`BitsAndBytesConfig` is a `transformers` object (`bitsandbytes` is the backend it calls) that you hand to `from_pretrained` via `quantization_config` . The knobs:

- `load_in_4bit` - store weights in 4 bits instead of 16/32.
- `bnb_4bit_quant_type="nf4"` - NF4 ("NormalFloat 4") is a 4-bit format designed for the bell-curve distribution of neural-net weights, giving better accuracy than plain int4.
- `bnb_4bit_use_double_quant=True` - quantizes the per-block scaling constants themselves, shaving a bit more memory at negligible quality cost.
- `bnb_4bit_compute_dtype` - weights are *stored* in 4-bit but *de-quantized* to this dtype for each matrix multiply, so the arithmetic still happens in bf16/fp16.

Crucially, the repo enables this **only on CUDA**:

```
if self.device.type != "cuda":
    logger.warning("Quantization requested but device is %s; ignoring.", ...)
    return None
```

`bitsandbytes` kernels are CUDA-only, so on a Mac (MPS) or CPU the code degrades gracefully to full-precision loading rather than crashing - important because the developer's laptop in this project is Apple Silicon.

Interview Q: In one sentence, how does QLoRA let you fine-tune a 7B model on a 16 GB GPU?

A: The frozen base weights are stored in 4-bit (NF4) via `bitsandbytes` so they occupy a quarter of the memory, and only the tiny LoRA adapters (in higher precision) receive gradients and optimizer state - so both the weight storage and the training-state memory stay small enough to fit.

Gotcha: You cannot call `.to(device)` on a `bitsandbytes`-quantized model - the 4-bit weights are already placed by `device_map` at load time. That is exactly why `qwen_vl.py` writes `if quant is None: self.model.to(self.device)` ; moving a quantized model manually raises an error.

10.6 How the pieces click together

Trace a single training run through the stack and the layering becomes concrete:

- 1 `datasets.load_dataset` pulls the corpus; `preprocessing/datasets.py` normalizes it to `DocSamples` and casts images to the `Image` feature; `cache.py` `save_to_disk` s the result.

- 2 `transformers.from_pretrained` loads Qwen2.5-VL - optionally 4-bit via `BitsAndBytesConfig` (bitsandbytes) - plus its `AutoProcessor` .
- 3 `peft.get_peft_model` freezes the base and injects LoRA adapters.
- 4 `training/trainer.py` builds a `Trainer` , which drives the loop and delegates device placement + mixed precision + gradient accumulation to **accelerate**.
- 5 The model's `collate_train` uses the **processor** and **chat template** to tensorize each batch with correct padding and answer-only loss masking.
- 6 After training, `save_adapter` writes the few-MB adapter (and the processor) for self-contained, reloadable inference.

Every arrow in that chain is one of the five libraries doing its one job. Hold that map and you can answer almost any "how does the HF stack work" interview question by pointing at the right layer.

Interview Q: Someone says "just use the Trainer, it does everything." What is the Trainer *not* responsible for, that this project still had to write?

A: The Trainer orchestrates the loop, checkpointing, evaluation cadence, and (via accelerate) devices/precision. It does **not** know how to turn a document image + question into masked, padded VLM tensors - that model-specific tensorization is `collate_train` , passed in as the `data_collator` . It also can't compute generation metrics like ANLS/EM inside its teacher-forced eval loop, so this project deliberately supplies `compute_metrics=None` , selects checkpoints on `eval_loss` , and computes real generation metrics post-hoc (see Ch. 12 on evaluation).

11. Core Tools III - Serving, Ops, and Data Libraries

A trained model is worthless until someone can *ask it a question and get an answer back*. This chapter covers the tools that turn VisionDoc AI from a folder of Python into a running product: the web server that answers HTTP requests, the dashboard a human clicks on, the container that packages it all, the tracker that records experiments, the numeric libraries that move pixels around, and the version control and cloud-GPU tooling underneath. Every tool below is defined from zero - assume you have only ever written a small script before. (Model internals live in Ch. 6-8; evaluation *math* lives in Ch. 9. Here we cover the *libraries* that serve and measure.)

11.1 FastAPI - the HTTP inference server

What it is. FastAPI is a Python library for building **web APIs** - programs that listen on a network port and answer requests from other programs. An **API** (Application Programming Interface) is just a contract: "send me this shape of data, I'll send back that shape." **HTTP** is the language browsers and servers speak; a **REST API** organizes that language around *resources* and *verbs* (`GET` to read, `POST` to send). Think of FastAPI as a receptionist: it takes a well-formed request at the front desk, hands it to the right worker, and returns the worker's answer in a tidy envelope.

Why it's used here. VisionDoc AI needs to expose its model over the network so the Streamlit dashboard (or any client, in any language) can call it. FastAPI is chosen because it is fast, async-capable, and - crucially - it *validates input automatically* and *generates its own documentation*.

Path operations. In FastAPI you attach a Python function to a URL + verb with a decorator. In `api/main.py` the endpoints are:

Verb + path	Function	Purpose
<code>GET /health</code>	<code>health()</code>	Liveness + config snapshot; never loads the model
<code>POST /ask</code>	<code>ask()</code>	JSON: base64 image + question -> answer, confidence, regions
<code>POST /extract</code>	<code>extract()</code>	JSON: pull structured fields from a document
<code>POST /predict</code>	<code>predict()</code>	Multipart: upload a file + question -> answer

```
@app.post("/ask", response_model=AskResponse, tags=["qa"])
def ask(request: AskRequest) -> AskResponse:
    image = _decode_base64_image(request.image_base64)
    ...
    return AskResponse(answer=result.answer, confidence=result.confidence, ...)
```

Pydantic models. `AskRequest` and `AskResponse` are **Pydantic** models - Python classes that declare the *type and shape* of data. **Pydantic** is a validation library: if a client sends a request missing `question`, or with `question` shorter than one character, FastAPI rejects it with a `422` error *before your code ever runs*. The project keeps these in `api/schemas.py` so tests and clients can import the shapes without pulling in the heavy ML stack. Note the deliberate asymmetry documented there: *input* models validate strictly, but *output* models (like `confidence: float`, unconstrained) never reject a value the model legitimately produced - a response-validation failure would turn a good

answer into an opaque 500.

Async and the threadpool. An **async** server can juggle many requests on one thread by switching between them whenever one is *waiting* (e.g. on the network). But VisionDoc's `generate()` call is *CPU/GPU-bound* - it doesn't wait, it computes - so it would *block* an async loop. The project's fix is subtle and worth understanding: the handlers are plain `def`, not `async def`. FastAPI runs a plain-`def` handler in a **worker thread** from a pool, so the slow model call never stalls the main event loop. `predict()` even reads the upload synchronously (`file.file.read()`) to stay in that threadpool model.

Lazy singleton - the load-time trick. The model is multiple gigabytes and slow to load, so `main.py` never loads it at import time. `get_predictor()` builds it *once*, on the first request that needs it, using **double-checked locking**:

```
def get_predictor():
    global _predictor
    if _predictor is None:
        # fast path: no lock once built
        with _predictor_lock:
            # only first callers contend
            if _predictor is None:
                # re-check inside the lock
                from inference.predictor import DocumentPredictor # deferred heavy import
                _predictor = DocumentPredictor.from_config(get_config())
    return _predictor
```

The payoff: the process boots instantly, `/health` answers without a GPU or any weights (perfect for a container health probe), and concurrent first requests still load the weights exactly once.

uvicorn. FastAPI describes *what* the endpoints do but doesn't itself talk TCP. **uvicorn** is the **ASGI server** (Asynchronous Server Gateway Interface - the async web-server standard for Python) that actually accepts connections and runs the app. The entrypoint passes an *import string* so uvicorn owns the lifecycle:

```
uvicorn.run("api.main:app", host="0.0.0.0", port=port, reload=False)
```

OpenAPI docs. FastAPI reads your Pydantic models and decorators and auto-generates an **OpenAPI** schema (a machine-readable description of the API). It serves two human UIs for free at `/docs` (Swagger) and `/redoc`, where you can click "Try it out" and fire real requests. The `examples` blocks in `api/schemas.py` show up there verbatim.

Interview Q: Why would you make a slow model endpoint a plain `def` instead of `async def` in FastAPI?

A: `async def` handlers run on the single event loop; a blocking, compute-bound call (like model generation) would freeze *all* concurrent requests. A plain `def` handler is dispatched to a threadpool, so the blocking work happens off the loop and the server stays responsive. Use `async def` only when the body genuinely awaits I/O.

Gotcha: The project sets CORS `allow_origins=["*"]` for the demo. CORS (Cross-Origin Resource Sharing) controls which websites' JavaScript may call your API. Wide-open is fine for a local demo but a data-exfiltration risk in production - the code comments explicitly say to replace it with an allowlist, and it keeps `allow_credentials=False` because `"*" + credentials` is forbidden by the spec.

11.2 Streamlit - the dashboard

What it is. Streamlit turns a plain Python script into an interactive web app with no HTML or JavaScript. You write top-to-bottom Python; Streamlit renders each `st.something(...)` call as a UI element.

The reactive script model. This is the one idea that confuses newcomers. Streamlit **re-runs your entire script from top to bottom every time the user interacts** with any widget. There is no callback spaghetti - a button click just triggers a full re-execution. That raises two problems the project solves cleanly:

- 1 **Don't reload the model on every click.** `@st.cache_resource` memoizes an expensive object across re-runs (and across users). `app/streamlit_app.py` caches the predictor, keyed on `(model_choice, adapter_path)`, so it loads once and only rebuilds if you switch models:

```
@st.cache_resource(show_spinner="Loading model - first run downloads/initializes weights...")
def get_predictor(model_choice, adapter_path):
    from inference.predictor import DocumentPredictor
    ...
```

- 1 **Remember results across re-runs.** `st.session_state` is a per-session dictionary that survives re-execution. `run_inference()` stashes the answer, image, and question there so the result stays on screen when the user tweaks an unrelated widget.

Widgets used in the app. `st.sidebar.file_uploader` (drop an image/PDF), `st.sidebar.radio` (Qwen vs Donut), `st.sidebar.selectbox` / `text_input` (question), `st.sidebar.checkbox` (highlight), `st.sidebar.button("Run")`, `st.columns` for layout, `st.image`, `st.json`, `st.progress` (a VRAM meter), `st.spinner`, and `st.tabs` (a "Document QA" tab and a "Research: Base vs Fine-tuned" tab). Errors are surfaced with `st.error` so an operator sees *why* something failed, not a stack trace.

Key insight: `st.cache_resource` is for *unhashable, long-lived objects* (models, DB connections); `st.cache_data` is for *serializable data* (dataframes, API results) and returns a copy. Mixing them up - caching a model with `cache_data` - would try to pickle multi-GB weights on every call.

Interview Q: Streamlit re-runs the whole script on every interaction - how do you keep a heavy model from reloading each time?

A: Wrap its construction in `@st.cache_resource`, keyed on the inputs that define the object (here the model choice and adapter path). Streamlit returns the same instance across re-runs and sessions until the key changes.

11.3 Docker & docker-compose - reproducible packaging

What a container is. A **Docker image** is a frozen, self-contained snapshot of an operating system plus your app and every dependency. A **container** is a running instance of an image. The point is **reproducibility**: "works on my machine" disappears because the machine *is* the image - same Python, same system libraries, same code, everywhere.

Layers. An image is built from a `Dockerfile` as a stack of **layers**, one per instruction. Docker caches layers and reuses them when nothing above changed. The VisionDoc `docker/Dockerfile` exploits this deliberately: it copies `requirements.txt` and installs dependencies *before* copying the source code:

```
COPY requirements.txt ./
RUN pip install -r requirements.txt # cached; heavy torch install reused
COPY . . # source changes invalidate only from here
RUN pip install -e .
```

Editing a Python file therefore does *not* trigger a multi-minute torch/transformers reinstall - only the cheap final layers rebuild.

System dependencies. The image is `python:3.11-slim` (Debian, glibc - *not* Alpine, because the prebuilt torch/opencv wheels need glibc). It `apt-get install`s the non-Python libraries the wheels load at runtime: `libgl1 / libglib2.0-0` for OpenCV, `poppler-utils` for PDF->image, `tesseract-ocr` for the OCR fallback, and `curl` for the healthcheck - all cleaned in the same layer (`rm -rf /var/lib/apt/lists/*`) so the junk isn't baked in.

One image, two services. A single image serves *both* the API and the dashboard; `docker-compose.yml` picks which by overriding the `command` . **docker-compose** is a tool for defining a multi-container app in one YAML file:

Service	Command	Port
api	uvicorn api.main:app	8000
app	streamlit run app/streamlit_app.py	8501

Both mount the same **named volume** `hf-cache` at `/app/.cache/huggingface` so the multi-GB weights are downloaded once and survive restarts. They share a user-defined **bridge network** so the dashboard reaches the API by name at `http://api:8000` (set via `VISIONDOC_API_URL`). `depends_on: condition: service_healthy` makes `app` wait until the API's `/health` healthcheck passes - with a generous `start_period: 180s` because the first request lazily loads the VLM.

Interview Q: Why copy `requirements.txt` and install deps *before* copying your source code in a Dockerfile?

A: Docker layer caching. Dependencies change rarely; source changes constantly. Installing deps in an earlier layer means routine code edits reuse the cached (slow) install layer and rebuild only the fast copy-code layer.

Gotcha: A `HEALTHCHECK /compose healthcheck` that probes an endpoint which *forces model load* would fail its timeout. VisionDoc's `/health` deliberately reports only configured metadata and never loads weights, so the probe is instant.

11.4 Weights & Biases - experiment tracking

What it is. When you train models you run dozens of experiments with different settings; you need to remember which run got which accuracy, and see loss curves. **Weights & Biases (W&B, wandb)** is a service that logs metrics, hyperparameters, and artifacts and plots them in a dashboard. Think of it as a lab notebook that draws its own graphs.

How the project uses it. Training reports to W&B when configured (`report_to=wandb`). The important detail in `training/train.py` is **offline mode**: on a CI box or sandbox with no W&B login, `report_to=wandb` would otherwise *block* on `wandb login` . So the project defaults `WANDB_MODE=offline` (logs to local disk, syncable later) using `setdefault` , which respects

an operator who *has* configured online W&B:

```
if tracker == "wandb":
    os.environ.setdefault("WANDB_MODE", "offline")
    os.environ.setdefault("WANDB_SILENT", "true")
    os.environ.setdefault("WANDB_PROJECT", config.name)
```

TensorBoard is the alternative tracker (`report_to=tensorboard`), writing local event files. (Training itself is Ch. 8.)

Key insight: Defaulting `WANDB_MODE=offline` is a *sandbox-safety* pattern: experiment tracking must never make training hang on an interactive login prompt in an environment with no credentials.

11.5 The numeric & image libraries - NumPy, Pandas, Pillow, OpenCV

These four do the low-level data-and-pixel work under everything else.

- **NumPy** - the foundation of numeric Python. Its core object is the **ndarray**, a fast, fixed-type N-dimensional grid of numbers. An image *is* a NumPy array: shape `(height, width, 3)` for RGB, each value 0-255. Nearly every ML library speaks NumPy.
- **Pandas** - tables. A **DataFrame** is a spreadsheet-in-code: named columns, typed rows. The Streamlit research tab uses `import pandas as pd` to read a metrics CSV and display base-vs-fine-tuned comparisons.
- **Pillow (PIL)** - image I/O and simple edits. The project's `utils.image_utils` converts between base64 strings, raw bytes, and PIL `Image` objects; `base64_to_pil` and `load_image` are what the API handlers call to turn a request into something the model can see.
- **OpenCV (cv2)** - heavier computer-vision toolkit (drawing boxes, transforms). It's why the Docker image installs `libgl1`. The project uses the `opencv-python-headless` build - "headless" means no GUI dependencies, correct for a server.

Gotcha: PIL and OpenCV disagree on channel order - PIL uses **RGB**, OpenCV uses **BGR**. Passing a `cv2` -loaded array straight into a PIL/RGB pipeline silently swaps red and blue. Convert explicitly when crossing between them.

11.6 Git & GitHub - version control

Git is a **version control system**: it records snapshots (commits) of your code so you can review history, undo mistakes, and work in parallel. A **branch** is an independent line of development; this project works on `experiment/prod-ready` instead of `main` so unfinished work never destabilizes the mainline. A **remote** (like **GitHub**, the hosting service) is a shared copy you `push` to and `pull` from, enabling collaboration and backup.

```
git checkout -b experiment/prod-ready # new branch
git add -p && git commit -m "harden: ..." # snapshot a change
git push -u origin experiment/prod-ready # publish to GitHub
```

A **pull request (PR)** on GitHub proposes merging one branch into another and is where code review happens. (You don't need deep Git for interviews, but you must be fluent in branch/commit/merge/PR vocabulary.)

11.7 Google Colab & GPUs - where training runs

Why a GPU at all. A **GPU** (Graphics Processing Unit) does thousands of arithmetic operations in parallel - exactly the matrix multiplies neural networks are made of - so it trains models tens to hundreds of times faster than a **CPU**. **CUDA** is NVIDIA's software layer that lets PyTorch drive the GPU; it's why the Dockerfile's GPU path uses an `nvidia/cuda` base image and a `cu124` torch build.

VRAM is the constraint. A GPU has its own memory, **VRAM** (Video RAM). The whole model, its activations, and (during training) its gradients must fit in VRAM. This is *the* number that decides what you can run:

GPU	Typical VRAM	Where
T4	16 GB	Free Google Colab
A100	40 / 80 GB	Colab Pro+, cloud

Google Colab is a free, browser-hosted Jupyter notebook environment that hands you a GPU (usually a T4) with no setup - ideal for training LoRA adapters when you don't own a GPU. VRAM pressure is the whole reason for **QLoRA** / `bitsandbytes` 4-bit quantization (Ch. 7): squeezing the model into a T4's 16 GB. Note `bitsandbytes` is Linux/CUDA-only - the Dockerfile comments call this out - which is why the default CPU image omits it.

Interview Q: You have a free Colab T4 (16 GB VRAM) and a 7B-parameter model that needs ~28 GB in fp16. How do you train it?

A: Quantize the base model to 4-bit with `bitsandbytes` (~4 GB) and fine-tune with LoRA/QLoRA, so only tiny adapter weights and their optimizer state consume gradient memory. Add gradient checkpointing and a small batch size with gradient accumulation to stay under 16 GB.

11.8 Evaluation libraries

Measuring a document-QA model needs text-comparison metrics. The project keeps the *core* metrics `stdlib`-only and imports the heavy libraries **lazily inside each function**, with graceful fallbacks, so a stripped-down install still runs (`evaluation/metrics.py`). Metric *meaning* is Ch. 9; here is what each library provides:

Library	Provides	Fallback in project
Levenshtein	C-accelerated edit distance -> ANLS (the DocVQA metric)	pure-Python DP
sacrebleu	standard corpus BLEU (translation-quality overlap)	<code>nltk</code> BLEU, then 0
rouge-score	ROUGE-1/2/L (summary overlap)	zeros
scikit-learn	general ML utilities / metrics	-
evaluate	HuggingFace's unified metric-loading hub	-

Levenshtein distance counts single-character edits (insert/delete/substitute) to turn one string into another; normalized into `[0, 1]` and averaged, it becomes **ANLS** (Average Normalized Levenshtein Similarity), which rewards *near-miss* OCR answers instead of demanding exact string equality. **BLEU**

and **ROUGE** measure n-gram overlap with reference text. `exact_match` and token-level **F1** are pure Python in the same file.

Key insight: The lazy-import-with-fallback pattern (`try: import sacrebleu ... except: use nltk/zero`) means a missing optional dependency degrades one metric to a safe default instead of crashing the whole evaluation - the same defensive philosophy as the API's deferred heavy imports.

Interview Q: Why is ANLS the standard metric for document VQA instead of plain exact-match?

A: Document answers come from imperfect OCR and free-text generation, so an answer can be *correct in meaning* yet differ by a character or two. Exact-match scores those as total failures; ANLS uses normalized Levenshtein similarity with a threshold, giving near-correct answers proportional credit and only zeroing out genuinely wrong ones.

Summary

FastAPI + uvicorn expose the model over HTTP with automatic validation and self-generating docs, using a lazy singleton so the container boots instantly. Streamlit gives operators a click-through dashboard, tamed with `cache_resource` and `session_state` against its re-run-everything model. Docker + compose package one image into two reproducible services sharing a weights cache. W&B (offline by default) records experiments; NumPy/Pandas/Pillow/OpenCV move the pixels and tables; Git/GitHub track the code; Colab/GPUs/CUDA/VRAM are where training happens; and a stack of edit-distance and n-gram libraries - behind graceful fallbacks - turn raw predictions into numbers you can trust.

12. Inside the Codebase I - Config, Utils, Preprocessing, Models

This is the first of two guided tours through the actual VisionDoc AI source tree. Here we read the *foundation* layers - the ones every other part of the system stands on: how the project is configured, the low-level utilities everyone shares, how raw datasets become uniform training examples, and how a single model interface hides the differences between two very different vision-language backbones. (The training loop, evaluation, inference server, and app are covered in later chapters.)

Before we open any file, fix one idea in your head, because it is the single design decision that shapes almost everything below:

Key insight: VisionDoc AI is built around **two contracts** - one *data* contract (`DocSample`) and one *model* contract (`VisionDocModel`). Every dataset in the world is squeezed into `DocSample` ; every model backbone is hidden behind `VisionDocModel` . Because those two shapes are fixed, the trainer, evaluator, API, and app never need to know whether you are using DocVQA or CORD, Qwen or Donut. "Contract" here just means an agreed-upon shape that both sides promise to honor - like a power socket: any appliance with the right plug works, and the wall never needs to know what the appliance is.

Let me define a few terms you'll meet immediately, in plain language:

- **Dataclass** - a Python class whose only job is to hold named fields, where Python auto-writes the boring `__init__` for you. Think of it as a labeled box with named slots.
- **Frozen dataclass** - the same, but read-only after creation. Trying to change a field raises an error. Good for configuration you never want silently mutated mid-run.
- **Tensor** - an n-dimensional array of numbers (a scalar is 0-D, a vector 1-D, a matrix 2-D, an image batch 4-D). It is the universal currency neural networks compute in. PyTorch's tensor is like NumPy's array but can live on a GPU and track gradients.
- **dtype** - the numeric precision of those numbers: `float32` (full), `float16` / `bfloat16` (half - half the memory, faster on GPUs).

12.1 `configs/config.py` - one typed, immutable config to rule them all

Every script (download, preprocess, train, evaluate, serve) is driven by **one** config object loaded from a YAML file. The config is a tree of frozen dataclasses:

```
ProjectConfig
+- ModelConfig      (which backbone, dtype, quantization, pixel budget)
+- LoRAConfig       (rank r, alpha, dropout, which layers to adapt)
+- DataConfig        (dataset name/id, image size, split fractions, caps)
+- TrainingConfig    (epochs, batch size, LR, eval/save cadence)
+- InferenceConfig   (max tokens, sampling, confidence method)
```

Why dataclasses instead of a plain dictionary? The module docstring states the three reasons, and they are excellent interview material:

- 1 **Misspelled keys fail loudly at load time** - not three hours into a GPU run.
- 2 **IDEs and type-checkers can autocomplete and verify field access.**
- 3 **Defaults live in one self-documenting place.**

Strict loading: turning a typo into an instant error

The clever part is `_from_dict`, the recursive builder. When you load YAML, a nested section like `model:` arrives as a plain sub-dictionary; `_from_dict` walks the dataclass fields and rejects anything unknown:

```
unknown = set(data) - set(field_map)
if unknown:
    raise ValueError(f"Unknown config key(s) for {cls.__name__}: {sorted(unknown)}. ...")
```

So writing `learnign_rate: 0.001` in YAML crashes immediately with a helpful message, instead of silently using the default learning rate and wasting a training run.

The `get_type_hints` subtlety (a favorite interview trap)

The file starts with `from __future__ import annotations`. That line makes Python store every type annotation as a **string** rather than as the real class object. It speeds up imports and allows forward references, but it creates a problem: `dataclasses.Field.type` for the `model` field becomes the literal string `"ModelConfig"`, not the `ModelConfig` class. If you naively checked `is_dataclass(field.type)` you'd be checking `is_dataclass("ModelConfig")` - always `False` - and nested sections would leak through as raw dicts.

The fix is `_resolve_field_types`, which calls `get_type_hints(cls)` to *evaluate those strings back into real classes* in the module's namespace:

```
def _resolve_field_types(cls):
    try:
        return get_type_hints(cls)          # "ModelConfig" -> the actual class
    except Exception:
        return {f.name: f.type for f in fields(cls)}
```

Only with the resolved type can `_from_dict` detect `is_dataclass(f.type)` and `isinstance(value, dict)` and recurse to build a real nested `ModelConfig` instead of a dict.

Interview Q: Why does `from __future__ import annotations` break naive nested-dataclass construction, and how do you fix it?

A: That import turns all annotations into strings (PEP 563 postponed evaluation), so `Field.type` is `"ModelConfig"`, not the class - an `is_dataclass()` check on it fails. You resolve the real types with `typing.get_type_hints()`, which evaluates the string annotations in the class's module globals, and then dispatch on those.

Environment overrides: same YAML, three machines

Because the config is frozen, you can't just mutate it. Instead `_with_env_overrides` returns a *copy* (via `dataclasses.replace`) with a few high-churn fields swapped from environment variables - `VISIONDOC_DEVICE`, `VISIONDOC_MODEL_ID`, `VISIONDOC_ADAPTER_PATH`. This is what lets the exact same `default.yaml` run unchanged on a laptop (`VISIONDOC_DEVICE=mps`), a CI runner (`cpu`), and a GPU box (`cuda`). The convenience loader `load_config` picks the file by precedence: explicit argument > `VISIONDOC_CONFIG` env var > `configs/default.yaml`.

A few `DataConfig` / `ModelConfig` fields worth flagging because they recur throughout the project:

Field	Meaning
<code>model_type</code>	Registry key selecting the backbone adapter (<code>qwen2_5_vl</code> / <code>donut</code> / <code>florence2</code>).
<code>min_pixels</code> / <code>max_pixels</code>	Qwen's dynamic-resolution bounds; capping <code>max_pixels</code> is the main VRAM lever for large scans.
<code>load_in_4bit</code>	Turns on QLoRA (4-bit quantization) - needs bitsandbytes, CUDA only.
<code>val_fraction</code> / <code>test_fraction</code>	Used only when a dataset lacks native splits (see 12.4).
<code>confidence_method</code>	<code>seq_prob</code> or <code>min_prob</code> - how a confidence score is derived from generation.

12.2 `utils/` - the shared toolbelt

These are small, dependency-light modules that everything else imports so no behavior gets re-implemented in five places.

`device.py` - never hard-code `"cuda"`

The whole point: training and inference must run *unchanged* on a CUDA GPU box, an Apple-Silicon laptop (**MPS** - Apple's Metal GPU backend), and a CPU-only CI runner. Centralizing device/dtype selection means no other module ever writes `"cuda"` literally.

- `resolve_device("auto")` prefers CUDA -> MPS -> CPU. An explicit-but-unavailable preference *falls back with a warning* instead of crashing - a demo should still run on a laptop even if the config says `cuda`.
- `resolve_dtype` maps a string like `"bf16"` to a `torch.dtype`, but respects hardware: **bfloat16** (a 16-bit float with a wide exponent range, great for training stability) is only kept on CUDA GPUs that support it; on MPS/CPU it downgrades to `float16` or `float32` to avoid silent numerical issues.
- `gpu_memory_stats` / `reset_peak_memory` feed the evaluation profiler and the dashboard's GPU panel (zeros on non-CUDA).
- `log_device_summary` is called once at the top of every script and prints a one-line environment banner.

Gotcha: `bfloat16` matmuls are poorly supported off-CUDA. The code deliberately returns `float32` on MPS/CPU rather than trusting the config, because a "successful" run producing garbage numbers is worse than a clear downgrade log line.

`seed.py` - reproducibility in one call

`set_seed(seed=42, deterministic=False)` seeds Python's `random`, NumPy, and Torch (plus CUDA), and sets `PYTHONHASHSEED`. Fixing every random-number generator (RNG) is what makes an evaluation number *trustworthy* and a bug *reproducible*. The `deterministic` flag additionally forces `cuDNN` into deterministic mode and calls `torch.use_deterministic_algorithms(True)` - bit-for-bit repeatable but slower, so it's opt-in (enable it for eval/debug, leave off for fast training where `cuda.nn.benchmark=True` picks fast kernels).

Interview Q: You set the seed but two training runs still diverge. Why?

A: Seeding alone isn't enough on GPU. cuDNN autotuning (`benchmark=True`) and non-deterministic CUDA kernels introduce run-to-run variation; multi-worker DataLoaders fork RNG state; and floating-point reductions aren't associative across thread orders. Full determinism needs `use_deterministic_algorithms(True)` , `torch.backends.cudnn.deterministic=True` , `CUBLAS_WORKSPACE_CONFIG` , and per-worker seeding - at a speed cost.

`logging_utils.py` - one logger, consistent format

`get_logger(name)` returns a logger namespaced under a single `visiondoc` root, configured exactly once (`_CONFIGURED` guard). If the optional `rich` library is installed you get colored output; otherwise it falls back to the stdlib format - logging never becomes a hard dependency on `rich` . Third-party module names like `"training.train"` get re-parented under `visiondoc` . so everything shares one level and format.

`image_utils.py` - anything-to- `PIL.Image` , plus visualization

Everything that turns *some* input (a path, raw bytes, base64 string, NumPy array, PDF page) into a normalized `PIL.Image` lives here, so the API and Streamlit app never re-implement decoding. (**PIL/Pillow** is Python's standard image library; a `PIL.Image` is the in-memory picture object.)

- `load_image(source)` is the single entry point: it dispatches on type and always returns an **RGB** image (documents may arrive grayscale, CMYK, or with an alpha channel). It even sniffs base64 with a cheap `_looks_like_base64` heuristic so the JSON API can pass image payloads as strings.
- `resize_keep_aspect(img, max_side)` only *downscales* (never upscales) to cap tokens/VRAM while preserving aspect ratio - distorting a document would smear the text.
- `normalize_box` / `denormalize_box` convert between absolute pixels and `[0,1]` fractions. A **bounding box** is a rectangle `(x0, y0, x1, y1)` marking a region on the page.
- `draw_boxes` and `overlay_heatmap` power the "highlight important regions" and attention-visualization features. `overlay_heatmap` min-max-normalizes a 2-D array, resizes it to the image, colorizes it (matplotlib 'jet' if present), and alpha-blends.
- `pdf_to_images` lazily imports PyMuPDF to rasterize PDF pages (with a `max_pages` guard).

Key insight: Note the recurring pattern - heavy or optional dependencies (`fitz` /PyMuPDF, `matplotlib` ,later `datasets` , `peft` , `transformers`) are imported *inside functions*, not at module top. This keeps import surfaces tiny: you can read a signature or compute a fingerprint without dragging in hundreds of megabytes, and a missing optional dep degrades to a clear error instead of breaking `import` .

`ocr.py` - an optional classical fallback

The VLM answers from pixels directly, but `OCREngine` wraps Tesseract (**OCR** = Optical Character Recognition, reading text out of an image) for two bonus cases: (1) an *abstention fallback* when the model returns an empty/low-confidence answer, and (2) *region grounding* - `find_answer_box` slides the model's answer string over OCR word boxes to highlight *where* on the page it appears. Crucially, `pytesseract` is imported lazily and probed with `get_tesseract_version()` ; if Tesseract isn't installed the engine reports `available = False` and every method degrades to a no-op (empty list / empty string / `None`) instead of crashing a request.

12.3 preprocessing/schema.py - the DocSample data contract

This one small file is the spine of the whole data side. `DocSample` is a dataclass representing **one document-understanding example**, and every loader must produce a list of these.

```
@dataclass
class DocSample:
    image: Any                # PIL image OR a lazy path/URL string
    question: str             # NL question, or a synthesized extraction prompt
    answer: str               # gold answer (JSON string for extraction tasks)
    answers: list[str] = ...  # all acceptable answers (DocVQA gives several)
    boxes: list[NormBox] = ... # normalized [0,1] bounding boxes
    box_labels: list[str] = ...
    sample_id: str = ""       # stable id for caching / error analysis
    task: str = "qa"          # "qa" | "extraction" -> picks a prompt template
    metadata: dict = ...
```

Two design details matter:

- `__post_init__` **guarantees** `answers` **always contains the primary** `answer`. Metrics like **ANLS** (Average Normalized Levenshtein Similarity - a fuzzy string match used for DocVQA that scores against the *best* reference) need the full list; this invariant means scoring code never has to special-case a missing primary answer.
- `to_record` / `from_record` round-trip a `DocSample` to/from a plain dict so it can live in a Hugging Face `datasets.Dataset` (and be cached to disk). The image is stored as-is: PIL objects get the `datasets.Image` feature; path strings stay lazy.

The `task` field is the little switch that lets a model adapter pick the right prompt: QA datasets carry a real question; extraction datasets (CORD/SROIE/FUNSD) carry a *synthesized* instruction and a JSON `answer`.

12.4 preprocessing/datasets.py - normalizing four corpora into DocSamples

This is the busiest file in the layer. Its job: funnel four very different corpora - **DocVQA** (document visual question answering), **CORD** (receipt parsing), **FUNSD** (form understanding), **SROIE** (receipt key fields) - through per-dataset loader functions that each return `list[DocSample]`. Because everything downstream sees only `DocSample`, swapping corpus is a one-line YAML change (`data.dataset_name`) with **zero** code edits anywhere else.

Defensive, mirror-tolerant row reading

Community mirrors of the same dataset disagree constantly about column names (`image` vs `img`, `ner_tags` vs `labels`). Rather than hard-code one spelling, every field access goes through `_first(row, *keys)`, which returns the first present non- `None` value. There's a small family of these tolerant helpers:

- `_canonical_split` normalizes `val` / `dev` / `eval` -> `"validation"`, and *raises* on an unrecognized split rather than silently mis-routing data.
- `_normalize_box` copes with the fact that layout datasets encode boxes in three incompatible scales - already-normalized `[0,1]`, absolute pixels, or the LayoutLM-style `0..1000` convention - detects which, and clamps the result to `[0,1]` so a bad box can't crash `draw_boxes`.

- `_flatten_fields` turns CORD's deeply nested ground-truth tree (`{ "menu": [ { "nm": ..., "price": ... }, ... ] }`) into a flat `{ dotted.key: str }` dict, which is both a friendlier *generation target* for a VLM and trivial to score field-by-field.
- `_entities_from_tokens` re-assembles BIO/BIOES token tags (the `B-` / `I-` prefixes marking the Beginning/Inside of an entity) into `{ entity: text }` groups plus normalized boxes.

Each loader (`_load_docvqa` , `_load_cord` , `_load_funsd` , `_load_sroie`) applies these to emit clean `DocSample` s - DocVQA as `task="qa"` keeping *all* answers, the rest as `task="extraction"` with a JSON answer. FUNSD even supports two on-Hub shapes (a linked-entity `form` list vs. token-level `words` / `ner_tags`) and prefers the richer one.

Deterministic split carving (the part interviewers love)

Not every corpus ships a labelled validation *and* test split. DocVQA's test set is an unlabelled challenge set; FUNSD/SROIE ship only train+test. A `_DatasetSpec` in the `_REGISTRY` records which canonical splits are `native` :

```
_REGISTRY = {
    "docvqa": _DatasetSpec(_load_docvqa, {"train": True, "validation": True, "test": False}),
    "cord": _DatasetSpec(_load_cord, {"train": True, "validation": True, "test": True}),
    "funsd": _DatasetSpec(_load_funsd, {"train": True, "validation": False, "test": True}),
    "sroie": _DatasetSpec(_load_sroie, {"train": True, "validation": False, "test": True}),
}
```

Any split marked `False` is *carved out of the train pool*, and it's done **deterministically** by `_partition_pool` :

```
indices = list(range(len(pool)))
random.Random(config.seed).shuffle(indices) # seeded -> same order every time
# reserve validation first, then test, in a FIXED order; the rest is train
```

Because the shuffle is seeded and the reservation order is fixed (`validation` then `test`), the partition is a *pure function* of (data, seed, fractions). Re-running the pipeline on any machine reproduces the identical train/val/test membership - non-negotiable if your eval numbers are to mean anything. The reservations are disjoint (each split takes the next contiguous slice of the shuffled indices), so there's no leakage, and each carved split gets at least one example so downstream code never hits an empty eval set.

Interview Q: A dataset has no validation split. How do you make one without poisoning your metrics?

A: Carve it from train with a *seeded* shuffle so the partition is reproducible across machines, take disjoint contiguous slices so no example lands in two splits, and make the split membership a pure function of (seed, fractions, data) - never a fresh `random` call per run. Do the carve *before* any augmentation or caching so train-only augmentation can't leak into val/test.

`load_samples` ties it together: canonicalize the split, look up the spec, and if the requested split is carved (or is train when carving shrinks it) build from the shared seeded partition; otherwise load the native split directly. Finally it applies `max_train_samples` / `max_eval_samples` caps so a smoke run finishes in minutes. `build_splits` just calls it for all three splits; `to_hf_dataset` converts the list to a `datasets.Dataset` and casts the `image` column to the `datasets.Image` feature for efficient on-disk storage.

12.5 `transforms.py` - document-safe augmentation

Data augmentation means randomly perturbing training images so the model generalizes instead of memorizing. But documents are a minefield: the usual vision transforms - horizontal flips, 90° rotations, heavy crops, hue shifts - *destroy the very signal we care about*. A horizontally flipped invoice is unreadable and would teach the model nonsense.

`DocumentAugmenter` therefore restricts itself to perturbations a real scanner or phone camera would plausibly introduce: **+/-2° rotation**, brightness/contrast jitter, sharpness jitter + light Gaussian blur, mild Gaussian pixel noise, and occasional grayscale (many real docs are B/W faxes). Each op fires only with some probability (e.g. `_P_ROTATE = 0.5`) so the model still sees plenty of clean examples. It uses only Pillow + NumPy (no torchvision) and holds its own seeded `RandomState` for reproducibility. `build_augmenter` returns `None` when `data.augment` is false so eval/inference - which *must* be deterministic - trivially opts out.

Gotcha: rotation uses `expand=False` with a white fill. Keeping the canvas size stable means batched tensor shapes stay predictable, and white matches paper background so the corners don't introduce black artifacts the model would learn as signal.

12.6 `cache.py` and `dataset.py`

`cache.py` persists the finished, split-carved `DatasetDict` to disk so you don't re-download and re-map every launch. The trick is *how it keys the cache*: `cache_fingerprint` hashes **only the config fields that actually change the produced data** - dataset name/id/subset, split names, image size, seed, fractions, and sample caps - with `sort_keys=True` for a canonical hash. Change the learning rate or model id and the cache is *reused*; change the seed or a split fraction and you get a *fresh* cache directory automatically.

Interview Q: Why fingerprint the config instead of caching to a fixed path?

A: Because silently training on a stale split is a brutal, hard-to-find bug. Two runs with different seeds/fractions/caps must not collide on one cache dir. Hashing the data-affecting subset of the config guarantees a fresh cache the moment any of those knobs changes, while identical configs reuse it. `load_cached` also swallows read errors and returns `None`, so a corrupt cache triggers a clean rebuild rather than aborting.

`dataset.py` is the thin `torch.utils.data.Dataset` adapter, `DocumentDataset`. A **map-style dataset** just answers "give me example `i`" via `__getitem__` and "how many?" via `__len__`. Its two deliberate choices: **lazy image loading** (decode via `load_image` only inside `__getitem__`, for the indices a worker is currently fetching - large corpora never sit fully decoded in RAM) and **augmentation at fetch time** (so every epoch sees freshly perturbed pixels; eval passes `augmenter=None`). It returns plain record dicts - *not* tensors - because tensorization is the model's job (next section). It even inherits from `object` if torch isn't importable, keeping the module usable in a torch-less context.

12.7 `models/base.py` - the `VisionDocModel` contract + registry

Now the model side. Everything downstream talks to a `VisionDocModel`, never to Qwen or Donut directly. An abstract base class (`ABC`) defines the contract; concrete backbones fill in the blanks.

The registry: config string -> class

```

_MODEL_REGISTRY: dict[str, type["VisionDocModel"]] = {}

def register_model(name):
    def _wrap(cls):
        _MODEL_REGISTRY[name] = cls
        cls.model_type = name
        return cls
    return _wrap

```

A subclass decorated `@register_model("qwen2_5_vl")` registers itself under that key. `build_model(config)` lazily imports the implementation modules (which triggers their registration), then looks up `config.model.model_type` and instantiates the matching class. Swapping backbones is a pure config change with no call-site edits - the payoff of the whole abstraction.

The three responsibilities and the abstract hooks

A `VisionDocModel` owns **loading** (weights + processor onto the right device/dtype), **adaptation** (LoRA wrapping + freezing), and **tensorization** (samples -> tensors and back). The abstract methods each subclass *must* implement are the tensorization hooks - the only places backbones truly differ:

Hook	Responsibility
<code>load()</code>	Load weights + processor, set eval mode.
<code>collate_train(batch)</code>	Turn samples into a padded supervised batch (<code>input_ids</code> , <code>attention_mask</code> , <code>labels</code> with prompt masked to <code>-100</code> , plus vision tensors). Passed straight to the HF <code>Trainer</code> as its <code>data_collator</code> .
<code>prepare_inference_inputs(images, questions)</code>	Build generation-ready inputs on device.
<code>decode(generated_ids, prompt_width)</code>	Turn output token ids back into strings.
<code>default_lora_target_modules</code>	Fallback list of layers to adapt.

Everything *shared* - the generation loop, confidence math, LoRA plumbing, adapter I/O - lives on the base class so each backbone stays small.

Shared LoRA plumbing

`apply_lora()` wraps the backbone with **LoRA** (Low-Rank Adaptation - see Ch. 6/7 for the theory: instead of updating all billions of weights, you train tiny low-rank "adapter" matrices bolted onto chosen layers and freeze everything else). The method builds a `peft.LoraConfig` from the typed `LoRAConfig` , enables input-require-grads when gradient checkpointing is on (the standard PEFT recipe), wraps via `get_peft_model` , and logs the trainable-vs-total parameter count - typically well under 1%. `save_adapter` / `load_adapter` persist and re-attach just the adapter, and `merge_and_unload` folds LoRA weights back into the base for adapter-free inference.

Key insight: `load_adapter` validates a local path *before* handing it to PEFT. PEFT otherwise misreads a missing local directory as a Hub repo id and throws a confusing `HFValidationError` . The code raises a precise `FileNotFoundError` explaining the usual cause - running eval before training finished writing the adapter. This kind of *error-message engineering* is exactly the "production maturity" signal interviewers probe for.

Generation + confidence (the shared crown jewel)

`generate()` is decorated `@torch.no_grad()` (no gradient tracking - we're not training) and handles both single and batched `(image, question)` input. It calls the subclass's `prepare_inference_inputs`, records `prompt_width`, then calls `self.model.generate(...)` with `return_dict_in_generate=True`, `output_scores=True` so it gets per-step score distributions back.

The confidence score comes from `_token_logprobs_from_scores`, a carefully-written free function (kept module-level so it's unit-testable). For each generated step it does a `log_softmax` over the vocabulary and gathers the log-probability of the *actually chosen* token. Three correctness details are worth calling out:

- **Generated tokens occupy the trailing** `len(scores)` **positions** of the sequence regardless of left/right padding, so it computes `gen_start = sequences.shape[1] - len(scores)`.
- **Per-sequence accumulation stops at the first EOS** (end-of-sequence token) so padding on early-finished sequences isn't counted.
- **Beam search** uses `beam_indices[b, t]` to find which flattened beam row produced example `b`'s token - you can't assume `row == example`.

Then `_sequence_confidence` maps those log-probs to `[0,1]`: `seq_prob = exp(mean log p)` (the geometric-mean token probability), or `min_prob` = the least-confident token's probability (a conservative floor for flagging risky extractions). This gives the API a calibrated-ish confidence *without* a separate scoring head.

Interview Q: How do you attach a confidence score to a free-text generation without training a separate model?

A: Ask `generate` for `output_scores`, `log_softmax` each step, gather the chosen token's log-prob, average them, and exponentiate - that's the geometric mean per-token probability, a proxy for the sequence likelihood. Stop at EOS so pads don't inflate it, handle the beam-index remap, and optionally report the min-token probability as a conservative floor. It's cheap and correlates with correctness, though it isn't truly calibrated.

12.8 `models/qwen_vl.py` - the default causal-VLM backbone

Qwen2.5-VL is a **causal language model** (it generates left-to-right, one token at a time, each conditioned on all previous ones) with vision folded in via image tokens. The adapter fills in the hooks:

- `load()` builds an `AutoProcessor` with `min_pixels` / `max_pixels` to bound Qwen's dynamic-resolution tiling (the main VRAM lever), optionally attaches a bitsandbytes 4-/8-bit `quantization_config` (CUDA only - else it warns and ignores), and places the model.
- `collate_train` **uses RIGHT padding and masks the prompt**. It builds a chat-format message list, tokenizes the prompt-only version *and* the full (prompt+answer) version, computes the prompt length (including expanded image tokens), then sets `labels` to `-100` for every pad token *and* for the whole prompt span - so cross-entropy loss is computed **only over the answer tokens**. (`-100` is PyTorch's magic "ignore this position" label.)
- `prepare_inference_inputs` **uses LEFT padding**.

That left-vs-right padding split is the single most important - and most interview-tested - detail in this file.

Interview Q: Why does Qwen use *right* padding for training but *left* padding for generation?

A: In training every position's label is fixed, so right-padding keeps the *prompt prefix* aligned across the batch, which makes per-sample prompt-length masking correct and lets pads be ignored via the attention mask. In generation you append new tokens to the *end* of each sequence; if sequences were right-padded, shorter ones would have their new tokens land after pad positions and misalign across the batch. Left-padding pushes all real tokens flush to the right so every sequence's "next token" starts at the same column. `decode` then slices off `prompt_width` because a causal LM returns prompt + completion.

12.9 `models/donut.py` - the seq2seq alternative (proving swappability)

`Donut (naver-clova-ix/donut-base)` is a **Vision-Encoder-Decoder**: a Swin image encoder feeds a BART-style text decoder. It exists mainly to *prove the abstraction works* - selecting it is a one-line config change. Its tensorization differs fundamentally from Qwen:

- **No image token** - pixels enter through the encoder as `pixel_values`; there's nothing to interleave into the text.
- **Seq2seq generation** - `generate` returns *only* the decoder output, seeded by a task/question prompt supplied as `decoder_input_ids`. So `decode` ignores `prompt_width` in the causal sense and instead strips a `_decoder_prompt_width` it recorded per batch.
- **Task-specific special tokens** - `<s_docvqa>`, `<s_cord-v2>`, etc., are added to the tokenizer and the decoder's embedding table is grown (`resize_token_embeddings`).

The most instructive part is that `DonutModel` **overrides** `generate` **to loop one example at a time**:

```
# Donut conditions generation on a decoder prompt whose length varies per sample.
# Right-padding the prefix makes generate emit the first token from a PAD position;
# left-padding shifts BART's absolute positions. Neither is safe -> loop batch-1.
outs = [super().generate(im, q) for im, q in zip(imgs, qs)]
```

Because a batch-1 call needs no padding at all, both hazards vanish - at the cost of throughput, an acceptable trade for a non-default backbone.

Interview Q: Qwen batches generation fine, but Donut generates one sample at a time. Why the difference?

A: Donut is seq2seq and conditions on a `decoder_input_ids` prompt whose length varies across samples. Right-padding that decoder prefix would make `generate` produce the first token from a PAD position; left-padding it would shift BART's *learned absolute* position embeddings and corrupt decoding. With no safe padding side, the correct fix is to run batch-1 (no padding involved) and accept lower throughput. Qwen, a causal LM with a chat template, has no such per-sample decoder-prefix constraint and left-pads cleanly.

`models/lora.py` rounds out the layer with pure parameter-accounting helpers (`count_parameters`, `format_parameter_summary`, `peft_config_from`) that the training logs, eval report, and research comparison table all share - kept separate from `base.py` so they can be unit-tested without loading a model.

12.10 The through-line

Read this layer top to bottom and one philosophy repeats at every level:

- **Two contracts** (`DocSample` , `VisionDocModel`) decouple data from models so both sides evolve independently.
- **Fail loud, fail early** - strict config keys, unknown-split errors, adapter-path validation - because a silent wrong default costs a GPU-day.
- **Determinism on purpose** - seeded splits, fingerprinted caches, opt-out augmentation - because an eval number you can't reproduce is worthless.
- **Lazy, optional dependencies** - heavy libraries imported inside functions, graceful no-ops when a tool is missing - so the core stays importable and portable across a laptop, CI, and a GPU box.

With these foundations in hand, Ch. 13 walks the layers that *use* them: the training loop, evaluation metrics, inference server, and the Streamlit app.

13. Inside the Codebase II - Training, Evaluation, Inference, Serving, Deployment

Chapter 12 walked the "front half" of the project - configs, data, and the model wrapper. This chapter walks the "back half": the code that actually *trains* the adapter, *measures* whether it helped, *answers* real documents, and *ships* the whole thing behind an HTTP API and a web dashboard. We read the real files so every claim here is true of the repo. By the end you will be able to run the full loop yourself and, just as importantly, read the numbers it produces without being fooled by them.

A one-line map of the packages we cover:

Package	Job	Key files
training/	Turn a config into a running fine-tune	trainer.py , train.py , callbacks.py
evaluation/	Score quality + serving cost, write a report	metrics.py , profiler.py , evaluate.py , report.py
inference/	Answer questions / extract fields at serving time	predictor.py , extract.py , batch.py
research/	Base-vs-fine-tuned comparison	compare.py
visualization/	Charts (loss curves, comparison bars, ...)	plots.py
api/ , app/	FastAPI service + Streamlit dashboard	main.py , streamlit_app.py
docker/	Containerized deployment	Dockerfile , docker-compose.yml

13.1 Training - training/trainer.py and training/train.py

What "training" means here, in plain terms

Training (or *fine-tuning*) means showing the model many (document image, question, correct-answer) examples and nudging its weights so its answers get closer to the correct ones. Because we use **LoRA** (Low-Rank Adaptation - see Ch. 7), we freeze the giant pretrained model and only train a tiny set of extra "adapter" weights. Everything in these two files is the plumbing that makes that happen reliably.

The project uses Hugging Face's `Trainer` - a prebuilt training loop that handles the epoch/step/optimizer/checkpoint machinery so you do not write a `for` loop over batches by hand. You configure it with a `TrainingArguments` object (a big bag of hyperparameters: learning rate, batch size, how often to evaluate, etc.).

Interview Q: Why use HuggingFace `Trainer` instead of writing your own PyTorch training loop?

A: `Trainer` gives you a battle-tested loop with gradient accumulation, mixed precision, distributed/multi-GPU support, checkpointing, resumption, logging integrations, and a callback system - all correct and edge-case-hardened. Writing that yourself is a large surface for subtle bugs (e.g. LR-scheduler stepping, gradient clipping order). You reserve a custom loop only when you need behavior `Trainer` cannot express, like a nonstandard multi-model optimization.

`trainer.py` - the config-to-Trainer translation layer

This module is deliberately the *single* place that maps the project's typed `TrainingConfig` onto HuggingFace's `TrainingArguments`. `build_training_arguments(config)` forwards every field - epochs, per-device batch size, gradient accumulation, learning rate, weight decay, warmup ratio, LR scheduler, precision flags, eval/save cadence, seed, dataloader workers. A few choices are load-bearing and worth understanding:

- `remove_unused_columns=False`. By default the `Trainer` strips any dataset column whose name does not match the model's `forward()` signature - *before* the collator runs. Our dataset yields raw record dicts (`image` , `question` , `answer` , ...) that the custom collator (`collate_train`) turns into tensors, so if the Trainer stripped them first there would be nothing left to collate. Disabling that is mandatory.
- `label_names=["labels"]`. Tells the Trainer where the supervision (the target token ids) lives, so it can still compute an evaluation loss.
- **bf16 vs fp16.** *Precision* is how many bits represent each number. **bf16** (bfloat16) and **fp16** (float16) are both 16-bit "mixed precision" formats that halve memory and speed up math versus 32-bit. They are mutually exclusive; if a config sets both, the code warns and prefers bf16 (wider dynamic range, the right default on modern NVIDIA "Ampere+" GPUs). On an older T4 GPU there is no bf16, so the Colab configs use fp16 (§13.7).
- **Version-robust construction.** `_construct_training_arguments` inspects the *installed* `TrainingArguments.__init__` signature and reconciles cross-version drift: it renames `eval_strategy` <-> `evaluation_strategy` and drops kwargs an older/newer `transformers` does not understand (with a warning) instead of crashing. This is why the same code runs across a range of library versions.

Key insight: `remove_unused_columns=False` is one of the most common "my custom collator gets empty batches" bugs when people first use `Trainer` with non-standard data. Knowing it exists is a strong signal you have actually trained with a custom pipeline.

Why the eval loop metric is `eval_loss` , not ANLS/EM. `make_compute_metrics()` deliberately returns `None`. The reasoning drives everything downstream:

- The vanilla `Trainer` eval loop is **teacher-forced** - it feeds the model the correct previous tokens and looks at next-token logits. It hands `compute_metrics` the raw *logits* over the whole vocabulary, shape (`batch` , `seq_len` , `|V|`) . For a VLM that tensor is enormous; accumulating it across an eval set reliably runs out of memory (**OOM**).
- Worse, teacher-forced accuracy is *not* the metric we care about. Real document-QA quality (ANLS, exact match) requires **autoregressive generation** - the model generating its answer token-by-token on its own - which the base `Trainer` does not do.

So the code selects checkpoints on `eval_loss` (a cheap, always-available proxy) during training, and computes the *real* generation metrics exactly once, after training, via `evaluation.evaluate_model`. `build_trainer` has a safety net: if no `compute_metrics` is supplied but the config asked to track (say) `eval_anls` for best-model

selection, it switches `metric_for_best_model` to `eval_loss` and forces `greater_is_better=False` (lower loss is better) - otherwise `load_best_model_at_end` would search for a metric that never gets produced and crash at the first evaluation.

Interview Q: You are LoRA-fine-tuning a generative VLM. Why not compute ANLS inside the training eval loop and select checkpoints on it?

A: The standard `Trainer` eval loop is teacher-forced and exposes only full-vocabulary logits, which (a) OOM when accumulated for a VLM and (b) measure next-token accuracy, not free-running answer quality. ANLS/EM need real autoregressive generation (`Seq2SeqTrainer` + `predict_with_generate` , which does not fit this multimodal causal-LM setup cleanly). The pragmatic, honest choice is: select on `eval_loss` (a good, cheap proxy) during training and run true generation-based metrics once, post-hoc, on a held-out subset.

`build_trainer` also wires:

- `model=model.model` - the underlying PEFT-wrapped `nn.Module` (the `VisionDocModel` wrapper is not itself a module, so we unwrap it).
- `data_collator=model.collate_train` - the backbone-specific tensorizer.
- Callbacks (below): throughput + GPU-memory always; early stopping only when evaluation is on, `patience > 0`, and `load_best_model_at_end` is true (the `EarlyStoppingCallback` asserts that flag, so the code skips it with a warning rather than crashing when it is off).
- The processor, attached under `processing_class` (new transformers) or `tokenizer` (old), so saved checkpoints are self-contained and reloadable for inference.

`callbacks.py` - observability and early stopping

A **callback** is a hook object the `Trainer` calls at defined moments (`on_log` , `on_evaluate` , `on_train_begin`) so you can add behavior without subclassing the loop. Three small ones:

- `build_early_stopping` - halts the run after `early_stopping_patience` consecutive evaluations with no meaningful improvement, saving GPU hours on a plateau.
- `GPUMemoryCallback` - at each evaluation logs CUDA allocated/reserved/peak memory, and folds those into the metrics dict (namespaced `gpu_*`) so they persist in `trainer_state.json` and can be plotted later. On CPU/MPS the helper returns zeros, so it is a harmless no-op.
- `ThroughputCallback` - between two `on_log` events computes a *windowed* `samples/second` (`delta_steps` x `per_device_batch` x `grad_accum` x `world_size` / `delta_time`). A rolling figure reveals mid-run stalls (e.g. a checkpoint save) that the single end-of-run average hides.

Gotcha: The samples-per-second math multiplies by `gradient_accumulation_steps` and `world_size` . One logged optimizer step corresponds to several micro-batches (accumulation) across several data-parallel replicas (world size), so forgetting either factor under-reports throughput by that multiple.

`train.py` - the end-to-end pipeline

`main()` runs a fixed sequence (kept identical in order to the evaluation CLI so numbers stay comparable):

- 1 **Load config** (`--config` , or `$VISIONDOC_CONFIG` , or `configs/default.yaml`).

- 2 **Seed** for reproducibility - with `deterministic=False`, because bit-exact determinism disables fast GPU kernels and materially slows LoRA runs; strict determinism is reserved for evaluation/debugging.
- 3 **Device + experiment-tracking setup.** If `report_to=wandb` (Weights & Biases, an experiment-tracking dashboard), it sets `WANDB_MODE=offline` by default so a CI box with no credentials logs locally instead of blocking on a login prompt.
- 4 **Build model** -> `apply_lora()` and log the parameter budget (trainable vs total - the LoRA efficiency story).
- 5 **Obtain splits** via cache-first resolution (`_obtain_splits`), wrap in the map-style `DocumentDataset` (augmentation on train, deterministic on eval).
- 6 **Build trainer** -> `trainer.train(resume_from_checkpoint=...)`.
- 7 **Persist artifacts:** save train metrics + `trainer.save_state()` (writes `trainer_state.json`, whose `log_history` feeds the plots), the LoRA adapter under `<output_dir>/adapter`, and the fully-resolved config as `run_config.yaml`.
- 8 **Final (real) evaluation** - `_final_evaluation` runs true generation on a capped validation subset (`max_eval_samples` or 200) and writes an HTML report under `<output_dir>/final_eval`. This is the run's headline quality number.

Key insight: Two evaluation regimes on purpose: cheap teacher-forced `eval_loss` during training (for checkpoint selection), and expensive-but-honest generation metrics once at the end. Conflating them is the single most common way people report inflated training-time "accuracy."

Heavy imports (`torch/transformers/datasets`) are deferred *inside* functions so merely importing `training.train` (which the package `__init__` does) stays cheap.

13.2 Evaluation - measuring quality and cost

`metrics.py` - the scoring toolbox

Document intelligence has several answer shapes (free-form QA, short exact fields, multi-key extraction), so no single number tells the whole story. This module is a toolbox of metrics plus one `aggregate_qa_metrics` roll-up. First, define the terms:

- **Normalization** (`normalize_text`): before comparing, lowercase, replace punctuation with spaces, drop the articles *a/an/the*, and collapse whitespace. This is the "SQuAD-style" convention so `"Total: $4.00"` and `"total 4 00"` score as semantically equal. Analogy: judging a spelling answer by the word, not the handwriting.
- **Exact Match (EM):** 1.0 if the normalized prediction equals *any* acceptable reference, else 0. (DocVQA answers come as a set of annotator variants, so a match against the best reference counts.)
- **ANLS** (Average Normalized Levenshtein Similarity) - the standard DocVQA metric. *Levenshtein distance* is the minimum single-character edits (insert/delete/substitute) to turn one string into another. ANLS computes `1 - (distance / longer_length)` (a similarity in [0,1]) against the best reference, then **zeros anything below a 0.5 threshold**. So a near-miss ("Aple" vs "Apple") earns partial credit, but a genuinely wrong answer earns nothing rather than leaking similarity points.
- **Token-level P/R/F1:** treat each answer as a bag of words. *Precision* = fraction of predicted words that are correct; *recall* = fraction of gold words you found; *F1* = their harmonic mean. Rewards partial answers ("New York" when gold is "New York City").

- **BLEU / ROUGE:** corpus-level overlap metrics borrowed from machine translation (BLEU, precision-oriented n-gram overlap) and summarization (ROUGE, recall-oriented). Informative for longer extractive answers.
- **Structured field P/R/F1** (`structured_field_metrics`): micro-averaged over (key, value) pairs across documents - a predicted field is a true positive only when the key exists in gold *and* the normalized value matches.

A recurring design theme: **graceful degradation over hard dependencies**. The heavy scoring libraries (`Levenshtein` , `sacrebleu` , `rouge_score` , `scikit-learn` , `nltk`) are *optional*, imported lazily inside the function that needs them; if one is missing the code logs a debug line and returns a neutral `0.0` or a pure-Python fallback (e.g. a two-row dynamic-programming Levenshtein) rather than crashing the whole evaluation.

`aggregate_qa_metrics(preds, refs, confidences)` rolls per-sample metrics (averaged) and corpus metrics (BLEU once, ROUGE per primary reference) into one flat dict with a **stable schema** - every key is always present (even if a metric degraded to 0) so downstream report tables and DataFrames never break.

Interview Q: Why is ANLS the primary metric for document VQA instead of plain accuracy?
A: Document answers come off noisy OCR and free-form generation, so exact string match is unforgivingly brittle - a single misread character scores 0 despite an essentially-correct answer. ANLS uses normalized edit-distance similarity so near-misses get partial credit, but a 0.5 threshold zeroes genuinely wrong answers so it does not reward garbage. It is the DocVQA community standard, which also makes results comparable to published numbers.

`profiler.py` - how much does it cost to serve?

A fine-tune is only "production ready" if you can quote its serving cost. `InferenceProfiler` (a context manager) and `measure_latency` (a one-shot probe) provide:

- **Latency** - wall-clock time per document (`time.perf_counter` , deliberately end-to-end, not raw CUDA kernel time, because the user-visible /ask latency includes pre/post-processing). Reported as mean, p50 (median), and p90 percentiles. A *percentile* p90 = "90% of requests were at least this fast"; tail latency matters for SLAs.
- **Throughput** - serial samples/second (derived from total measured time).
- **Peak GPU memory** - the high-water VRAM mark inside the profiled window.

Two details worth internalizing: **warmup is mandatory** (the first call after model load pays one-time CUDA context/autotune costs that never recur; reporting it would slander the model, so warmup passes are discarded), and the code reports the **median** of the timed calls (robust to a single GC/scheduler outlier that would skew a mean). Percentiles are computed with a hand-rolled linear-interpolation helper so the profiler has zero heavy dependencies and behaves identically on the CPU CI runner and the GPU box.

`evaluate.py` - the driver

`evaluate_model(model, samples, config, profile=True)` runs the model over a split and returns a report-ready dict with a stable schema: `metrics` , `profile` , `predictions` (per-sample records), `params` (trainable/total counts), `device` . Notable choices:

- **Batched generation.** It calls `model.generate` on lists sized by `config.inference.batch_size`, because batching is how the real endpoint amortizes the (expensive) vision encoder. Per-batch wall-clock is spread evenly across its samples to get an amortized per-document latency that reflects production, not an artificially slow one-at-a-time loop.
- **Never crash a long eval on one bad sample.** A single undecodable image becomes a tiny blank placeholder (keeping the batch aligned with its questions); a generation error records empty predictions for that batch and continues. Crucially, failures still count in the metric *denominator* - a crash on hard documents is a miss, not a silently shrunken test set.
- **Extraction scoring.** For samples tagged `task == "extraction"`, both model output and gold answer are parsed as JSON (via `inference.extract.parse_json_answer`) and scored with `structured_field_metrics`, merged into the same flat metrics dict.

The CLI (`python -m evaluation.evaluate`) mirrors training's wiring order, supports `--adapter` (overriding the config immutably, since `ProjectConfig` is a frozen dataclass), `--split`, `--max-samples`, and `--report`, and prints a terse JSON headline for CI logs.

`report.py` - a self-contained HTML report

`build_report` turns the results dict into one **inline-styled HTML file** with no external CSS/JS/CDN - it opens anywhere (email, PR comment, archived folder). Alongside it, it always dumps the raw `evaluation_results.json` (written *first*, so the authoritative numbers survive even if HTML assembly hits an edge case). Everything interpolated (predictions, gold text) is **HTML-escaped** so document content can never break or XSS the page. The report has sections for run context, quality metrics (curated label order), inference profile, parameter footprint, and up to 10 sample-prediction rows with a confidence bar and a *loose* [OK]/[x] hint (the header notes the real scoring is SQuAD-normalized). Missing sections degrade to a short note instead of raising.

Key insight: "Write the JSON before the HTML" is a small resilience pattern with a big payoff - a rendering bug never costs you the numbers of a multi-hour eval.

13.3 Inference - `predictor.py`, `extract.py`, `batch.py`

`DocumentPredictor` - the one facade everyone uses

`inference/predictor.py` defines the single object the API, the Streamlit app, and batch jobs all talk to. It owns three moving parts and hides their wiring: a loaded `VisionDocModel` (optionally LoRA-adapted), an optional `OCREngine` for *region grounding*, and a lazily-built `FieldExtractor`.

OCR (Optical Character Recognition) reads the printed text and its pixel coordinates off the page. Here it is used only for *grounding* - locating the answer string on the image to draw a highlight box - not to answer the question. Every entry point returns the same `PredictionResult` (answer, confidence, latency, region boxes, an optional base64 PNG with the boxes drawn, and a `raw` diagnostics dict), built in one private `_build_result` so single/batch/PDF paths are byte-for-byte consistent.

The adapter fallback is the safety-critical detail for interviews. When the predictor builds its own model and a `config.inference.adapter_path` is set, it attempts `load_adapter` inside a try/except: a bad or missing adapter path logs the exception and **falls back to serving the base model** (which still answers, just zero-shot) rather than bricking the service.

Other graceful-degradation behaviors: no Tesseract -> highlighting silently disabled (regions come back empty); a PDF page that fails to generate -> an empty result for that page, not an aborted document (`predict_pdf` answers each page independently). All heavy imports are deferred into methods so importing the module while the API is cold needs no ML stack.

Interview Q: How should a serving layer behave when the fine-tuned adapter fails to load in production?

A: Degrade, do not crash. The predictor best-effort-attaches the adapter and, on any failure, logs it and serves the base model - the endpoint keeps answering (zero-shot) instead of returning 500s. That is the right availability trade-off for a demo/service; you would pair it with an alert so the degraded state is noticed, and expose the adapter status via `/health`.

`extract.py` - schema-aware prompting, grounding, normalization

Field extraction asks the *same* VLM to return a machine-readable JSON object (`{"total": "42.00", ...}`) for a known document type - implemented as a prompting + parsing + grounding layer, no second model. Four quality levers, each a legitimately good interview talking point:

- 1 **Schema-aware prompt** (`build_extraction_prompt`). Instead of a bare key list, the prompt lists each key *with a one-line description and format hint*, adds explicit rules ("copy exactly; never invent"; money = digits and a decimal point only; dates = `YYYY-MM-DD` when unambiguous), and includes a worked `null`-filled skeleton (`{"invoice_number": null, ...}`). An instruction-tuned VLM honors a described, constrained key contract far more reliably than an open-ended "extract everything." Canonical field sets live in `DOC_TYPE_FIELDS` for invoice/receipt/id_card/form; `resolve_fields` picks explicit fields -> doc-type set -> a compact generic fallback.
- 2 **OCR grounding**. When OCR is available, the page's transcription is pasted into the prompt as *reference text* (truncated to ~1500 chars so a huge page cannot blow the context budget; the image stays authoritative). After generation, each emitted value is checked against the normalized OCR text; values not found are *flagged* (not deleted - flagging protects recall against OCR misses) as a strong hallucination signal.
- 3 **Value normalization**. Money fields are stripped to bare numbers (`"$1,234.50"` -> `"1234.50"`) so downstream comparison/aggregation is numeric-friendly.
- 4 **Defensive parsing** (`parse_json_answer`). It never trusts the model's formatting: strips ````` ``` `json` ``` fences, then extracts the first **balanced** `{...}` span with a brace-counter that respects string literals and escapes, and repairs a trailing comma - falling back to `}` so one malformed generation degrades a single row, never the batch. `never raises`` is part of its contract.

The `extract()` result is a dict: `fields` (schema-projected - every requested key present, missing -> `None`, money normalized), `confidence`, `raw` (undecorated text for audit), `grounded` (`{key: bool}`), and `ocr_used`.

Interview Q: A zero-shot VLM emits messy, sometimes-hallucinated JSON for invoice extraction. What non-training changes improve reliability?

A: Prompt engineering and post-processing: (1) a described, constrained key contract with format rules and a `null` skeleton dramatically cuts garbage; (2) inject OCR text as grounding and flag values absent from the page as likely hallucinations; (3) parse defensively (strip fences, extract the first balanced JSON object, repair trailing commas, never raise); (4) normalize values (money -> bare numbers). These are the single biggest levers *before* you spend a GPU-hour fine-tuning.

`batch.py` - a folder in, a spreadsheet out

`batch_infer_dir(predictor, input_dir, question, output_csv)` walks one directory level, answers `question` for every `.png/.jpg/.jpeg/.pdf`, and returns a pandas DataFrame (one row per image, one row per PDF *page*, 1-based `page`). Files are sorted for deterministic output; any per-file error becomes an empty-answer row carrying the error message (so a 10k-document run finishes and surfaces failures instead of aborting midway). pandas is imported lazily; the column order is stable, with an `error` column appended only if at least one row set it.

13.4 Research - `research/compare.py`

The question a stakeholder actually asks is comparative: *did the fine-tune earn its keep?* `compare_models` puts the **base (zero-shot)** backbone and the **LoRA-adapted** model side by side on the **same** held-out split, with the **same** metric/profiler machinery, and diffs every axis that matters.

Two design points to remember:

- **Load the backbone once.** A Qwen2.5-VL-3B checkpoint is several GB; two `VisionDocModel` objects would pin ~2x host+GPU memory and double the slow weight load. Instead the code builds the base model, evaluates it, then attaches the LoRA adapter *in place* (`load_adapter` wraps the resident base weights in a `PeftModel`) and evaluates again. The wrap is **one-way**, which is exactly why the order is fixed: **base first, then adapter**.
- **Both models see identical samples.** The test split is loaded once and reused, so a `--max-samples` cap cannot confound the comparison with different subsets.

Training time is recovered, not re-measured. A multi-hour fine-tune's wall-clock is read straight out of `trainer_state.json` - the HF `Trainer` appends a terminal `log_history` record with `train_runtime`, `total_flos`, `epoch`, and `step` - located next to the adapter or under `output_dir` (falling back to the newest `checkpoint-*/` state). If no state file exists (e.g. an externally supplied adapter), training time renders as "n/a" rather than a fabricated number.

The output is a comparison dict (`base`, `finetuned` - `None` if no adapter, `delta`, `training_time`, ...), rendered to `comparison.csv` (machine-readable source of truth), `comparison.md` (with ^v improvement markers that respect each axis's preferred direction - higher EM/ANLS/F1, *lower* latency/memory), `comparison.json`, and a bar chart. Cleverly, the "trainable LoRA params" figure is derived as `finetuned.total - base.total` - the number of parameters the adapter *added* - which is the true parameter-efficiency story (a fresh base model has all weights marked trainable, so its raw "trainable" count is meaningless for a zero-shot baseline).

Interview Q: When comparing a base model to its LoRA fine-tune, what must you hold constant, and what is the memory-smart way to run both?

A: Hold the evaluation split, sample cap, metric code, profiler, seed, and device constant so only the adapter changes. For memory, load the multi-GB backbone once, evaluate the base first, then attach the adapter in place (a one-way `PeftModel` wrap) and evaluate again - instead of instantiating two full models. Report deltas with a direction-aware "better" marker since lower latency/memory is an improvement while higher accuracy is.

13.5 Visualization - `visualization/plots.py`

Numbers persuade nobody on their own; this module renders the story in pictures with one consistent visual language. Design guarantees:

- **Headless by construction.** `matplotlib.use("Agg")` is called *at import time, before* `pyplot` is imported, so every consumer (Docker, CI, a Streamlit worker with no display) gets a non-interactive backend with no `$DISPLAY` dependency.
- **Optional deps degrade, never crash.** `seaborn` (nicer theme), `scipy` (KDE), `sklearn` (confusion matrix) are optional; each has a small NumPy fallback so a plot is *always* produced.
- **Every function is a pure "data -> file" transform** that creates its parent dir, writes one tight 150-dpi PNG, **closes the figure** (so a loop over hundreds of samples never leaks figure handles), and returns the `Path`.

The public plots: `plot_training_curves` (train+eval loss on one axis so the generalization gap is visible, plus a validation-metric curve - reading either an in-memory `log_history` list or a `trainer_state.json` path); `plot_confidence_distribution` (histogram + KDE - a calibrated model's confidences should spread across `[0,1]`, not pile at `1.0`); `plot_confusion_matrix`; `plot_prediction_grid` (thumbnails framed green/red using the same normalization as the metrics); `plot_attention_map` (a best-effort 2D reduction of a VLM's `layersxheadsxqueryxkey` attention - explicitly qualitative, with an honest caveat and a placeholder when attention was not captured); and `plot_model_comparison` (the grouped Base-vs-Fine-tuned bar chart consumed by `research/compare.py`, charting only the comparable `[0,1]` quality metrics so it never plots milliseconds next to a 0.3 ANLS).

Gotcha: In a headless environment, importing `pyplot` before selecting the Agg backend can crash or hang trying to reach a GUI backend. `plots.py` puts `matplotlib.use("Agg")` at the very top of the module for exactly this reason - a real-world footgun worth naming in interviews.

13.6 Serving - `api/main.py` and `app/streamlit_app.py`

FastAPI service (`api/main.py`)

FastAPI is a Python web framework that builds an HTTP API (and auto-generates OpenAPI docs) from typed function signatures. The whole file is organized around one constraint: **the model is huge and slow to load, so it must not load at import time.** How that is honored:

- **Lazy, thread-safe predictor singleton.** `get_predictor()` builds the `DocumentPredictor` exactly once, on the first request that needs it, guarded by **double-checked locking** (a fast `is None` check outside the lock for the hot path, then a lock + re-check to guarantee exactly one construction under concurrent first requests - so the multi-GB weights load a single time). Consequently the process boots instantly (good for container health checks/autoscalers) and `/health` answers with no GPU and cold weights.
- **Deferred heavy imports and lazily cached config** (`get_config`), so importing `api.main` (e.g. to generate the OpenAPI schema in a test) costs nothing heavy.

Endpoints:

Route	Method	Purpose
<code>/health</code>	GET	Liveness + configured model/adaptor/device. Never forces the model to load.
<code>/ask</code>	POST	Answer a question about a base64 image (optional highlight).
<code>/extract</code>	POST	Structured fields by explicit list or <code>doc_type</code> .
<code>/predict</code>	POST	Multipart "upload a file + ask", text-only answer.

Error contract: bad client input (undecodable image, empty question) -> **400**; a failure inside inference -> **500** with a short message - never a leaked traceback, but always logged. `/predict` is a `sync def` on purpose: FastAPI runs sync handlers in a worker thread, so the heavy blocking `generate()` call does not stall the async event loop, and it reads the upload via the synchronous `file.file.read()` to stay in that threadpool model.

Gotcha (CORS): The demo sets `allow_origins=["*"]` with `allow_credentials=False`. The code comment flags that a production API must replace `*` with an explicit allowlist - an open, credentialed API is a data-exfiltration risk (and the browser spec forbids `*` together with credentials, which is why credentials stay `False` here).

Interview Q: How do you serve a multi-GB model behind an HTTP API without a slow, fragile cold start?

A: Never load it at import time. Boot the web process instantly, keep `/health` weight-free so orchestrator probes pass, and lazily build the model on the first inference request behind a double-checked lock so concurrent first requests load the weights exactly once. Defer the heavy ML imports into the handlers, and run blocking generation in a threadpool (`sync def`) so it does not block the event loop.

Streamlit dashboard (`app/streamlit_app.py`)

Streamlit turns a Python script into an interactive web app - but it *re-runs the entire script on every interaction*, which makes two things expensive if done naively: loading the model, and the heavy imports themselves.

- `@st.cache_resource` **keyed on** `(model_choice, adapter_path)` builds each distinct predictor once and reuses it across every re-run and browser session. This is the single most important performance lever: without it, every checkbox toggle would reload the multi-GB model.
- Heavy imports live *inside* functions so the page shell renders even without torch or a GPU.
- **Nothing may hard-crash the page:** a missing config, un-loadable model, no Tesseract, no GPU, a corrupt upload - each is caught and shown as an `st.error` / `st.warning`.
- **Base vs. fine-tuned is a first-class toggle.** A sidebar radio flips between the zero-shot base and the LoRA adapter. `_effective_adapter_path` forces the base choice to `None` regardless of the adapter text box - otherwise the whole comparison would be meaningless (and it stabilizes the cache key). A dedicated research tab renders the offline `reports/comparison.csv` produced by `make compare`, with guidance shown when the file is absent.

The app talks to the *same* `DocumentPredictor` the API uses, so a question answered in the dashboard and via `POST /ask` go through byte-for-byte identical code. It renders the answer with a

confidence meter, latency, region highlighting (falling back to the original upload when OCR found nothing), and a live device/GPU panel.

Note (Preview quirk): Per the project's environment notes, the Claude Preview browser does not fire `ResizeObserver`, so some React-Flow-style edge rendering will not appear there - an environment limitation, not a bug in this app.

13.7 Running everything, and reading the results

`docker/` - one image, two services

A single `python:3.11-slim` image serves **both** the FastAPI server (8000) and the Streamlit dashboard (8501); `docker-compose.yml` picks which by overriding the `command`. Building one image keeps the two services byte-for-byte identical and halves build time. Key details:

- System libs installed for the wheels' runtime needs: `libgl1 / libgl1-mesa-glx` (OpenCV's `libGL.so.1`), `poppler-utils` (PDF->image via `pdf2image`), `tesseract-ocr` (OCR), `curl` (the compose healthcheck probing `/health`).
- **Layer caching:** `requirements.txt` is copied and installed *before* the source, so editing code does not trigger a slow torch/transformers reinstall.
- `HF_HOME` points at a path mounted as a named volume (`hf-cache`) so multi-GB weights survive restarts instead of re-downloading. The compose healthcheck has a generous `start_period: 180s` because the first request lazily loads the VLM.
- **CPU by default;** the Dockerfile and compose file document the GPU swap (an `nvidia/cuda` base image, the CUDA build of torch, `--gpus all` /the `deploy` block, and `device: cuda` in config). **bitsandbytes** (4-bit QLoRA) only works on that CUDA path - it is intentionally Linux-only.

Local commands (via the Makefile)

```
make install      # editable install + runtime deps
make download     # download + split the dataset named in the config
make train        # python -m training.train --config $(CONFIG)
make evaluate     # python -m evaluation.evaluate --config $(CONFIG)
make compare      # python -m research.compare --config $(CONFIG)
make api          # uvicorn api.main:app --host 0.0.0.0 --port 8000 --reload
make app          # streamlit run app/streamlit_app.py
make docker-up    # docker compose -f docker/docker-compose.yml up
```

Any module can also be invoked directly, e.g. `python -m research.compare --config configs/default.yaml --adapter outputs/<run>/adapter`.

Colab quickstart (free T4 GPU)

The two Colab configs exist so you can see the *whole* loop end-to-end on a free 16 GB Turing T4. Both handle the same T4 constraints: **no bf16** (so `torch_dtype: float16`, `fp16: true`, `bf16: false`), tight VRAM (so `load_in_4bit: true` QLoRA + `gradient_checkpointing: true` + a small `max_pixels` budget), `attn_implementation: sdpa` (flash-attention needs Ampere+), and the tiny **CORD** receipts dataset so the download is seconds and training is fast.

	colab_t4.yaml	colab_t4_fast.yaml
Purpose	~10-15 min real-ish run	~5 min smoke test of the full loop
max_train_samples	400	64 (~32 optimizer steps)
num_train_epochs	1.0	2.0
metric_for_best_model	eval_f1 -> falls back to eval_loss	eval_loss

Typical flow: `python -m training.train --config configs/colab_t4_fast.yaml` , then `python -m research.compare --config configs/colab_t4_fast.yaml --adapter outputs/qwen2_5vl-cord-lora-colab-fast/adapter` .

How to read the results (the important part)

Why zero-shot base scores look low. *Zero-shot* means the base model answers with no fine-tuning on this dataset. It will often produce a *reasonable* answer phrased differently from the gold string - "\$42.00" vs the gold "42.00", or a full sentence where the dataset wants a bare value. Under EM/ANLS's exact-ish scoring, that surface mismatch costs points. So a low base ANLS usually reflects **format/verbosity mismatch, not incapability** - which is precisely what a short fine-tune fixes cheaply.

Base vs. fine-tuned - what "good" looks like. In `comparison.md` , expect EM/ANLS/Token-F1 to rise (^) after fine-tuning, driven largely by the model learning the dataset's answer *format*. Latency and peak memory should be roughly flat (a LoRA adapter adds a tiny number of parameters), and "Trainable LoRA params (M)" should be a small fraction of "Total params (M)" - that ratio *is* the parameter-efficiency headline of the whole project.

Interview Q: Your fine-tuned model's ANLS jumps from 0.18 to 0.55 after only ~30 steps on 64 samples. Is that "learning the task"?

A: Partly - but be honest about mechanism. On a tiny run, most of that gain is the model learning the dataset's *answer format/verbosity* (bare values, no currency symbols, the expected date format), which the exact-match-style metrics reward heavily, rather than acquiring new document-reading ability. It is a real, useful improvement and a valid demo, but I would not claim a general capability gain without a larger run and error analysis (the `plot_prediction_grid` and the per-sample rows in the report) to see *which* documents improved.

The extraction improvements are the second story. Even with the same weights, the schema-aware prompt + OCR grounding + defensive parsing + money normalization turn "messy, sometimes-hallucinated JSON" into clean, validated, gap-filled objects with a per-field hallucination flag. When reading an extraction result, check `grounded` (a `False` on a non-null value is a red flag for a reviewer) and `ocr_used` (whether grounding was even available).

Key insight: The most honest way to present this project's numbers is a triad: (1) the base-vs-fine-tuned quality delta with the parameter-efficiency ratio next to it, (2) the serving-cost profile (latency p50/p90, peak memory) proving it is shippable, and (3) qualitative extraction examples showing the prompt/grounding layer at work. Any one alone is easy to misread; together they tell the real story.

Cross-references: LoRA/PEFT mechanics and the `VisionDocModel` wrapper are in Ch. 7 and Ch. 12; the config system and dataset pipeline in Ch. 12; the metric definitions here recur in the interview-prep chapter (Ch. 15+). The training->evaluation->comparison ordering (`load_config -> seed -> device summary -> work -> report`) is intentionally identical across `train.py` , `evaluate.py` , and `compare.py` so their numbers are directly comparable.

14. Interview Prep I - Conceptual Question Bank (Zero to Advanced)

This chapter is a large, organized bank of interview questions with strong model answers. Each answer is written so you could say it aloud in an interview and sound competent. Questions rise in difficulty within each topic, and most topics include an "explain like I'm five" (ELI5) opener and a "senior deep dive" at the end. Everything here connects back to VisionDoc AI - a system that fine-tunes a Vision-Language Model (a model that reads both pictures and text) with LoRA to extract fields from invoices, receipts, forms, and IDs.

How to use this chapter: Cover the answer, try to say your own, then compare. In real interviews, aim for a crisp 20-40 second answer, then offer to go deeper. The model answers below give you both layers.

14.1 ML / Deep Learning Fundamentals

Interview Q (ELI5): What is machine learning?

A: Instead of writing rules by hand, we show a program lots of examples and let it find the pattern itself. For VisionDoc AI, we don't write "the total is the number after the word Total"; we show thousands of invoices with the correct answers, and the model learns the pattern of where totals live.

Interview Q: What's the difference between supervised, unsupervised, and self-supervised learning?

A: *Supervised* learning uses labeled examples - input plus the correct answer (an invoice image plus its extracted fields). *Unsupervised* learning finds structure with no labels (clustering similar documents). *Self-supervised* learning invents its own labels from raw data - for example, hiding part of the text and asking the model to predict it. Large language and vision-language models are pretrained self-supervised on huge unlabeled corpora, then fine-tuned supervised on our small labeled set.

Interview Q: What is a loss function, and why do we need one?

A: A loss function is a single number that measures how wrong the model's prediction is. Training is just the process of nudging the model's parameters to make that number smaller. For text generation we use *cross-entropy loss*, which is large when the model assigns low probability to the correct next token and near zero when it's confident and right.

Interview Q: What is gradient descent?

A: Imagine the loss as a hilly landscape and the model sitting somewhere on it. The *gradient* is the direction of steepest uphill. Gradient descent takes a small step in the opposite direction - downhill - to reduce loss. We repeat this millions of times. The step size is the *learning rate*.

Key insight: Backpropagation is not a different algorithm from gradient descent - it's just the efficient way to *compute* the gradients for every parameter in a deep network using the chain rule from calculus, working backward from the loss.

Interview Q: What are parameters vs. hyperparameters?

A: *Parameters* are the numbers the model learns - the weights inside the network. *Hyperparameters* are the knobs we set before training and don't learn: learning rate, batch size, number of epochs, LoRA rank. Tuning hyperparameters well is a big part of the job.

Interview Q: What is the bias-variance tradeoff?

A: *Bias* is error from a model too simple to capture the pattern (underfitting). *Variance* is error from a model so flexible it memorizes noise in the training data (overfitting). More capacity lowers bias but raises variance. The art is finding the sweet spot - often via regularization, more data, or early stopping.

Interview Q (senior deep dive): Why do deep networks generalize at all when they have enough parameters to memorize the training set?

A: This is the classic puzzle behind "double descent." Several factors help: gradient descent has an *implicit bias* toward simpler, lower-norm solutions; techniques like weight decay and dropout regularize explicitly; and overparameterized models trained with SGD tend to find flat minima that generalize better. For VisionDoc AI it matters less because we fine-tune a pretrained model - most generalization comes from the pretraining, and LoRA further limits how much we can overfit by constraining the number of trainable parameters.

14.2 Neural Networks & Training Mechanics

Interview Q (ELI5): What is a neuron in a neural network?

A: It takes several numbers in, multiplies each by a weight, adds them up, adds a bias, and passes the result through a simple "activation" function that decides how strongly to fire. Stack millions of these in layers and you can approximate very complex functions.

Interview Q: Why do we need activation functions?

A: Without a nonlinear activation, stacking layers is pointless - a chain of linear operations collapses into one linear operation. Nonlinearities like ReLU (outputs the input if positive, else zero) or GELU (a smooth version used in Transformers) let the network model curved, complex relationships.

Interview Q: What is an epoch, a batch, and an iteration?

A: A *batch* is a small group of examples processed together in one forward/backward pass. An *iteration* is one such pass (one weight update). An *epoch* is one full sweep through the entire training dataset. If you have 1,000 examples and a batch size of 10, one epoch is 100 iterations.

Interview Q: What is gradient accumulation and why use it?

A: It lets you simulate a large batch on limited GPU memory. Instead of updating weights every batch, you sum gradients over several small batches, then update once. If you can only fit batch size 2 but want an effective batch size of 16, you accumulate over 8 steps. VisionDoc AI relies on this because VLMs plus high-resolution images are memory-hungry.

Interview Q: What is the vanishing/exploding gradient problem?

A: In deep networks, gradients multiplied through many layers can shrink toward zero (vanishing - early layers stop learning) or blow up (exploding - training diverges). Fixes include normalization layers, residual connections (skip connections that let gradients flow directly), careful initialization, and gradient clipping (capping gradient magnitude).

Interview Q (senior deep dive): Explain LayerNorm vs. BatchNorm and why Transformers use LayerNorm.

A: *BatchNorm* normalizes each feature across the batch dimension, so it depends on batch statistics and behaves differently at train vs. inference. *LayerNorm* normalizes across the features of a single example, independent of other examples in the batch. Transformers process variable-length sequences where batch statistics are noisy and sequence positions shouldn't leak across examples, so LayerNorm (or RMSNorm, a cheaper variant used in many modern LLMs) is the natural fit.

14.3 Transformers & Attention

Interview Q (ELI5): What does "attention" mean in a Transformer?

A: When processing a word, the model looks back at all the other words and decides which ones matter most for understanding it. In "the invoice total is due," to understand "due" it pays attention to "total" and "invoice." Attention is a weighted lookup where the model learns what to focus on.

Interview Q: Explain query, key, and value.

A: Each token produces three vectors. The *query* is what this token is looking for; the *key* is what each token offers; the *value* is the actual content to retrieve. We score the query against every key (dot product), softmax those scores into weights, and take a weighted sum of the values. Analogy: a query is a search box, keys are document titles, values are the documents.

```
# Scaled dot-product attention (the heart of a Transformer)
scores = (Q @ K.transpose(-2, -1)) / sqrt(d_k) # similarity of each query to each key
weights = softmax(scores, dim=-1) # normalize into probabilities
output = weights @ V # weighted blend of values
```

Interview Q: Why divide by the square root of the key dimension?

A: For large dimensions, dot products grow large in magnitude, pushing softmax into saturated regions where gradients vanish. Dividing by $\sqrt{d_k}$ keeps the scores at a stable scale so training stays healthy.

Interview Q: What is multi-head attention?

A: Instead of one attention computation, we run several in parallel ("heads"), each with its own learned projections. One head might track subject-verb agreement, another might track layout position. Their outputs are concatenated and projected. It lets the model attend to different kinds of relationships simultaneously.

Interview Q: Why do Transformers need positional encodings?

A: Attention is order-agnostic - it treats the input as a set, not a sequence. Positional encodings inject "where am I" information. Classic Transformers used fixed sinusoidal encodings; modern LLMs like Qwen use *RoPE* (Rotary Position Embedding), which rotates the query/key vectors by an angle proportional to position, giving good handling of relative distances and longer contexts.

Interview Q: What's the difference between encoder-only, decoder-only, and encoder-decoder architectures?

A: *Encoder-only* (BERT) reads the whole input bidirectionally - good for classification/understanding. *Decoder-only* (GPT, Qwen2.5-VL) generates text left-to-right with causal masking - good for generation. *Encoder-decoder* (T5, Donut) encodes an input then decodes an output - natural for input-to-output transformation. VisionDoc AI's default model Qwen2.5-VL is decoder-only with a vision encoder bolted on; the swappable alternative, Donut, is encoder-decoder.

Interview Q: What is causal masking?

A: In a generation model, each position may only attend to earlier positions, never future ones - otherwise the model could "cheat" by peeking at the answer it's supposed to predict. We enforce this by masking out future positions (setting their attention scores to negative infinity before softmax).

Interview Q (senior deep dive): Attention is $O(n^2)$ in sequence length. Why does that matter for document AI, and what mitigates it?

A: Every token attends to every other, so cost grows quadratically with sequence length. Documents produce long sequences - high-resolution images become many visual tokens, plus the text prompt and answer. Mitigations include *FlashAttention* (an IO-aware exact-attention kernel that avoids materializing the full score matrix, saving memory and time), *KV caching* at inference, sliding-window or sparse attention, and - crucially for VLMS - controlling image resolution so we don't explode the visual token count.

14.4 LLMs & Decoding

Interview Q (ELI5): How does a language model generate text?

A: One token (word-piece) at a time. It reads everything so far, predicts a probability for every possible next token, picks one, appends it, and repeats. It's autocomplete on steroids.

Interview Q: What is a token, and what is tokenization?

A: A *token* is the unit the model actually processes - often a sub-word piece, not a whole word. "invoicing" might split into "invoic" + "ing." *Tokenization* converts raw text into these units using an algorithm like Byte-Pair Encoding (BPE). It matters for cost (you pay per token), context limits, and handling rare words.

Interview Q: Explain greedy, temperature, top-k, and top-p (nucleus) sampling.

A:

- *Greedy*: always take the highest-probability token - deterministic, can be repetitive.
- *Temperature*: scales the logits before softmax; low temperature sharpens toward the top choice, high temperature flattens for more diversity.
- *Top-k*: sample only from the k most likely tokens.
- *Top-p (nucleus)*: sample from the smallest set of tokens whose probabilities sum to p (e.g., 0.9), adapting the pool size to the model's confidence.

For VisionDoc AI's field extraction we want *deterministic, exact* output, so we use greedy or very low temperature - creativity is the enemy of correct extraction.

Interview Q: What is a context window?

A: The maximum number of tokens the model can consider at once (prompt plus generated output). Everything beyond it is invisible to the model. Long documents can exceed it, forcing truncation, chunking, or a model with a larger window.

Interview Q: What is KV caching?

A: During generation, the keys and values for already-processed tokens don't change. Instead of recomputing them each step, we cache them, so each new token only computes attention against the cache. It makes generation dramatically faster at the cost of memory.

Interview Q: What is hallucination and how do you reduce it in an extraction system?

A: Hallucination is when the model produces fluent but unfounded output - inventing a field value not in the document. In VisionDoc AI we reduce it by fine-tuning on grounded examples, using low/greedy decoding, forcing structured JSON output with schema validation, attaching *confidence scores*, and using *region highlighting* so a human can verify the answer against the actual pixels.

Interview Q (senior deep dive): Pretraining vs. instruction tuning vs. RLHF - where does VisionDoc AI's fine-tuning sit?

A: *Pretraining* teaches raw language/vision from massive unlabeled data. *Instruction tuning* (supervised fine-tuning) teaches the model to follow task-style prompts from labeled input-output pairs. *RLHF* (Reinforcement Learning from Human Feedback) further aligns behavior to human preferences via a reward model. VisionDoc AI is *supervised fine-tuning* of an already-instruction-tuned VLM on a narrow domain (documents) using LoRA - we're specializing, not aligning from scratch. RLHF is overkill here because extraction correctness is objectively checkable against ground truth.

14.5 Vision-Language Models (VLMs)

Interview Q (ELI5): What is a Vision-Language Model?

A: A model that understands pictures and words together. You show it an invoice image and ask "What's the total?" in plain English, and it answers using what it sees in the image.

Interview Q: How does a VLM combine an image and text into one model?

A: A *vision encoder* (often a Vision Transformer) turns the image into a sequence of embedding vectors - "visual tokens." A *projector/connector* maps those into the same space as text token embeddings. The language model then processes the visual tokens and text tokens as one combined sequence and generates an answer. It's like translating the picture into the model's internal "language" so text and image can be reasoned about together.

Interview Q: What is a Vision Transformer (ViT)?

A: It splits an image into fixed-size patches (say 16x16 pixels), flattens each patch, linearly projects it into an embedding, adds positional info, and feeds the patch sequence through a standard Transformer. It treats image patches like words in a sentence.

Interview Q: Why is Qwen2.5-VL well-suited to documents, and why is Donut a useful alternative?

A: Qwen2.5-VL handles *dynamic resolution* - it can process high-resolution document images without forcing everything to a tiny fixed size, which preserves small text like line-item amounts, and it's a strong general VLM good at reasoning and free-form VQA. Donut is *OCR-free* and encoder-decoder, purpose-built to read a document image and emit structured output directly, making it lightweight and fast for pure structured extraction. VisionDoc AI keeps them swappable so you can trade generality (Qwen) against a lean specialist (Donut).

Interview Q: What is OCR, and why is "OCR-free" notable?

A: OCR (Optical Character Recognition) extracts text strings from an image as a separate pre-step. An *OCR-free* model like Donut skips that pipeline and reads directly from pixels to structured output, avoiding cascading OCR errors and layout-reconstruction headaches - though it must learn character reading itself.

Interview Q: What is visual grounding / region highlighting?

A: Tying an answer back to the specific location in the image it came from - e.g., drawing a box around the total. It boosts trust and enables human verification. VisionDoc AI produces bounding regions so users can see *where* each extracted value was found.

Interview Q (senior deep dive): What are the main failure modes of VLMs on documents, and how do you defend against them?

A: Key failures: (1) *small-text loss* from downsampling high-res scans - mitigated by dynamic/tiled resolution; (2) *layout confusion* on multi-column or tabular forms - mitigated by layout-aware training data and positional cues; (3) *hallucinated values* for missing fields - mitigated by teaching the model to emit an explicit "not present" and by confidence thresholds; (4) *OCR-style character errors* (0 vs. O, 1 vs. l) - mitigated by domain fine-tuning and post-hoc validation (e.g., checksum/format checks on totals and IDs). Evaluation must use metrics tolerant of minor formatting (ANLS) alongside strict ones (Exact Match).

14.6 Fine-Tuning, LoRA & PEFT

Interview Q (ELI5): What is fine-tuning?

A: Taking a model that already knows a lot (from pretraining) and giving it a little extra training on *your* specific task, so it gets good at exactly what you need - reading invoices - without learning language from scratch.

Interview Q: What is PEFT?

A: PEFT (Parameter-Efficient Fine-Tuning) is a family of methods that adapt a large model by training only a tiny fraction of parameters, leaving the original weights frozen. It slashes memory, storage, and compute versus full fine-tuning, and makes it feasible to specialize a billion-parameter VLM on a single GPU.

Interview Q: Explain LoRA in one paragraph.

A: LoRA (Low-Rank Adaptation) freezes the original weight matrix W and learns a small additive update expressed as the product of two skinny matrices, B ($d \times r$) and A ($r \times k$), where the rank r is tiny (e.g., 8 or 16). The effective weight becomes $W + (\alpha/r)BA$. Because r is small, you train orders of magnitude fewer parameters. The insight is that the *change* needed to adapt a model to a task is low-rank - it lives in a small subspace.

```
# LoRA idea in pseudocode
frozen_W      # original pretrained weight, not updated
A = randn(r, k) # small, trainable
B = zeros(d, r) # small, trainable (init zero so training starts as a no-op)
h = x @ frozen_W.T + (alpha / r) * (x @ A.T @ B.T)
```

Interview Q: Why initialize B to zero?

A: So the LoRA update starts at exactly zero and the fine-tuned model initially behaves identically to the base model. Training then gently moves it, which is stable and avoids a disruptive jump at step zero.

Interview Q: What do the LoRA hyperparameters rank (r) and alpha do?

A: Rank r controls capacity - how expressive the adapter is; higher r means more parameters and more ability to fit, at higher memory/overfit risk. Alpha scales the update (the effective scale is α/r); it's like a learning-rate multiplier for the adapter. A common practice is to set alpha to r or $2r$ and tune from there. You also choose which modules to target (typically the attention projection matrices).

Interview Q: LoRA vs. full fine-tuning - tradeoffs?

A: Full fine-tuning updates every weight: maximum flexibility but huge memory, a full-size checkpoint per task, and higher overfitting risk on small data. LoRA trains ~0.1-1% of parameters: tiny adapters (a few MB) you can swap per customer, far less memory, and it doubles as regularization. The cost is a slight ceiling on adaptation for tasks that need to reshape the base model deeply - rarely an issue for domain specialization like document extraction.

Interview Q: What is catastrophic forgetting, and how does LoRA help?

A: Catastrophic forgetting is when fine-tuning on a new task erodes the model's original abilities. Because LoRA freezes the base weights and only adds a small side update, the original knowledge is preserved - and you can remove the adapter to recover the base model exactly.

Interview Q (senior deep dive): How would you serve many customers each with their own VisionDoc AI model, cost-effectively?

A: Keep one copy of the frozen base VLM in GPU memory and store a small LoRA adapter per customer. At request time, load or batch the relevant adapter on top of the shared base (adapter hot-swapping, or techniques like multi-LoRA serving that batch different adapters together). This gives per-customer specialization with the memory footprint of essentially one model, versus one full model per customer, which would be prohibitively expensive.

14.7 Quantization, Precision & Memory

Interview Q (ELI5): What is quantization?

A: Storing the model's numbers with fewer digits so the model takes less memory and runs faster - like rounding prices to the nearest dollar. You lose a little precision but save a lot of space.

Interview Q: What are FP32, FP16, and BF16?

A: *FP32* is 32-bit floating point - high precision, high memory. *FP16* is 16-bit - half the memory but a narrow numeric range that can overflow/underflow. *BF16* (bfloat16) is also 16-bit but keeps *FP32*'s wide exponent range at the cost of fractional precision, making it more stable for training.

Mixed-precision training uses 16-bit for speed while keeping a 32-bit master copy of weights for stability.

Interview Q: What is QLoRA?

A: QLoRA combines 4-bit quantization of the frozen base model with LoRA adapters trained in higher precision on top. It lets you fine-tune very large models on a single consumer GPU. Key pieces: *NF4* (a 4-bit data type tuned for normally-distributed weights), *double quantization* (quantizing the quantization constants too), and paged optimizers to survive memory spikes. VisionDoc AI uses bitsandbytes to enable this.

Interview Q: Where does GPU memory go during training?

A: Four buckets: (1) model weights, (2) gradients (same size as trainable weights), (3) optimizer states (Adam keeps two extra values per parameter - momentum and variance), and (4) activations stored for backprop (scales with batch size and sequence length). LoRA shrinks buckets 2 and 3 massively because only tiny adapters are trainable; gradient checkpointing trades compute to shrink bucket 4.

Interview Q: What is gradient checkpointing?

A: Instead of storing every intermediate activation for the backward pass, you store only a few and recompute the rest on the fly during backprop. It cuts activation memory substantially at the cost of extra compute (roughly one extra forward pass) - a common lever for fitting VLMs on limited hardware.

Interview Q (senior deep dive): Quantization-aware training vs. post-training quantization - when do you use each?

A: *Post-training quantization (PTQ)* quantizes an already-trained model with little or no retraining - fast and cheap, good when accuracy loss is tolerable, typical for deployment. *Quantization-aware training (QAT)* simulates quantization during training so the model learns to be robust to it, recovering more accuracy at low bit-widths but costing a full training run. QLoRA is a clever middle ground: the base stays quantized (PTQ-style) while trainable adapters learn in higher precision, getting most of QAT's accuracy for a fraction of the cost.

14.8 Document AI & Evaluation Metrics

Interview Q (ELI5): How do we know if the model is any good?

A: We hold back some documents the model never trained on, ask it to extract the fields, and compare its answers to the correct ones. If it usually matches, it's good.

Interview Q: Define Exact Match (EM), and its limitation.

A: EM is 1 if the prediction equals the ground truth exactly, else 0. It's strict and unforgiving - "\$1,000.00" vs. "1000" scores zero despite meaning the same. Great for well-normalized fields, too harsh for free-form text, so we pair it with softer metrics.

Interview Q: What is ANLS and why is it the standard for document VQA?

A: ANLS (Average Normalized Levenshtein Similarity) measures string similarity based on edit distance, normalized to 0-1, and averaged over questions - often with a threshold below which the score is set to zero. It rewards near-misses (a single OCR-style character error) instead of the all-or-nothing EM, which is why document-VQA benchmarks like DocVQA adopted it.

Interview Q: Explain precision, recall, and F1 for field extraction.

A: Treating each correctly extracted field as a "hit": *precision* = of the fields the model returned, what fraction were correct (penalizes hallucinated fields). *Recall* = of the fields that should have been found, what fraction were (penalizes misses). *F1* is their harmonic mean, balancing both. For invoices, low precision means made-up values; low recall means missed values - both matter.

Interview Q: What are BLEU and ROUGE, and when are they appropriate?

A: *BLEU* measures n-gram precision (how much of the prediction appears in the reference) - from machine translation. *ROUGE* emphasizes recall (how much of the reference is covered) - from summarization. They suit longer free-text answers (a descriptive VQA response) but are weak for short exact fields, where EM/ANLS are better.

Interview Q: How should you evaluate confidence scores?

A: A confidence score is only useful if it's *calibrated* - when the model says 90% confident, it should be right about 90% of the time. Check with a reliability diagram or Expected Calibration Error, and validate that thresholding on confidence actually improves precision (feeding low-confidence extractions to human review). VisionDoc AI's confidence outputs are what let a business auto-accept high-confidence fields and route the rest to a person.

Interview Q (senior deep dive): Design an evaluation suite for VisionDoc AI end to end.

A: (1) *Held-out test set* stratified by document type (invoice/receipt/form/ID) so per-type performance is visible. (2) *Per-field metrics*: EM and ANLS for each field, plus precision/recall/F1 over the whole field set to catch hallucinations and misses. (3) *Robustness slices*: low-quality scans, rotations, unusual layouts, unseen vendors. (4) *Calibration* of confidence scores. (5) *Grounding accuracy*: IoU (Intersection over Union) of predicted highlight boxes against annotated regions. (6) *Regression guardrails* in CI so a new adapter can't silently worsen a metric. (7) Track everything in Weights & Biases for run-to-run comparison. The headline number is per-field ANLS, but the slices and calibration are what make it trustworthy in production.

14.9 PyTorch & Practical Engineering

Interview Q (ELI5): What is PyTorch?

A: A Python library for building and training neural networks. It handles the math on GPUs and automatically computes the gradients you need to train, so you focus on the model, not the calculus.

Interview Q: What is a tensor?

A: A multi-dimensional array of numbers - the fundamental data type in PyTorch. A scalar is 0-D, a vector 1-D, a matrix 2-D, and a batch of RGB images is a 4-D tensor (batch, channels, height, width). Tensors live on CPU or GPU and track gradients.

Interview Q: What is autograd?

A: PyTorch's automatic differentiation engine. As you run operations, it builds a graph of what happened; calling `.backward()` on the loss walks that graph backward and fills in `.grad` for every parameter - implementing backprop for you.

Interview Q: Walk through a minimal PyTorch training loop.

A: For each batch: zero old gradients, run the forward pass to get predictions and loss, backpropagate, then step the optimizer.

```
for batch in dataloader:
    optimizer.zero_grad()      # clear last step's gradients
    out = model(**batch)       # forward pass
    loss = out.loss             # HF models return loss directly
    loss.backward()            # compute gradients (autograd)
    optimizer.step()           # update parameters
```

Gotcha: Forgetting `optimizer.zero_grad()` makes gradients accumulate across batches - which is exactly what gradient accumulation exploits on purpose, but a silent bug if unintended.

Interview Q: What's the difference between `model.train()` and `model.eval()` ?

A: They set the module's mode. `train()` enables dropout and uses batch statistics in normalization; `eval()` disables dropout and uses running statistics. Always switch to `eval()` (and wrap in `torch.no_grad()`) for validation and inference, or your metrics will be wrong and slower.

Interview Q: What do HuggingFace Transformers, Datasets, Accelerate, and PEFT each do?

A: *Transformers* provides the model architectures and the `Trainer`. *Datasets* efficiently loads and preprocesses data with memory-mapping. *Accelerate* abstracts device placement and multi-GPU/mixed-precision so the same code runs on a laptop or a cluster. *PEFT* implements LoRA/QLoRA adapters. VisionDoc AI's training package stitches these together.

Interview Q (senior deep dive): Your training loss looks fine but validation is garbage and training is slow. How do you debug?

A: Systematically: confirm `model.eval()` and `no_grad()` at validation (a common cause of "garbage"); verify train/val use the same preprocessing and no label leakage; overfit a single batch first - if the model can't drive that loss to near zero, there's a bug in the data pipeline or loss. For speed: check the GPU is actually being used and utilized (data loading may be the bottleneck - raise `num_workers`), enable mixed precision and gradient checkpointing appropriately, and profile to see whether time goes to compute, data, or CPU-GPU transfers.

14.10 Training Dynamics: Overfitting, Schedules, Early Stopping

Interview Q: What is overfitting and how do you detect it?

A: Overfitting is when the model memorizes the training set and fails on new data. You detect it when training loss keeps dropping while validation loss flattens then rises - the gap between the two curves widens. That divergence point is your signal to stop.

Interview Q: Name several ways to combat overfitting.

A: More/diverse data and data augmentation; regularization (weight decay, dropout); early stopping; reducing model capacity (or, with LoRA, a smaller rank); and cross-validation to tune. Because VisionDoc AI fine-tunes with LoRA on a frozen base, the parameter constraint itself is a strong regularizer.

Interview Q: What is a learning-rate schedule, and why use warmup?

A: The learning rate changes over training rather than staying fixed. *Warmup* ramps it up slowly for the first few hundred steps so early, noisy gradients don't destabilize the pretrained weights; then a *decay* (cosine or linear) gradually lowers it so the model settles into a good minimum. Cosine decay with warmup is a standard, robust choice for fine-tuning.

Interview Q: Why does learning rate matter so much?

A: Too high and training diverges or oscillates past good minima; too low and it crawls or gets stuck. It's usually the single most impactful hyperparameter. For LoRA fine-tuning the adapter tolerates a higher learning rate than full fine-tuning because the base is frozen.

Interview Q: What is early stopping?

A: Monitor a validation metric each epoch and stop training once it stops improving for a set number of checks ("patience"), keeping the best checkpoint. It saves compute and prevents overfitting past the optimal point.

Interview Q: What is the difference between SGD and Adam/AdamW?

A: *SGD* updates weights with a single global learning rate (optionally with momentum). *Adam* adapts a per-parameter learning rate using running estimates of the gradient's mean and variance, converging faster with less tuning. *AdamW* fixes Adam's weight-decay handling by decoupling it from the gradient update - it's the default optimizer for fine-tuning Transformers.

Interview Q (senior deep dive): How do you set batch size, and how does it interact with learning rate and generalization?

A: Batch size is bounded by memory, so we often fix an *effective* batch size via gradient accumulation. Larger batches give lower-variance gradient estimates and better hardware utilization but can converge to sharper minima that generalize slightly worse; they also usually need a proportionally higher learning rate (the "linear scaling rule") and more warmup. Smaller batches inject helpful gradient noise that can regularize. In practice for LoRA VLM fine-tuning you pick the largest batch that fits, reach a sensible effective batch via accumulation, then tune the learning rate around that.

14.11 Rapid-Fire Round (Say These in One Breath)

Question	One-line answer
Softmax does what?	Turns a vector of scores into a probability distribution that sums to 1.
Dropout does what?	Randomly zeroes activations during training to prevent co-adaptation and overfitting.
Embedding is what?	A learned dense vector representing a token/patch in a meaningful space.
Logits are what?	Raw pre-softmax model outputs.
Why freeze the base in LoRA?	To save memory, prevent forgetting, and keep tiny swappable adapters.
Epoch vs. step?	Epoch = one full data pass; step = one weight update.
Why greedy decoding for extraction?	Deterministic, exact output; no unwanted creativity.
Metric for document VQA?	ANLS (with EM as the strict complement).
What makes BF16 stable?	FP32-range exponent, so fewer overflows than FP16.
What is IoU used for here?	Scoring predicted highlight boxes against ground-truth regions.

Closing tip: Interviewers reward *structured* answers. State the definition, give one concrete VisionDoc AI example, then name a tradeoff or gotcha. That three-beat pattern - define, ground, qualify - signals real understanding rather than memorization. For scenario and system-design questions (serving, data pipelines, monitoring), see the applied interview chapter (Ch. 15).

15. Interview Prep II - Project Deep-Dive, System Design, War Stories, Glossary

This chapter turns everything you learned into interview-ready answers. It has four parts: (1) project-specific Q&A that shows deep ownership of *this* codebase, (2) "walk me through your project" scripts at three lengths, (3) system-design and behavioral prompts backed by real war stories from the repo, and (4) a one-page cheat sheet plus a complete glossary of every term used across the whole guide. For general ML fundamentals (bias/variance, overfitting, gradient descent) see Ch. 14; here we stay concrete and project-specific.

15.1 Project deep-dive Q&A

These are the questions an interviewer asks after you say "I fine-tuned a vision-language model." A **vision-language model (VLM)** is a neural network that takes *both* an image and text as input and produces text - think of it as a chatbot that can also see. VisionDoc AI fine-tunes one to read documents.

Interview Q: Why LoRA instead of full fine-tuning?

A: Full fine-tuning updates all ~3 billion weights of Qwen2.5-VL, which needs the optimizer to store gradients and momentum for every weight - roughly 4x the model size in VRAM, far beyond a single consumer GPU. **LoRA (Low-Rank Adaptation)** freezes the original weights and injects a tiny pair of trainable matrices (rank `r=16` in our `LoRAConfig`) beside each target layer. We train only ~0.1-1% of the parameters, the adapter file is a few megabytes instead of gigabytes, and we can keep one frozen base model in memory and hot-swap task-specific adapters. For document extraction that's not just cheaper - it's *better*, because a small adapter is far less likely to catastrophically forget the base model's general OCR-and-reasoning ability.

Interview Q: Why Qwen2.5-VL as the default backbone, with Donut as an alternative?

A: Qwen2.5-VL is an instruction-tuned, *causal* language model with a native dynamic-resolution vision encoder - it reads dense document images (small fonts, tables) without a fixed downscale, controlled by `min_pixels` / `max_pixels` in `ModelConfig`. Because it's instruction-tuned you can ask arbitrary questions ("what is the total?") in natural language, which is exactly what visual question answering needs. **Donut** is an OCR-free document-understanding model built on a **seq2seq** (encoder-decoder) BART backbone; it's smaller and great for fixed structured-parsing tasks like CORD receipts, but less flexible for free-form Q&A. Keeping Donut swappable lets us benchmark a specialist against the generalist.

Interview Q: Explain the frozen typed-config design. Why not just use dicts?

A: Every entry point - download, preprocess, train, evaluate, serve - is driven by one immutable config object loaded from YAML (`configs/config.py`). We use `@dataclass(frozen=True)` for three reasons the docstring spells out: misspelled keys fail *loudly at load time* instead of three hours into training; IDEs and type-checkers can autocomplete and verify field access; and defaults live in exactly one self-documenting place. `_from_dict` raises `ValueError` on any key that isn't a real field, so a typo like `learning_rat` is caught in milliseconds. `frozen=True` means the config can't be mutated mid-run, which kills a whole class of "why did the LR change?" bugs. A handful of high-churn values (device, adapter path, config path) can still be overridden by environment variables so the same YAML runs unchanged on a laptop, CI, or a multi-GPU box.

Interview Q: Why a swappable backbone, and how does the swap actually work?

A: `ModelConfig.model_type ( qwen2_5_vl | donut | florence2 )` selects an adapter class through a registry (`@register_model` in `models/base.py`). Each backbone subclasses `VisionDocModel` and implements only the model-specific tensorization hooks - how to build prompts, mask labels, and decode. The *shared* logic (the generation loop, confidence math, LoRA attach/save) lives once in the base class. So switching Qwen for Donut is a one-line YAML change with zero code edits, and any new backbone only implements a handful of hooks.

Interview Q: How is the confidence score computed?

A: During `generate` we pass `output_scores=True` , so HuggingFace returns the raw **logits** (unnormalized scores) for every generation step. `_token_logprobs_from_scores` applies `log_softmax` - **softmax** turns logits into a probability distribution over the vocabulary; taking its log gives us the log-probability the model assigned to the *token it actually chose*. `_sequence_confidence` then maps those per-token log-probs to a [0,1] score. The default `confidence_method="seq_prob"` returns $\exp(\text{mean log } p)$ - the **geometric mean** of the per-token probabilities, i.e. the typical per-token confidence. An alternative `"min_prob"` returns $\exp(\text{min log } p)$ - the least-confident token's probability, a conservative floor that's good for flagging risky extractions. This is a *self-reported* confidence, so it can be overconfident; it's a triage signal, not a calibrated probability.

Interview Q: How is field extraction grounded - i.e. how do you tell a real read from a hallucination?

A: Two independent mechanisms in `inference/extract.py` . (1) **OCR grounding on input:** if an `OCREngine` is available, we run OCR on the page and inject the raw text into the extraction prompt, giving the VLM a literal transcript to copy from rather than inventing values. (2) **OCR verification on output:** after the model returns fields, `_value_grounded` normalizes each extracted value and checks whether it actually appears as a substring in the page's OCR text. The result dict carries `grounded: {key: bool}` per field, and ungrounded values are logged as "possibly hallucinated." Separately, `Predictor.highlight_regions` uses the model's attention to draw absolute-pixel boxes over the image region an answer was localized to - visual grounding for a human reviewer.

Interview Q: How do you handle out-of-memory (OOM) errors?

A: Several levers, cheapest first. (1) Cap `max_pixels` in `ModelConfig` - the docstring calls this "the single most effective lever for controlling VRAM on document images," because attention cost grows with the number of image patches. (2) Use **QLoRA**: set `load_in_4bit=True` to load the frozen base in 4-bit via bitsandbytes, cutting base-model VRAM ~4x. (3) Reduce batch size and use gradient accumulation to keep the effective batch large. (4) Use **bf16** (`torch_dtype="bfloat16"`) to halve activation memory versus fp32. (5) Gradient checkpointing trades compute for memory. LoRA itself already removes the optimizer state for the frozen weights, which is the biggest single saving.

Interview Q: How do you evaluate this system?

A: Extraction and QA aren't graded like classification, so `evaluation/metrics.py` uses text-overlap metrics with SQuAD-style normalization (lowercase, strip punctuation/articles) so scores are comparable across mirrors. **Exact Match (EM)** is the strict fraction of predictions equal to a reference after normalization. **ANLS (Average Normalized Levenshtein Similarity)** is the DocVQA standard: $1 - \text{edit_distance}/\text{max_len}$, thresholded at 0.5, so "Acme Corp." vs "Acme Corp" scores near 1.0 instead of 0 - it tolerates minor OCR/spelling slips. We also report token-level **F1** (overlap of predicted vs reference words) and generation metrics **BLEU/ROUGE** for longer answers. See Ch. 9 for the metric derivations.

Interview Q: How would you improve it?

A: (1) Calibrate the confidence score against human-labeled correctness so a "0.8" means 80% right. (2) Add a retrieval/OCR-fusion step so long multi-page PDFs don't overflow context. (3) Train a lightweight task head (`modules_to_save`) for structured JSON schema enforcement instead of free-text parsing. (4) Active learning: route low-confidence, ungrounded extractions to human review and fold corrections back into training. (5) Distill the tuned adapter into a smaller model for cheaper serving, or export to ONNX (the repo has `models/onnx_export.py`) for latency.

15.2 "Walk me through your project" scripts

30-second version: "VisionDoc AI is a document-intelligence system. It fine-tunes an open-source vision-language model, Qwen2.5-VL, using LoRA so it runs on a single GPU, and teaches it to answer questions and extract structured fields - totals, dates, IDs - from invoices, receipts, forms, and IDs. It returns each answer with a confidence score and highlights the region of the page it came from, and it's served behind a FastAPI backend with a Streamlit demo UI."

2-minute version: Add the *how*. "The whole pipeline is driven by one frozen, typed config so nothing silently drifts. Preprocessing normalizes DocVQA/CORD-style datasets into a common `DocSample` schema. Training uses PEFT/LoRA - we freeze the 3-billion-parameter base and train tiny rank-16 adapter matrices on the attention and MLP projections of the language decoder, logging to Weights & Biases. The backbone is swappable through a registry: Qwen2.5-VL is the default generalist, Donut is a seq2seq specialist for structured parsing. At inference we compute a real confidence from the generation logits - the geometric-mean token probability - and we ground extractions two ways: we feed OCR text into the prompt and we verify each returned value actually appears in the page. Evaluation uses EM, ANLS, and F1. It's all containerized with Docker."

5-minute version: Structure it as Problem -> Approach -> Architecture -> Hard parts -> Results -> Future.

- *Problem:* Businesses drown in semi-structured documents; generic OCR gives you text but not answers, and closed APIs are expensive and un-tunable.
- *Approach:* Fine-tune an open VLM cheaply with LoRA so it's private, ownable, and adaptable per document type.
- *Architecture:* Walk the packages - `configs` (typed frozen config), `preprocessing` (schema + dataset loaders), `models` (registry of backbone adapters over a shared `VisionDocModel` base), `training` (PEFT loop), `evaluation` (EM/ANLS/F1), `inference` (confidence + grounding + region highlighting), `api / app` (FastAPI + Streamlit), `docker` .
- *Hard parts:* Tell one war story from 15.4 - the frozen-config annotations bug is the best because it shows Python depth.
- *Results:* Report your ANLS/EM numbers and adapter size vs base size.

- *Future*: Confidence calibration, multi-page retrieval, active learning.

Key insight: Interviewers grade *structure and ownership*, not vocabulary. Naming a real file, a real design decision, and a real bug you fixed beats reciting buzzwords every time.

15.3 System-design questions

Interview Q: How would you deploy and scale this in production?

A: Separate two workloads. *Model serving* runs on GPU nodes: load the frozen base once, attach the LoRA adapter, and keep the process warm - cold-loading a 3B model per request is fatal to latency. Put it behind a queue/autoscaler keyed on GPU utilization, not request count. *The API* (FastAPI) is stateless and scales horizontally on cheap CPU nodes; it validates uploads, runs OCR, and forwards to the model service. Multi-tenant per-document-type tuning is where LoRA shines: one base model in VRAM, many small adapters swapped by request - you can serve dozens of "models" from one GPU.

Interview Q: Batch vs real-time inference - when each?

A: Real-time for the interactive UI: one document, low latency, greedy decoding (`do_sample=False` , `num_beams=1`) which is deterministic and best for extraction. Batch for bulk back-office jobs (process last night's 50k invoices): use `inference/batch.py` , maximize throughput with larger batches and left-padded generation, and don't care about per-request latency. The key gotcha is that *correct batching differs by backbone* - see the padding war stories in 15.4.

Interview Q: What would you monitor?

A: Beyond infra (GPU/mem/latency/error rate): the **confidence distribution** (a sudden drop signals a new document layout the model hasn't seen), the **grounded-rate** (fraction of extracted fields verified in OCR - a drop means rising hallucination), input distribution drift (new languages, resolutions), and a human-review sample rate. Alert when low-confidence + ungrounded extractions spike together.

Interview Q: Describe the data / retraining loop.

A: Serve -> capture low-confidence and ungrounded predictions -> route to human labeling -> append corrections to the training set -> periodically retrain a fresh LoRA adapter -> evaluate on a frozen held-out set (guard against regressions, see Ch. 9) -> shadow-deploy the new adapter, compare ANLS/grounded-rate against the incumbent, then promote. Because adapters are tiny and hot-swappable, rollback is instant - just point `adapter_path` back at the old directory.

15.4 War stories (behavioral prompts)

Behavioral questions ("hardest bug", "something you learned") want a concrete story with a diagnosis and a fix. Each of these is real to this repo.

1. The frozen-dataclass nested-config bug (`from __future__ import annotations` breaks `is_dataclass`). *Symptom*: nested config sections like `model:` and `lora:` in the YAML were loading as raw `dict`s instead of `ModelConfig` / `LoRAConfig` objects, so `config.model.max_pixels` blew up with attribute errors. *Diagnosis*: the module starts with `from __future__ import annotations` , which stores every type annotation as a **string** at

class-definition time. So `dataclasses.Field.type` for the `model` field was the literal string `"ModelConfig"`, not the class - and `is_dataclass("ModelConfig")` is `False`, so the recursion in `_from_dict` never fired and the dict leaked through. *Fix:* `_resolve_field_types` calls `typing.get_type_hints(cls)`, which *evaluates* those annotation strings back into real class objects in the module namespace. Now `is_dataclass(f.type)` correctly returns `True` for nested sections and they're built into real dataclasses. *Lesson:* PEP 563 string annotations interact badly with any runtime that introspects `.type`; always resolve hints before reasoning about types at runtime. This is a fantastic story because it shows real Python internals depth.

2. Qwen batched-generation padding (right vs left). *Symptom:* batched generation produced garbage for shorter prompts in a batch. *Diagnosis:* Qwen2.5-VL is a *causal* LM. When you batch prompts of different lengths you must pad them to equal width. If you **right-pad**, the newly generated tokens start immediately after real content for the long prompt but *after pad tokens* for the short one, so the model continues from a PAD position - nonsense. *Training* uses right padding (loss is masked anyway), but *generation must use left padding* so all real tokens are flush against the newly generated ones. The code sets `padding_side = "left"` in `prepare_inference_inputs` and `"right"` for training. *Lesson:* padding side is not cosmetic; it's semantically required and it differs between training and generation.

3. Seq2seq Donut decode padding. *Symptom:* trying to batch Donut generation the same way as Qwen gave wrong first tokens. *Diagnosis:* Donut is encoder-decoder (BART). It conditions on a task/question prompt passed as `decoder_input_ids`, and those prefixes differ in length across a batch. Right-padding the decoder prefix makes `generate` emit the first token from a PAD position; left-padding shifts BART's *absolute* position embeddings - both wrong. There's no safe batched padding here. *Fix:* `DonutModel.generate` loops one sample at a time (batch-1, so no padding exists), trading throughput for correctness on this non-default backbone. *Lesson:* "just batch it" isn't universal - the correct batching strategy depends on the model architecture.

4. Adapter-path-as-repo-id confusion. *Symptom:* running evaluation right after kicking off training threw a cryptic `HFValidationError`. *Diagnosis:* PEFT's `PeftModel.from_pretrained` treats a path that doesn't exist locally as a **Hugging Face Hub repo id** and tries to download it - so a *missing local adapter directory* (because training hadn't finished writing it yet) surfaced as a confusing "invalid repo id" error. *Fix:* `load_adapter` in `models/base.py` validates up front - if the path doesn't exist and doesn't match a `namespace/name` regex, it raises a clear `FileNotFoundError` explaining that the adapter is written only after training completes; if the directory exists but lacks `adapter_config.json`, it says so explicitly. *Lesson:* wrap third-party errors that leak implementation details into actionable messages at your boundary.

5. `trust_remote_code` on new datasets. *Symptom:* some dataset mirrors errored on load; blindly setting `trust_remote_code=True` everywhere is a security smell (it executes arbitrary loading-script code). *Fix:* `preprocessing/datasets.py` loads *without* `trust_remote_code` first (standard Parquet/Arrow mirrors need nothing), and only *retries with it* when the specific error message indicates a script-based dataset. Least privilege by default, escalate on evidence. *Lesson:* `trust_remote_code=True` runs remote Python - treat it as a privilege you grant deliberately, not a default.

Gotcha: In all three padding stories the root cause is the same theme: a batch of variable-length sequences forces a padding decision, and the *correct* decision depends on (a) train vs generate and (b) causal vs seq2seq. If an interviewer probes padding, lead with that framework.

15.5 One-page cheat sheet

Fact	Value / detail
Default backbone	Qwen/Qwen2.5-VL-3B-Instruct (causal VLM), Donut/Florence-2 swappable
Adaptation method	LoRA/PEFT, <code>r=16</code> , <code>alpha=32</code> , <code>dropout=0.05</code> , <code>bias="none"</code>
LoRA targets	<code>q/k/v/o_proj</code> + <code>gate/up/down_proj</code> (LLM decoder only, not vision encoder)
Compute dtype	<code>bfloat16</code> default; QLoRA via <code>load_in_4bit</code> (bitsandbytes, CUDA)
VRAM lever #1	Cap <code>max_pixels</code> (<code>1280*28*28</code>) - attention scales with image patches
Decoding	Greedy by default (<code>do_sample=False</code> , <code>num_beams=1</code>) - deterministic for extraction
Confidence	<code>seq_prob = exp(mean token log-prob)</code> ; <code>alt_min_prob</code> = worst token
Grounding	OCR text injected into prompt + <code>_value_grounded</code> substring check -> <code>grounded{}</code>
Padding rule	Train = right; Qwen generate = left; Donut generate = per-sample (no padding)
Metrics	EM, ANLS (DocVQA, threshold 0.5), token-F1, BLEU/ROUGE
Config	Frozen typed dataclasses, strict unknown-key rejection, env overrides
Serving	FastAPI (stateless, CPU) + GPU model service; Streamlit demo; Docker

15.6 Comprehensive glossary

- **Adapter** - the small set of trainable weights (LoRA matrices) added to a frozen model; also the saved directory containing them (`adapter_config.json` + weights).
- **ANLS (Average Normalized Levenshtein Similarity)** - DocVQA metric; `1 - edit_distance/max_len`, thresholded (0.5) then averaged. Tolerant of minor text slips.
- **Attention** - mechanism letting a model weigh which input tokens/image-patches matter for each output; used here for region highlighting.
- **Autoregressive / causal LM** - a model that generates text one token at a time, each conditioned on all previous tokens (Qwen2.5-VL).
- **bf16 (bfloat16)** - a 16-bit float with the same exponent range as fp32 but fewer mantissa bits; halves memory versus fp32 with good training stability.
- **bitsandbytes** - library providing 4-bit/8-bit quantization that powers QLoRA.
- **BLEU / ROUGE** - n-gram-overlap metrics for generated text (precision-leaning / recall-leaning respectively).
- **Backbone** - the base pretrained model being adapted (Qwen2.5-VL, Donut, Florence-2).
- **Confidence score** - a [0,1] self-reported certainty derived from the generated tokens' probabilities; a triage signal, not calibrated truth.

- **Dataclass (frozen)** - a Python class auto-generating `__init__` /fields; `frozen=True` makes instances immutable. Used for the typed config.
- **Donut** - an OCR-free, seq2seq (encoder-decoder) document-understanding model; the swappable specialist backbone.
- **EM (Exact Match)** - fraction of predictions exactly equal to a reference after normalization.
- **F1** - harmonic mean of precision and recall; here computed over shared word tokens.
- **Fine-tuning** - continuing training of a pretrained model on task-specific data.
- **Geometric mean** - nth root of a product of n values; `exp(mean of logs)`. Used for `seq_prob` confidence.
- **Greedy decoding** - always picking the highest-probability next token; deterministic, no sampling.
- **Grounding** - tying a model output to evidence in the input (OCR substring match, or an image region).
- **HuggingFace** - the ecosystem (Transformers, Datasets, PEFT, Accelerate) the project is built on.
- **Instruction-tuned** - a model further trained to follow natural-language instructions/questions.
- **Levenshtein distance** - minimum single-character edits to turn one string into another; the core of ANLS.
- **Logits** - raw, unnormalized scores a model outputs before softmax.
- **Log-probability** - the natural log of a probability; summed/averaged for numerical stability in confidence math.
- **LoRA (Low-Rank Adaptation)** - freezes base weights and trains small low-rank matrices beside chosen layers; parameter-efficient fine-tuning.
- **maxpixels / minpixels** - bounds on Qwen's dynamic-resolution vision input; the main VRAM control for document images.
- **OCR (Optical Character Recognition)** - extracting machine text from an image; used here to ground prompts and verify outputs.
- **OOM (Out Of Memory)** - GPU VRAM exhaustion; mitigated via pixel caps, QLoRA, smaller batches, bf16, checkpointing.
- **ONNX** - an open model-exchange format for optimized cross-platform inference (`models/onnx_export.py`).
- **PEFT (Parameter-Efficient Fine-Tuning)** - the HuggingFace library / umbrella of methods (incl. LoRA) that tune few parameters.
- **Padding** - filling shorter sequences to a common length in a batch; side (left/right) is semantically important.
- **QLoRA** - LoRA on top of a 4-bit-quantized frozen base; drastically cuts VRAM.
- **Quantization** - storing weights in fewer bits (e.g. 4-bit) to save memory.
- **Rank (r)** - the inner dimension of LoRA's two matrices; controls adapter capacity (16 here).
- **Registry** - the `@register_model` name->class map enabling one-line backbone swaps.
- **seq2seq (sequence-to-sequence)** - encoder-decoder architecture (Donut/BART); contrast with causal decoder-only.
- **Softmax** - turns a vector of logits into a probability distribution summing to 1.
- **Streamlit** - Python framework for the interactive demo UI.
- **Token / tokenizer** - sub-word units a model reads/writes; the tokenizer maps text<->token-ids.
- **trustremotecode** - flag that lets a HuggingFace repo/dataset execute its own Python; a deliberate privilege, escalated only on evidence.
- **VLM (Vision-Language Model)** - a model consuming image + text and producing text.
- **ViT (Vision Transformer)** - transformer that splits an image into patches and attends over them; the vision encoder inside a VLM.
- **VRAM** - GPU memory; the binding constraint for training/serving large models.
- **Weights & Biases (W&B)** - experiment-tracking service logging metrics/artifacts during training.

Key insight: If you can define every term in this glossary in one plain sentence *and* connect five of them to a specific file in this repo, you will out-interview candidates who only memorized definitions. Depth of ownership, not breadth of jargon, is what gets the offer.